Pandas II

CONTENTS

Conditional Selection

Adding, Removing, and Modifying Columns

Useful Utility Functions

NumPy

.shape and .size

.describe()

.sample()

.value_counts()

Learning Outcomes

Last time, we introduced the `pandas` library as a toolkit for processing data. We learned the `DataFrame` and `Series` data structures, familiarized ourselves with the basic syntax for manipulating tabular data, and began writing our first lines of `pandas` code.

In this lecture, we'll start to dive into some advanced `pandas` syntax. You may find it helpful to follow along with a notebook of your own as we walk through these new pieces of code.

We'll start by loading the `babynames` dataset.

Click to see the code

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Conditional Selection

Conditional selection allows us to select a subset of rows in a `DataFrame` that satisfy some specified condition.

To understand how to use conditional selection, we must look at another possible input of the `.loc` and `[]` methods – a boolean array, which is simply an array or `Series` where each element is either `True` or `False`. This boolean array must have a length equal to the number of rows in the `DataFrame`. It will return all rows that correspond to a value of `True` in the array. We used a very similar technique when performing conditional extraction from a `Series` in the last lecture.

To see this in action, let's select all even-indexed rows in the first 10 rows of our `DataFrame`.

```
# Ask yourself: why is :9 is the correct slice to select the first 10 rows?
babynames_first_10_rows = babynames.loc[:9, :]

# Notice how we have exactly 10 elements in our boolean array argument
babynames_first_10_rows[[True, False, True, False, True, False, True, False, True, False]]
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101

We can perform a similar operation using `.loc`:

```
babynames_first_10_rows.loc[[True, False, True, False, True, False, True, False, True, Fal
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
2	CA	F	1910	Dorothy	220
4	CA	F	1910	Frances	134
6	CA	F	1910	Evelyn	126
8	CA	F	1910	Virginia	101

These techniques worked well in this example, but you can imagine how tedious it might be to list out `True` and `False`for every row in a larger `DataFrame`. To make things easier, we can instead provide a logical condition as an input to `.loc` or `[]` that returns a boolean array with the necessary length.

For example, to return all names associated with `F` sex:

```
# First, use a logical condition to generate a boolean array
logical_operator = (babynames["Sex"] == "F")

# Then, use this boolean array to filter the DataFrame
babynames[logical_operator].head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Recall from the previous lecture that `.head()` will return only the first few rows in the `DataFrame`. In reality, `babynames[logical operator]` contains as many rows as there are entries in the original `babynames DataFrame` with sex `"F"`.

Here, `logical_operator` evaluates to a `Series` of boolean values with length 407428.

Click to see the code

```
There are a total of 407428 values in 'logical_operator'
```

Rows starting at row 0 and ending at row 239536 evaluate to `True` and are thus returned in the `DataFrame`. Rows from 239537 onwards evaluate to `False` and are omitted from the output.

Click to see the code

```
The 0th item in this 'logical_operator' is: True
The 239536th item in this 'logical_operator' is: True
The 239537th item in this 'logical_operator' is: False
```

Passing a `Series` as an argument to `babynames[]` has the same effect as using a boolean array. In fact, the `[]` selection operator can take a boolean `Series`, array, and list as arguments. These three are used interchangeably throughout the course.

We can also use `.loc` to achieve similar results.

```
babynames.loc[babynames["Sex"] == "F"].head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Boolean conditions can be combined using various bitwise operators, allowing us to filter results by multiple conditions. In the table below, p and q are boolean arrays or `Series`.

Symbol	Usage	Meaning
~	~p	Returns negation of p
|	p | q	p OR q
&	p & q	p AND q
^	p ^ q	p XOR q (exclusive or)

When combining multiple conditions with logical operators, we surround each individual condition with a set of parenthesis `()`. This imposes an order of operations on `pandas` evaluating your logic and can avoid code erroring.

For example, if we want to return data on all names with sex `"F"` born before the year 2000, we can write:

```
babynames[(babynames["Sex"] == "F") & (babynames["Year"] < 2000)].head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163

	State	Sex	Year	Name	Count
4	CA	F	1910	Frances	134

Note that we're working with `Series`, so using `and` in place of `&`, or `or` in place `|` will error.

```
# This line of code will raise a ValueError
babynames[(babynames["Sex"] == "F") and (babynames["Year"] < 2000)].head()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
/var/folders/pp/4jn7bkwj0878mp_5c_2sllyhtrk6mx/T/ipykernel_54061/1860076842.py in ?()
      1 # This line of code will raise a ValueError
----> 2 babynames[(babynames["Sex"] == "F") and (babynames["Year"] < 2000)].head()

~/Documents/data100/course-notes/venv/lib/python3.11/site-packages/pandas/core/generic.py in
   1575     @final
   1576     def __nonzero__(self) -> NoReturn:
-> 1577         raise ValueError(
   1578             f"The truth value of a {type(self).__name__} is ambiguous. "
   1579             "Use a.empty, a.bool(), a.item(), a.any() or a.all()."
   1580         )

ValueError: The truth value of a Series is ambiguous. Use a.empty, a.bool(), a.item(), a.any
```

If we want to return data on all names with sex `"F"` *or* all born before the year 2000, we can write:

```
babynames[(babynames["Sex"] == "F") | (babynames["Year"] < 2000)].head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Boolean array selection is a useful tool, but can lead to overly verbose code for complex conditions. In the example below, our boolean condition is long enough to extend for several lines of code.

```
# Note: The parentheses surrounding the code make it possible to break the code on to mult
(
    babynames[(babynames["Name"] == "Bella") |
              (babynames["Name"] == "Alex") |
              (babynames["Name"] == "Ani") |
              (babynames["Name"] == "Lisa")]
).head()
```

	State	Sex	Year	Name	Count
6289	CA	F	1923	Bella	5
7512	CA	F	1925	Bella	8
12368	CA	F	1932	Lisa	5
14741	CA	F	1936	Lisa	8
17084	CA	F	1939	Lisa	5

Fortunately, `pandas` provides many alternative methods for constructing boolean filters.

The `.isin` function is one such example. This method evaluates if the values in a `Series` are contained in a different sequence (list, array, or `Series`) of values. In the cell below, we achieve equivalent results to the `DataFrame` above with far more concise code.

```
names = ["Bella", "Alex", "Narges", "Lisa"]
babynames["Name"].isin(names).head()
```

```
0    False
1    False
2    False
3    False
4    False
Name: Name, dtype: bool
```

```
babynames[babynames["Name"].isin(names)].head()
```

	State	Sex	Year	Name	Count
6289	CA	F	1923	Bella	5
7512	CA	F	1925	Bella	8
12368	CA	F	1932	Lisa	5
14741	CA	F	1936	Lisa	8
17084	CA	F	1939	Lisa	5

The function `str.startswith` can be used to define a filter based on string values in a `Series` object. It checks to see if string values in a `Series` start with a particular character.

```
# Identify whether names begin with the letter "N"
babynames["Name"].str.startswith("N").head()
```

```
0    False
1    False
2    False
3    False
4    False
Name: Name, dtype: bool
```

```
# Extracting names that begin with the letter "N"
babynames[babynames["Name"].str.startswith("N")].head()
```

	State	Sex	Year	Name	Count
76	CA	F	1910	Norma	23
83	CA	F	1910	Nellie	20
127	CA	F	1910	Nina	11
198	CA	F	1910	Nora	6
310	CA	F	1911	Nellie	23

Adding, Removing, and Modifying Columns

In many data science tasks, we may need to change the columns contained in our `DataFrame` in some way. Fortunately, the syntax to do so is fairly straightforward.

To add a new column to a `DataFrame`, we use a syntax similar to that used when accessing an existing column. Specify the name of the new column by writing `df["column"]`, then assign this to a `Series` or array containing the values that will populate this column.

```
# Create a Series of the length of each name.
babyname_lengths = babynames["Name"].str.len()

# Add a column named "name_lengths" that includes the length of each name
babynames["name_lengths"] = babyname_lengths
babynames.head(5)
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7

If we need to later modify an existing column, we can do so by referencing this column again with the syntax `df["column"]`, then re-assigning it to a new `Series` or array of the appropriate length.

```
# Modify the “name_lengths” column to be one less than its original value
babynames["name_lengths"] = babynames["name_lengths"] - 1
babynames.head()
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4

	State	Sex	Year	Name	Count	name_lengths
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6

We can rename a column using the `.rename()` method. It takes in a dictionary that maps old column names to their new ones.

```
# Rename “name_lengths” to “Length”
babynames = babynames.rename(columns={"name_lengths":"Length"})
babynames.head()
```

	State	Sex	Year	Name	Count	Length
0	CA	F	1910	Mary	295	3
1	CA	F	1910	Helen	239	4
2	CA	F	1910	Dorothy	220	6
3	CA	F	1910	Margaret	163	7
4	CA	F	1910	Frances	134	6

If we want to remove a column or row of a `DataFrame`, we can call the `.drop` (documentation) method. Use the `axis` parameter to specify whether a column or row should be dropped. Unless otherwise specified, `pandas` will assume that we are dropping a row by default.

```
# Drop our new "Length" column from the DataFrame
babynames = babynames.drop("Length", axis="columns")
babynames.head(5)
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220

	State	Sex	Year	Name	Count
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Notice that we *re-assigned* `babynames` to the result of `babynames.drop(...)`. This is a subtle but important point: `pandas` table operations **do not occur in-place**. Calling `df.drop(...)` will output a *copy* of `df` with the row/column of interest removed without modifying the original `df` table.

In other words, if we simply call:

```
# This creates a copy of `babynames` and removes the column "Name"...
babynames.drop("Name", axis="columns")

# ...but the original `babynames` is unchanged!
# Notice that the "Name" column is still present
babynames.head(5)
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Useful Utility Functions

`pandas` contains an extensive library of functions that can help shorten the process of setting and getting information from its data structures. In the following section, we will give overviews of each of the main utility functions that will help us in Data 100.

Discussing all functionality offered by `pandas` could take an entire semester! We will walk you through the most commonly-used functions and encourage you to explore and experiment on your own.

- `NumPy` and built-in function support
- `.shape`
- `.size`
- `.describe()`
- `.sample()`
- `.value_counts()`
- `.unique()`
- `.sort_values()`

The `pandas` documentation will be a valuable resource in Data 100 and beyond.

NumPy

`pandas` is designed to work well with `NumPy`, the framework for array computations you encountered in Data 8. Just about any `NumPy` function can be applied to `pandas` `DataFrame`s and `Series`.

```
# Pull out the number of babies named Yash each year
yash_count = babynames[babynames["Name"] == "Yash"]["Count"]
yash_count.head()
```

```
331824     8
334114     9
336390    11
338773    12
341387    10
Name: Count, dtype: int64
```

```
# Average number of babies named Yash each year
np.mean(yash_count)
```

```
17.142857142857142
```

```
# Max number of babies named Yash born in any one year
np.max(yash_count)
```

```
29
```

`.shape` and `.size`

`.shape` and `.size` are attributes of `Series` and `DataFrame`s that measure the "amount" of data stored in the structure. Calling `.shape` returns a tuple containing the number of rows and columns present in the `DataFrame` or `Series`. `.size` is used to find the total number of elements in a structure, equivalent to the number of rows times the number of columns.

Many functions strictly require the dimensions of the arguments along certain axes to match. Calling these dimension-finding functions is much faster than counting all of the items by hand.

```
# Return the shape of the DataFrame, in the format (num_rows, num_columns)
babynames.shape
```

```
(407428, 5)
```

```
# Return the size of the DataFrame, equal to num_rows * num_columns
babynames.size
```

```
2037140
```

`.describe()`

If many statistics are required from a `DataFrame` (minimum value, maximum value, mean value, etc.), then `.describe()` (documentation) can be used to compute all of them at once.

```
babynames.describe()
```

	Year	Count
count	407428.000000	407428.000000
mean	1985.733609	79.543456
std	27.007660	293.698654

	Year	Count
min	1910.000000	5.000000
25%	1969.000000	7.000000
50%	1992.000000	13.000000
75%	2008.000000	38.000000
max	2022.000000	8260.000000

A different set of statistics will be reported if `.describe()` is called on categorical data.

```
babynames["Sex"].describe()
```

```
count     407428
unique         2
top            F
freq      239537
Name: Sex, dtype: object
```

`.sample()`

As we will see later in the semester, random processes are at the heart of many data science techniques (for example, train-test splits, bootstrapping, and cross-validation). `.sample()` (documentation) lets us quickly select random entries (a row if called from a `DataFrame`, or a value if called from a `Series`).

By default, `.sample()` selects entries *without* replacement. Pass in the argument `replace=True` to sample with replacement.

```
# Sample a single row
babynames.sample()
```

Naturally, this can be chained with other methods and operators (`iloc`, etc.).

```
# Sample 5 random rows, and select all columns after column 2
babynames.sample(5).iloc[:, 2:]
```

	Year	Name	Count
206424	2014	Vida	43
19230	1942	Nanette	12
362321	2007	Coleman	14
208140	2014	Angelika	8
263843	1952	Wally	8

```
# Randomly sample 4 names from the year 2000, with replacement, and select all columns aft
babynames[babynames["Year"] == 2000].sample(4, replace = True).iloc[:, 2:]
```

`.value_counts()`

The `Series.value_counts()` (documentation) method counts the number of occurrence of each unique value in a `Series`. In other words, it *counts* the number of times each unique *value* appears. This is often useful for determining the most or least common entries in a `Series`.

In the example below, we can determine the name with the most years in which at least one person has taken that name by counting the number of times each name appears in the `"Name"` column of `babynames`. Note that the return value is also a `Series`.

```
babynames["Name"].value_counts().head()
```

```
Name
Jean         223
Francis      221
Guadalupe    218
Jessie       217
Marion       214
Name: count, dtype: int64
```

`.unique()`

If we have a `Series` with many repeated values, then `.unique()` (documentation) can be used to identify only the *unique* values. Here we return an array of all the names in `babynames`.

```
babynames["Name"].unique()
```

```
array(['Mary', 'Helen', 'Dorothy', ..., 'Zae', 'Zai', 'Zayvier'],
      dtype=object)
```

`.sort_values()`

Ordering a `DataFrame` can be useful for isolating extreme values. For example, the first 5 entries of a row sorted in descending order (that is, from highest to lowest) are the largest 5 values. `.sort_values` (documentation) allows us to order a `DataFrame` or `Series` by a specified column. We can choose to either receive the rows in `ascending` order (default) or `descending` order.

```
# Sort the "Count" column from highest to lowest
babynames.sort_values(by="Count", ascending=False).head()
```

	State	Sex	Year	Name	Count
268041	CA	M	1957	Michael	8260
267017	CA	M	1956	Michael	8258
317387	CA	M	1990	Michael	8246
281850	CA	M	1969	Michael	8245
283146	CA	M	1970	Michael	8196

Unlike when calling `.value_counts()` on a `DataFrame`, we do not need to explicitly specify the column used for sorting when calling `.value_counts()` on a `Series`. We can still specify the ordering paradigm – that is, whether values are sorted in ascending or descending order.

```
# Sort the "Name" Series alphabetically
babynames["Name"].sort_values(ascending=True).head()
```

```
366001      Aadan
384005      Aadan
369120      Aadan
398211    Aadarsh
370306      Aaden
Name: Name, dtype: object
```

Custom Sorts

Now, let's try to solve a sorting problem using different approaches. Assume we want to find the longest baby names and sort our data accordingly.

We'll start by loading the `babynames` dataset. Note that this dataset is filtered to only contain data from California.

Click to see the code

	State	Sex	Year	Name	Count
407418	CA	M	2022	Zach	5
407419	CA	M	2022	Zadkiel	5
407420	CA	M	2022	Zae	5
407421	CA	M	2022	Zai	5
407422	CA	M	2022	Zay	5
407423	CA	M	2022	Zayvier	5
407424	CA	M	2022	Zia	5
407425	CA	M	2022	Zora	5
407426	CA	M	2022	Zuriel	5
407427	CA	M	2022	Zylo	5

Approach 1: Create a Temporary Column

One method to do this is to first start by creating a column that contains the lengths of the names.

```
# Create a Series of the length of each name
babyname_lengths = babynames["Name"].str.len()

# Add a column named "name_lengths" that includes the length of each name
babynames["name_lengths"] = babyname_lengths
babynames.head(5)
```

	State	Sex	Year	Name	Count	name_lengths
0	CA	F	1910	Mary	295	4
1	CA	F	1910	Helen	239	5
2	CA	F	1910	Dorothy	220	7
3	CA	F	1910	Margaret	163	8
4	CA	F	1910	Frances	134	7

We can then sort the `DataFrame` by that column using `.sort_values()`:

```
# Sort by the temporary column
babynames = babynames.sort_values(by="name_lengths", ascending=False)
babynames.head(5)
```

	State	Sex	Year	Name	Count	name_lengths
334166	CA	M	1996	Franciscojavier	8	15
337301	CA	M	1997	Franciscojavier	5	15
339472	CA	M	1998	Franciscojavier	6	15
321792	CA	M	1991	Ryanchristopher	7	15
327358	CA	M	1993	Johnchristopher	5	15

Finally, we can drop the `name_length` column from `babynames` to prevent our table from getting cluttered.

```
# Drop the 'name_length' column
babynames = babynames.drop("name_lengths", axis='columns')
babynames.head(5)
```

	State	Sex	Year	Name	Count
334166	CA	M	1996	Franciscojavier	8
337301	CA	M	1997	Franciscojavier	5
339472	CA	M	1998	Franciscojavier	6
321792	CA	M	1991	Ryanchristopher	7
327358	CA	M	1993	Johnchristopher	5

Approach 2: Sorting using the `key` Argument

Another way to approach this is to use the `key` argument of `.sort_values()`. Here we can specify that we want to sort `"Name"` values by their length.

```
babynames.sort_values("Name", key=lambda x: x.str.len(), ascending=False).head()
```

	State	Sex	Year	Name	Count
334166	CA	M	1996	Franciscojavier	8
327472	CA	M	1993	Ryanchristopher	5
337301	CA	M	1997	Franciscojavier	5
337477	CA	M	1997	Ryanchristopher	5
312543	CA	M	1987	Franciscojavier	5

Approach 3: Sorting using the `map` Function

We can also use the `map` function on a `Series` to solve this. Say we want to sort the `babynames` table by the number of `"dr"`'s and `"ea"`'s in each `"Name"`. We'll define the function `dr_ea_count` to help us out.

```
# First, define a function to count the number of times "dr" or "ea" appear in each name
def dr_ea_count(string):
    return string.count('dr') + string.count('ea')

# Then, use `map` to apply `dr_ea_count` to each name in the "Name" column
babynames["dr_ea_count"] = babynames["Name"].map(dr_ea_count)

# Sort the DataFrame by the new "dr_ea_count" column so we can see our handiwork
babynames = babynames.sort_values(by="dr_ea_count", ascending=False)
babynames.head()
```

	State	Sex	Year	Name	Count	dr_ea_count
115957	CA	F	1990	Deandrea	5	3
101976	CA	F	1986	Deandrea	6	3
131029	CA	F	1994	Leandrea	5	3
108731	CA	F	1988	Deandrea	5	3
308131	CA	M	1985	Deandrea	6	3

We can drop the `dr_ea_count` once we're done using it to maintain a neat table.

```
# Drop the `dr_ea_count` column
babynames = babynames.drop("dr_ea_count", axis = 'columns')
babynames.head(5)
```

	State	Sex	Year	Name	Count
115957	CA	F	1990	Deandrea	5
101976	CA	F	1986	Deandrea	6
131029	CA	F	1994	Leandrea	5
108731	CA	F	1988	Deandrea	5
308131	CA	M	1985	Deandrea	6

Parting Note

Manipulating `DataFrames` is not a skill that is mastered in just one day. Due to the flexibility of `pandas`, there are many different ways to get from point A to point B. We recommend trying multiple different ways to solve the same problem to gain even more practice and reach that point of mastery sooner.

Next, we will start digging deeper into the mechanics behind grouping data.

← Data 100: Principles and Techniques of Data Science
Pandas I

Data 100: Principles and Techniques of Data Science
Pandas III →

Visualization II

CONTENTS

- Relationships Between Quantitative Variables
  - Scatter Plots
  - `lmplot` and `jointplot`
  - Hex plots
  - Contour Plots
- Transformations
  - Linearization and Applying Transformations
  - Additional Remarks
- Visualization Theory
  - Information Channels
    - Encodings in Rugplots
    - Multi-Dimensional Encodings
  - Harnessing X|Y

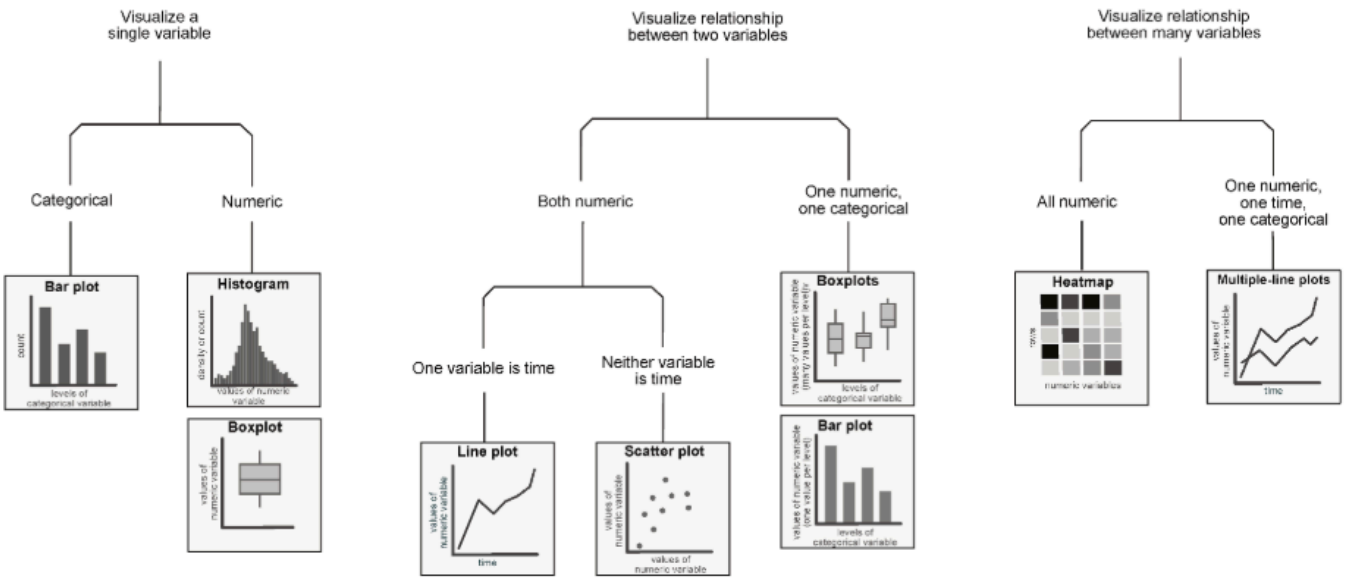
Learning Outcomes

- Using transformations to analyze the relationship between two variables.
- Evaluating the quality of a visualization based on visualization theory concepts.

Relationships Between Quantitative Variables

Up until now, we've discussed how to visualize single-variable distributions. Going beyond this, we want to understand the **relationship** between variables (i.e., bivariate or multivariate distributions)

The following diagram is a high-level overview of univariate, bivariate, and multivariate distributions.



Scatter Plots

Scatter plots are one of the most useful tools in representing the relationship between **pairs** of quantitative variables. They are particularly important in gauging the strength, or correlation, of the relationship between variables. Knowledge of these relationships can then motivate decisions in our modeling process.

In `matplotlib`, we use the function `plt.scatter` to generate a scatter plot. Notice that, in the example below (using data from the World Bank dataset `wb`) unlike our examples of plotting single-

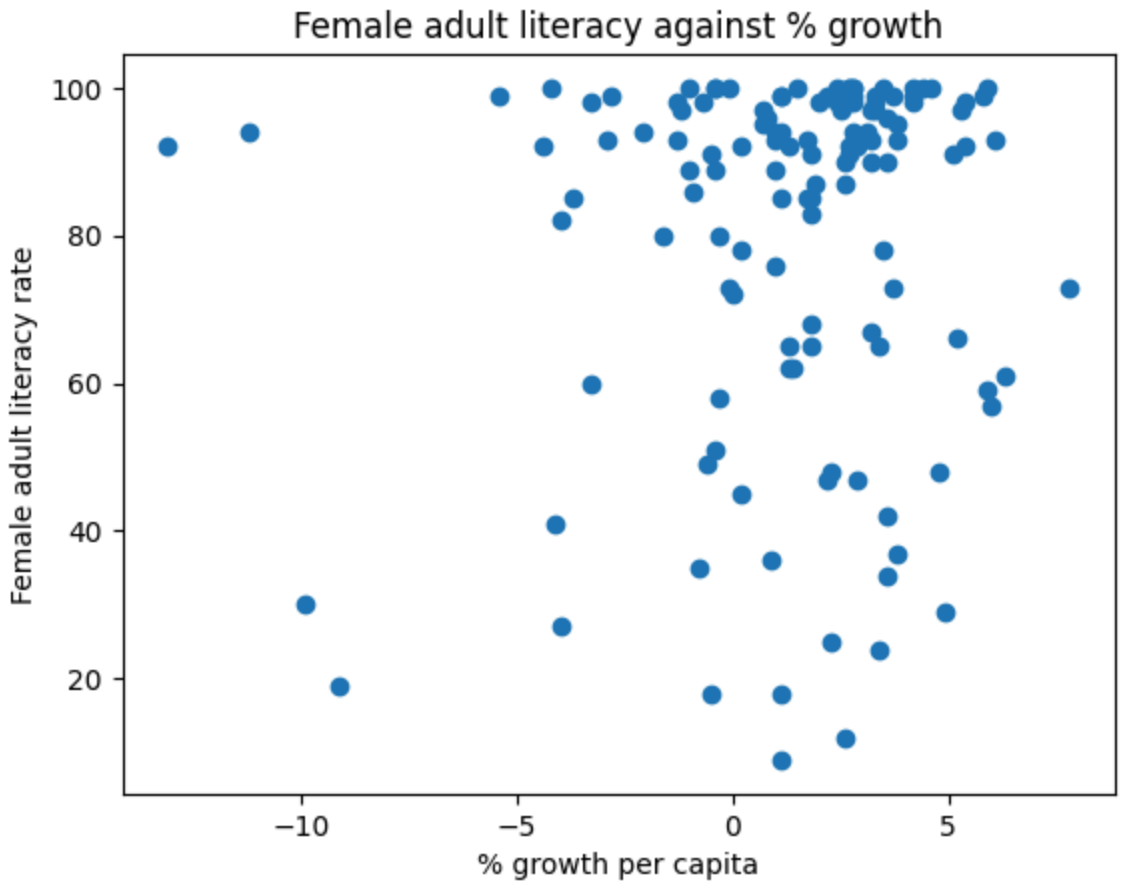
variable distributions, now we specify sequences of values to be plotted along the x-axis *and* the y-axis.

Click to see the code

	Continent	Country	Primary completion rate: Male: % of relevant age group: 2015	Primary completion rate: Female: % of relevant age group: 2015	Lower secondary completion rate: Male: % of relevant age group: 2015	Lower secondary completion rate: Female: % of relevant age group: 2015	Youth literacy rate: Male: % of ages 15-24: 2005-14	Youth literacy rate: Female: % of ages 15-24: 2005-14	Adult literacy rate Male % ages 15 and older 2005-14
0	Africa	Algeria	106.0	105.0	68.0	85.0	96.0	92.0	83.0
1	Africa	Angola	NaN	NaN	NaN	NaN	79.0	67.0	82.0
2	Africa	Benin	83.0	73.0	50.0	37.0	55.0	31.0	41.0
3	Africa	Botswana	98.0	101.0	86.0	87.0	96.0	99.0	87.0
5	Africa	Burundi	58.0	66.0	35.0	30.0	90.0	88.0	89.0

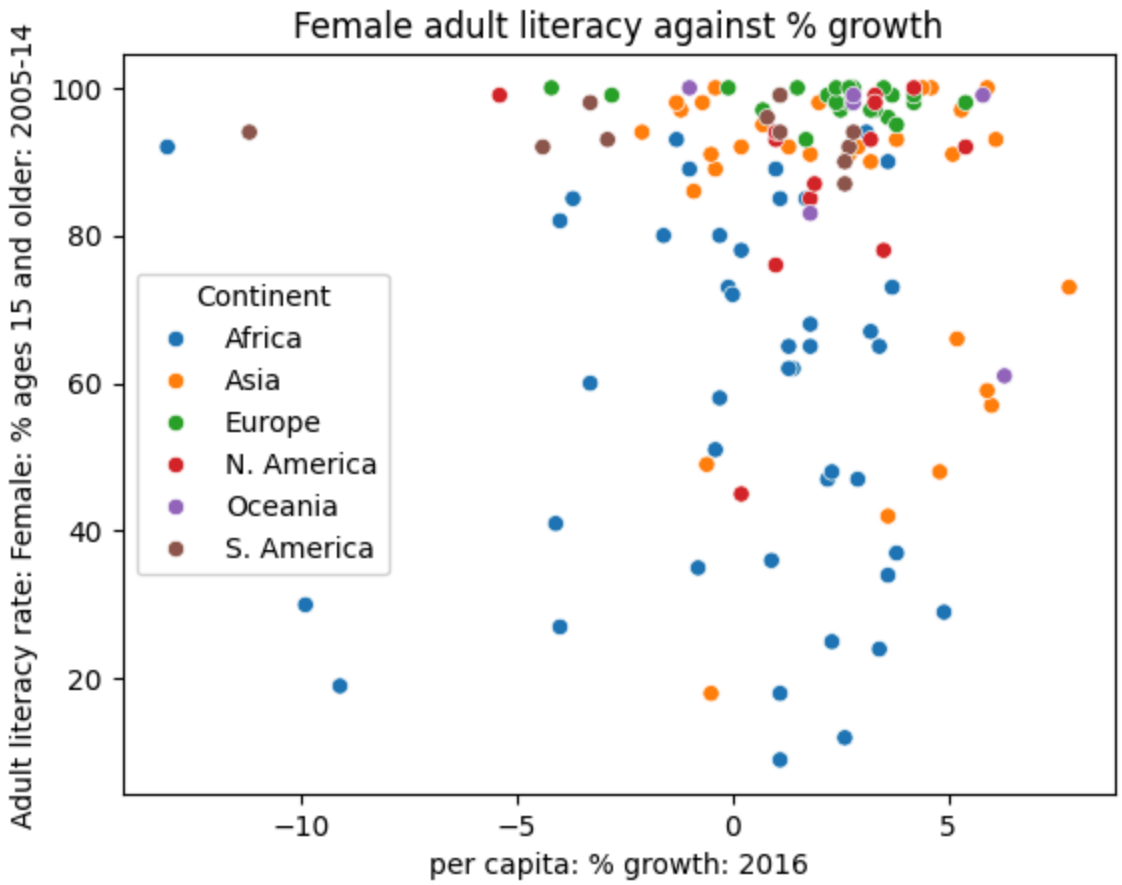
5 rows × 47 columns

```
plt.scatter(wb["per capita: % growth: 2016"], \
            wb['Adult literacy rate: Female: % ages 15 and older: 2005-14'])

plt.xlabel("% growth per capita")
plt.ylabel("Female adult literacy rate")
plt.title("Female adult literacy against % growth");
```



In `seaborn`, we call the function `sns.scatterplot`. We use the `x` and `y` parameters to indicate the values to be plotted along the x and y axes, respectively. By using the `hue` parameter, we can specify a third variable to be used for coloring each scatter point.

```
sns.scatterplot(data = wb, x = "per capita: % growth: 2016", \
                y = "Adult literacy rate: Female: % ages 15 and older: 2005-14",
                hue = "Continent")

plt.title("Female adult literacy against % growth");
```



Overplotting

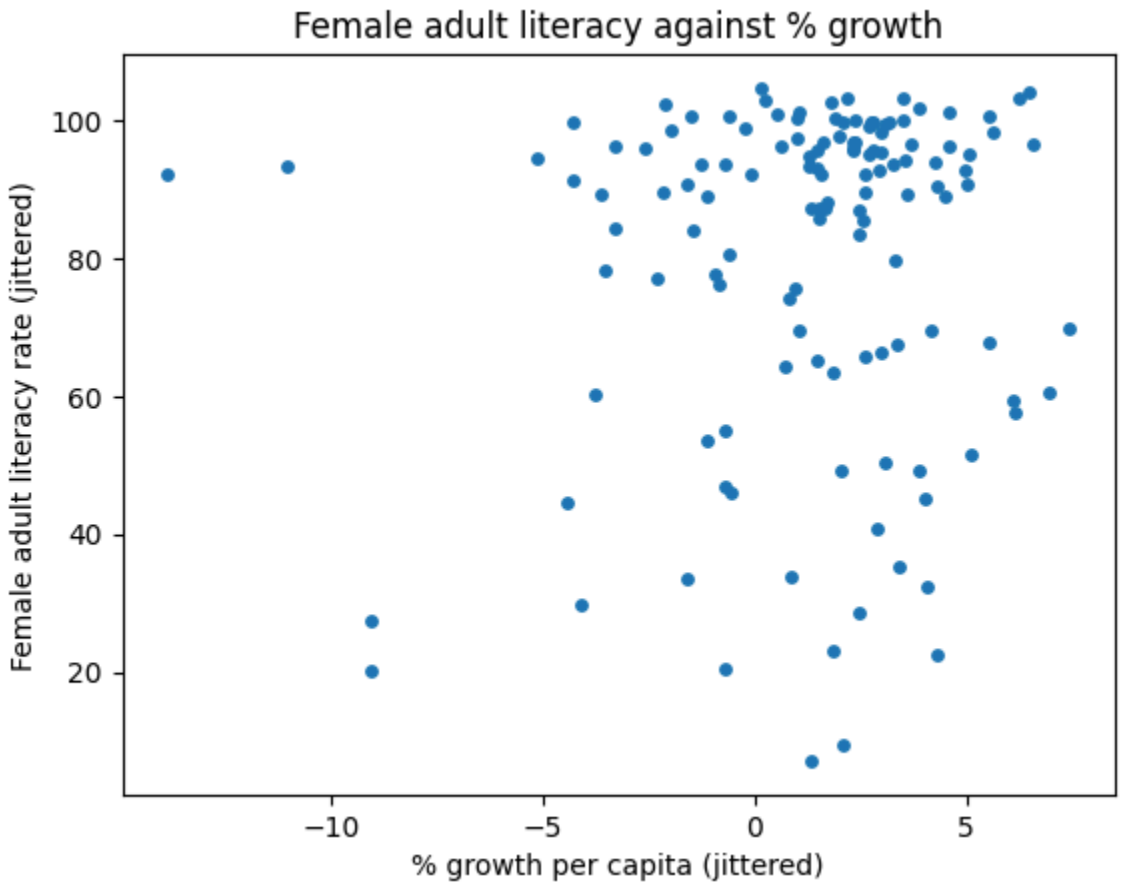
Although the plots above communicate the general relationship between the two plotted variables, they both suffer a major limitation – **overplotting**. Overplotting occurs when scatter points with similar values are stacked on top of one another, making it difficult to see the number of scatter points actually plotted in the visualization. Notice how in the upper righthand region of the plots, we cannot easily tell just how many points have been plotted. This makes our visualizations difficult to interpret.

We have a few methods to help reduce overplotting:

- Decreasing the size of the scatter point markers can improve readability. We do this by setting a new value to the size parameter, `s`, of `plt.scatter` or `sns.scatterplot`.
- **Jittering** is the process of adding a small amount of random noise to all x and y values to slightly shift the position of each datapoint. By randomly shifting all the data by some small distance, we can discern individual points more clearly without modifying the major trends of the original dataset.

In the cell below, we first jitter the data using `np.random.uniform`, then re-plot it with smaller markers. The resulting plot is much easier to interpret.

```
# Setting a seed ensures that we produce the same plot each time
# This means that the course notes will not change each time you access them
np.random.seed(150)

# This call to np.random.uniform generates random numbers between -1 and 1
# We add these random numbers to the original x data to jitter it slightly
x_noise = np.random.uniform(-1, 1, len(wb))
jittered_x = wb["per capita: % growth: 2016"] + x_noise

# Repeat for y data
y_noise = np.random.uniform(-5, 5, len(wb))
jittered_y = wb["Adult literacy rate: Female: % ages 15 and older: 2005-14"] + y_noise

# Setting the size parameter `s` changes the size of each point
plt.scatter(jittered_x, jittered_y, s=15)

plt.xlabel("% growth per capita (jittered)")
plt.ylabel("Female adult literacy rate (jittered)")
plt.title("Female adult literacy against % growth");
```

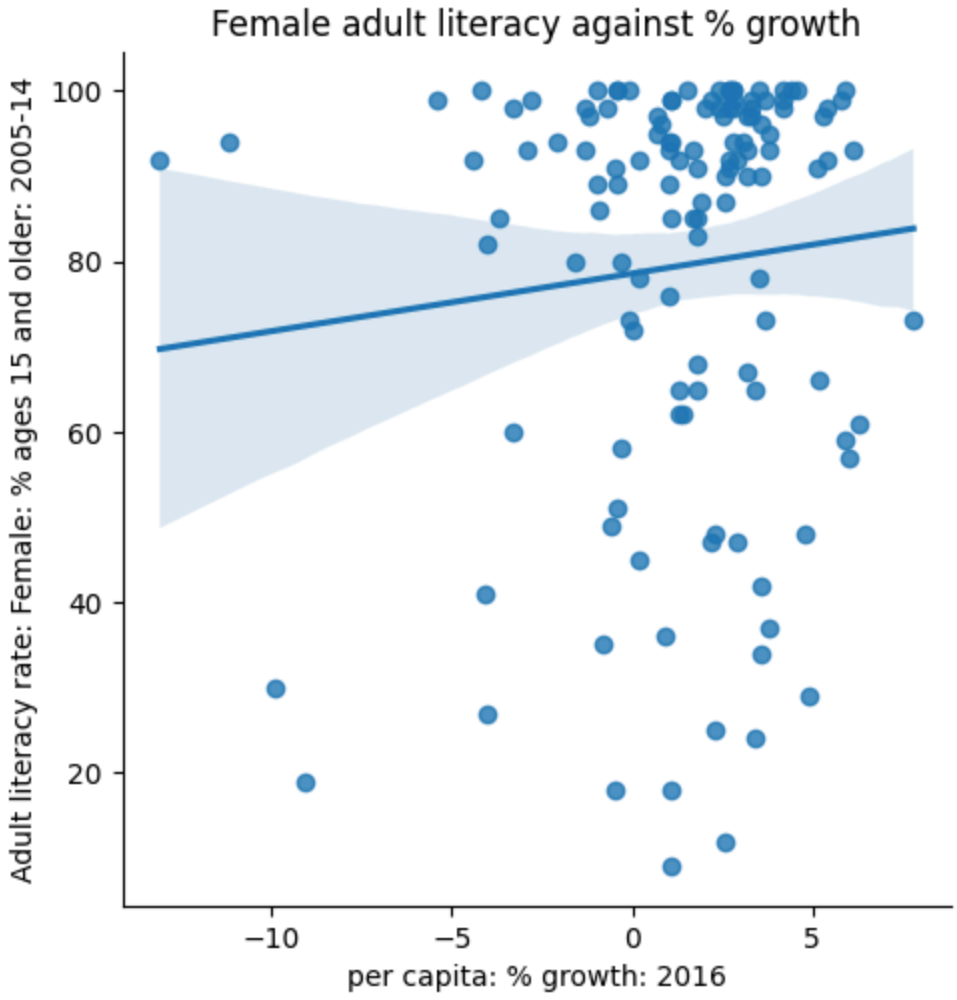


`lmplot` and `jointplot`

`seaborn` also includes several built-in functions for creating more sophisticated scatter plots. Two of the most commonly used examples are `sns.lmplot` and `sns.jointplot`.

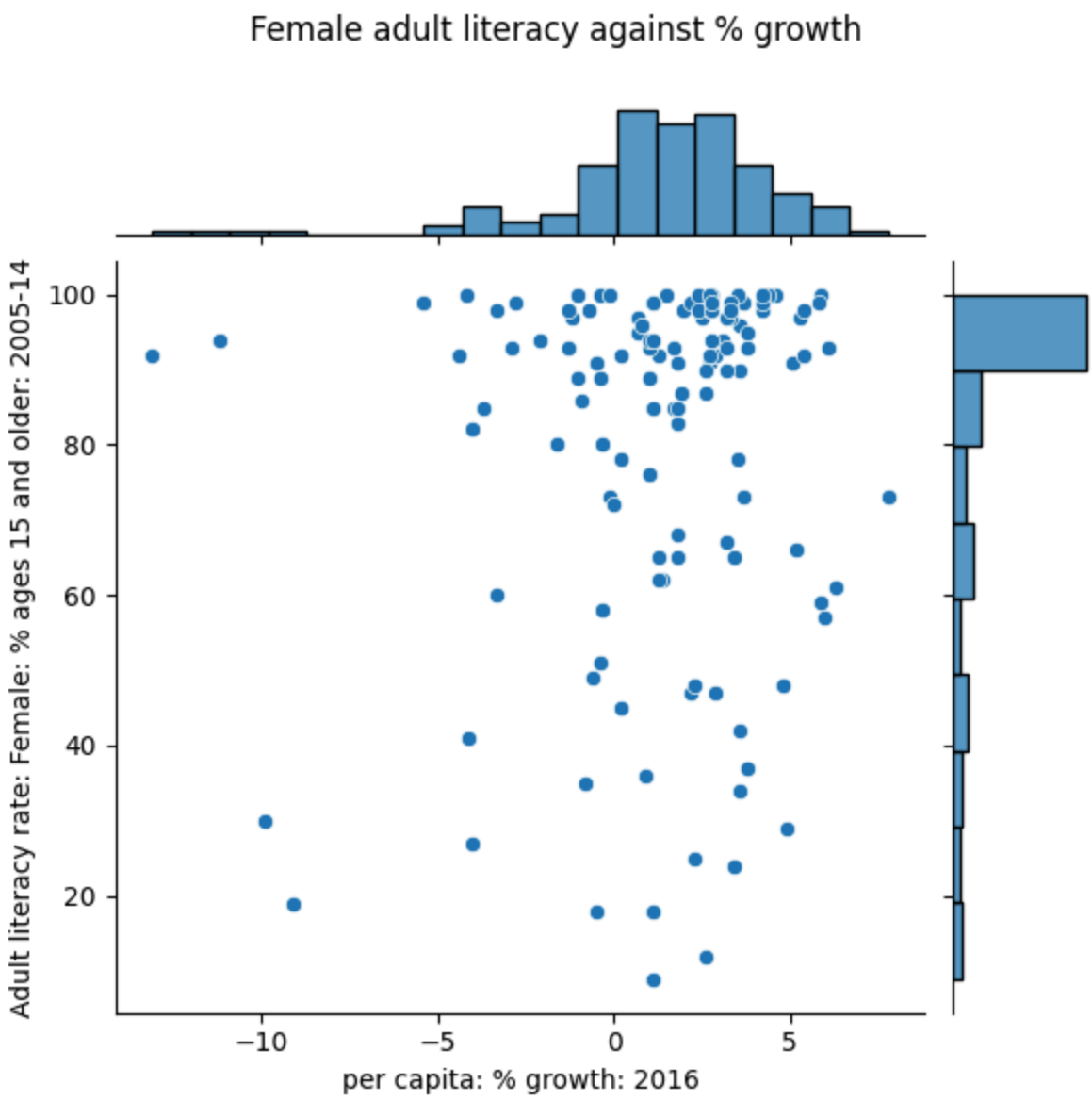
`sns.lmplot` plots both a scatter plot *and* a linear regression line, all in one function call. We'll discuss linear regression in a few lectures.

```
sns.lmplot(data = wb, x = "per capita: % growth: 2016", \
           y = "Adult literacy rate: Female: % ages 15 and older: 2005-14")

plt.title("Female adult literacy against % growth");
```



`sns.jointplot` creates a visualization with three components: a scatter plot, a histogram of the distribution of x values, and a histogram of the distribution of y values.

```
sns.jointplot(data = wb, x = "per capita: % growth: 2016", \
              y = "Adult literacy rate: Female: % ages 15 and older: 2005-14")

# plt.suptitle allows us to shift the title up so it does not overlap with the histogram
plt.suptitle("Female adult literacy against % growth")
plt.subplots_adjust(top=0.9);
```



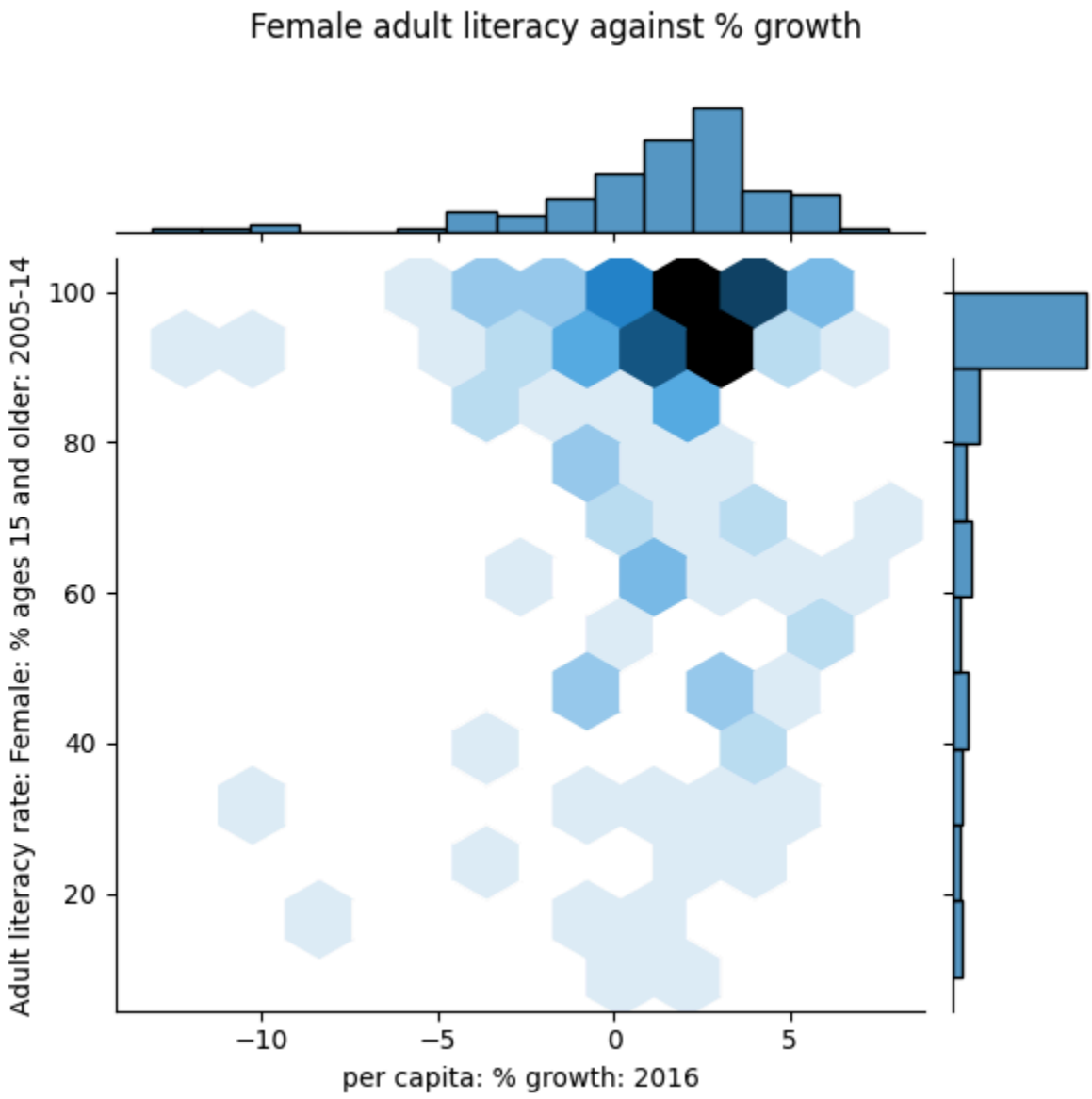
Hex plots

For datasets with a very large number of datapoints, jittering is unlikely to fully resolve the issue of overplotting. In these cases, we can attempt to visualize our data by its *density*, rather than displaying each individual datapoint.

Hex plots can be thought of as two-dimensional histograms that show the joint distribution between two variables. This is particularly useful when working with very dense data. In a hex plot, the x-y plane is binned into hexagons. Hexagons that are darker in color indicate a greater density of data – that is, there are more data points that lie in the region enclosed by the hexagon.

We can generate a hex plot using `sns.jointplot` modified with the `kind` parameter.

```
sns.jointplot(data = wb, x = "per capita: % growth: 2016", \
              y = "Adult literacy rate: Female: % ages 15 and older: 2005-14", \
              kind = "hex")

# plt.suptitle allows us to shift the title up so it does not overlap with the histogram
plt.suptitle("Female adult literacy against % growth")
plt.subplots_adjust(top=0.9);
```

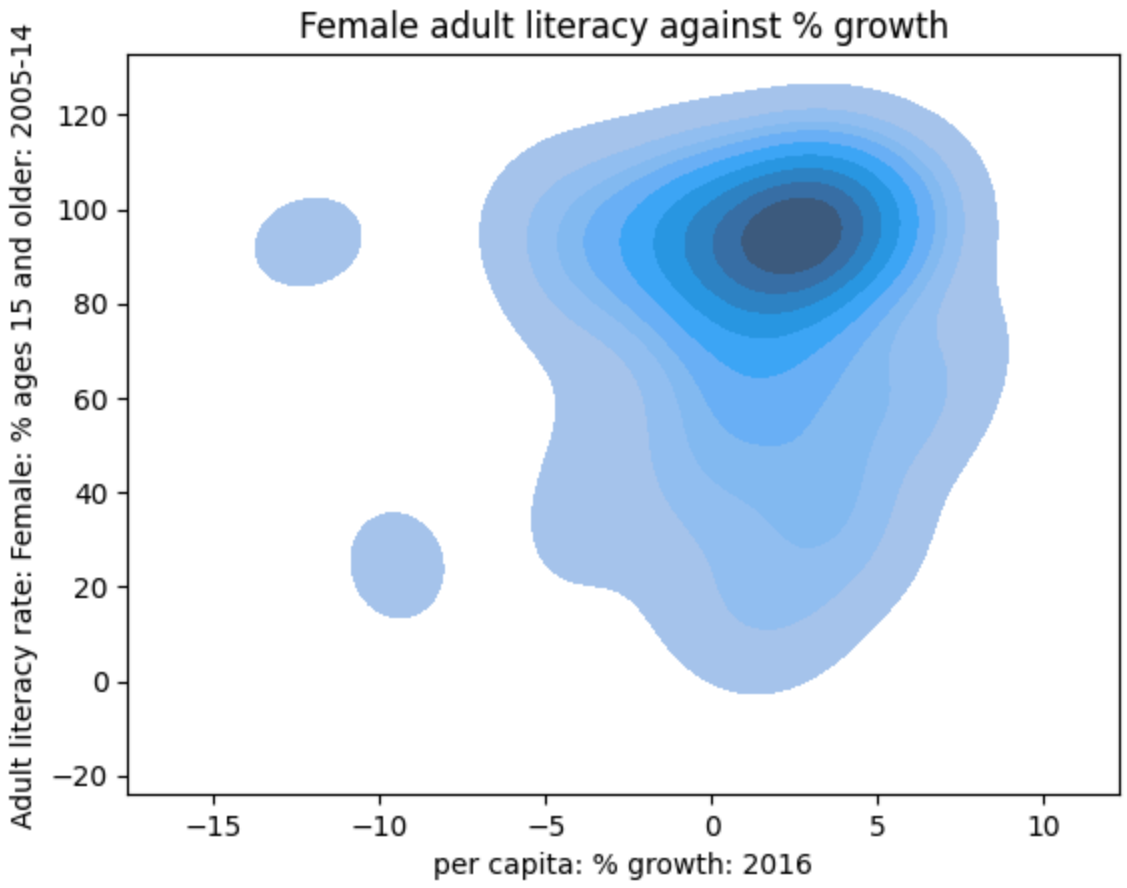


Contour Plots

Contour plots are an alternative way of plotting the joint distribution of two variables. You can think of them as the 2-dimensional versions of KDE plots. A contour plot can be interpreted in a similar way to a topographic map. Each contour line represents an area that has the same *density* of datapoints throughout the region. Contours marked with darker colors contain more datapoints (a higher density) in that region.

`sns.kdeplot` will generate a contour plot if we specify both x and y data.

```
sns.kdeplot(data = wb, x = "per capita: % growth: 2016", \
            y = "Adult literacy rate: Female: % ages 15 and older: 2005-14", \
            fill = True)

plt.title("Female adult literacy against % growth");
```



Transformations

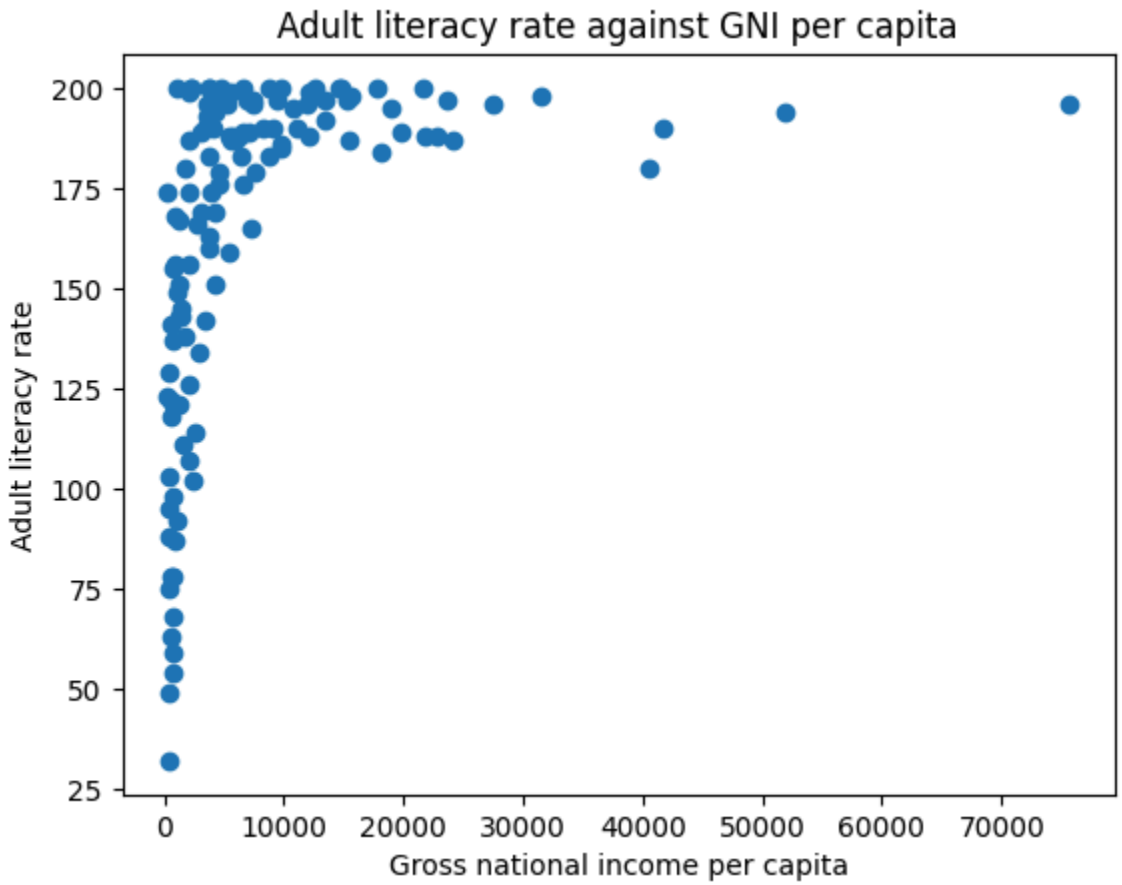
We have now covered visualizations in great depth, looking into various forms of visualizations, plotting libraries, and high-level theory.

Much of this was done to uncover insights in data, which will prove necessary when we begin building models of data later in the course. A strong graphical correlation between two variables hints at an underlying relationship that we may want to study in greater detail. However, relying on visual relationships alone is limiting - not all plots show association. The presence of outliers and other statistical anomalies makes it hard to interpret data.

Transformations are the process of manipulating data to find significant relationships between variables. These are often found by applying mathematical functions to variables that "transform" their range of possible values and highlight some previously hidden associations between data.

To see why we may want to transform data, consider the following plot of adult literacy rates against gross national income.

Click to see the code



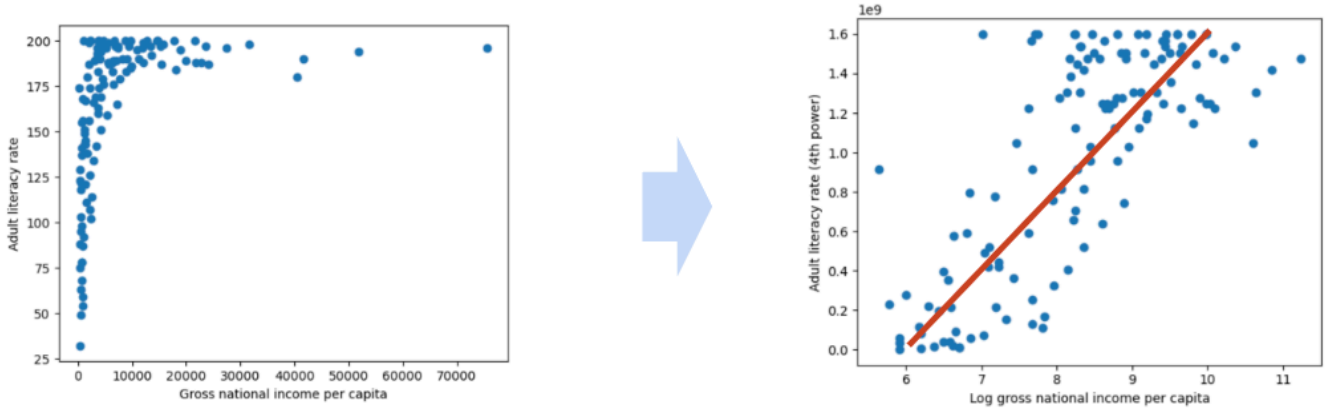
This plot is difficult to interpret for two reasons:

- The data shown in the visualization appears almost "smushed" – it is heavily concentrated in the upper lefthand region of the plot. Even if we jittered the dataset, we likely would not be able to fully assess all datapoints in that area.
- It is hard to generalize a clear relationship between the two plotted variables. While adult literacy rate appears to share some positive relationship with gross national income, we are not able to describe the specifics of this trend in much detail.

A transformation would allow us to visualize this data more clearly, which, in turn, would enable us to describe the underlying relationship between our variables of interest.

We will most commonly apply a transformation to **linearize a relationship** between variables. If we find a transformation to make a scatter plot of two variables linear, we can "backtrack" to find the exact relationship between the variables. This helps us in two major ways. Firstly, linear relationships are particularly simple to interpret – we have an intuitive sense of what the slope and intercept of a linear trend represent, and how they can help us understand the relationship between two variables. Secondly, linear relationships are the backbone of linear models. We will begin exploring linear modeling in great detail next week. As we'll soon see, linear models become much more effective when we are working with linearized data.

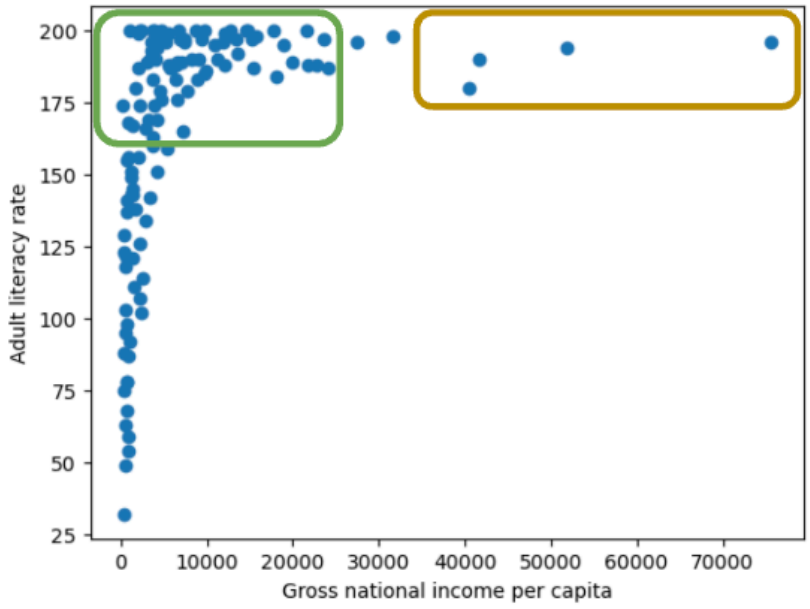
In the remainder of this note, we will discuss how to linearize a dataset to produce the result below. Notice that the resulting plot displays a rough linear relationship between the values plotted on the x and y axes.



Linearization and Applying Transformations

To linearize a relationship, begin by asking yourself: what makes the data non-linear? It is helpful to repeat this question for each variable in your visualization.

Let's start by considering the gross national income variable in our plot above. Looking at the y values in the scatter plot, we can see that many large y values are all clumped together, compressing the vertical axis. The scale of the horizontal axis is also being distorted by the few large outlying x values on the right.



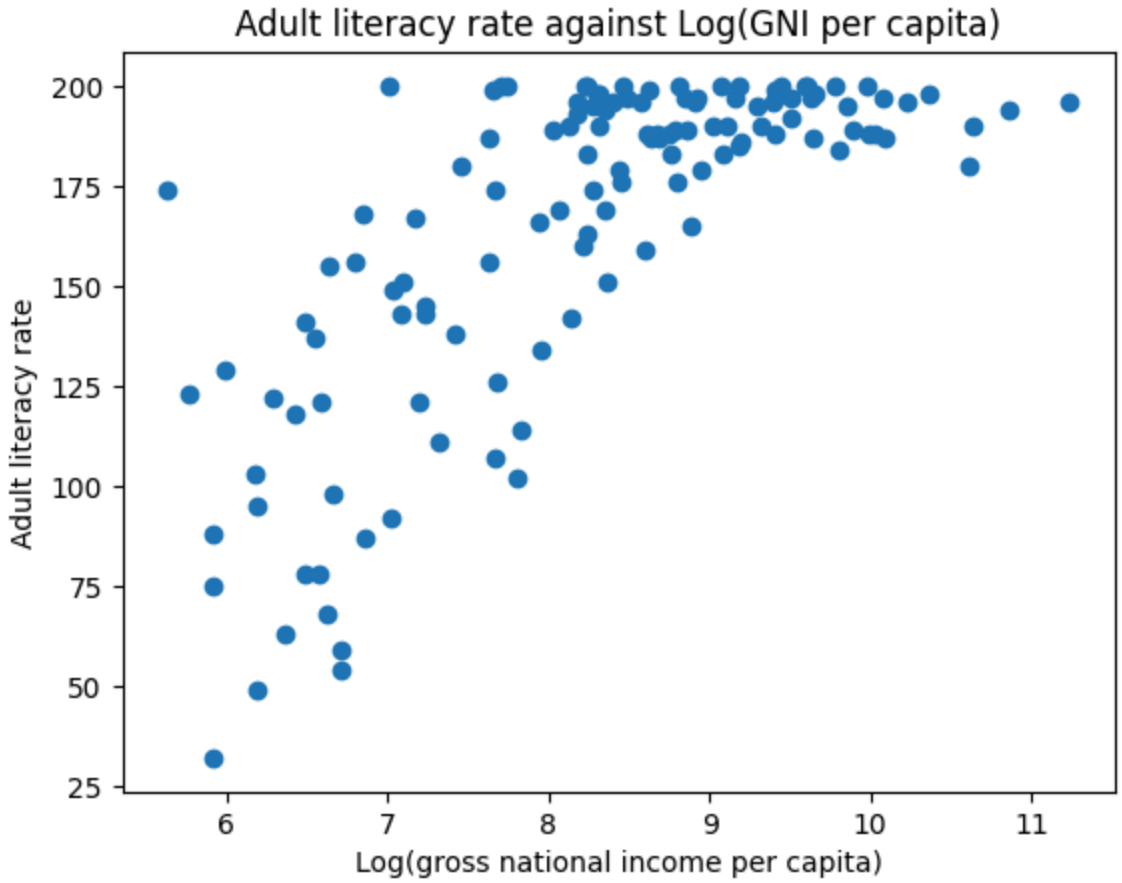
If we decreased the size of these outliers relative to the bulk of the data, we could reduce the distortion of the horizontal axis. How can we do this? We need a transformation that will:

- Decrease the magnitude of large x values by a significant amount.
- Not drastically change the magnitude of small x values.

One function that produces this result is the **log transformation**. When we take the logarithm of a large number, the original number will decrease in magnitude dramatically. Conversely, when we take the logarithm of a small number, the original number does not change its value by as significant of an amount (to illustrate this, consider the difference between $\log(100) = 4.61$ and $\log(10) = 2.3$).

In Data 100 (and most upper-division STEM classes), $\log$ is used to refer to the natural logarithm with base e.

```
# np.log takes the logarithm of an array or Series
plt.scatter(np.log(df["inc"]), df["lit"])

plt.xlabel("Log(gross national income per capita)")
plt.ylabel("Adult literacy rate")
plt.title("Adult literacy rate against Log(GNI per capita)");
```



After taking the logarithm of our x values, our plot appears much more balanced in its horizontal scale. We no longer have many datapoints clumped on one end and a few outliers out at extreme values.

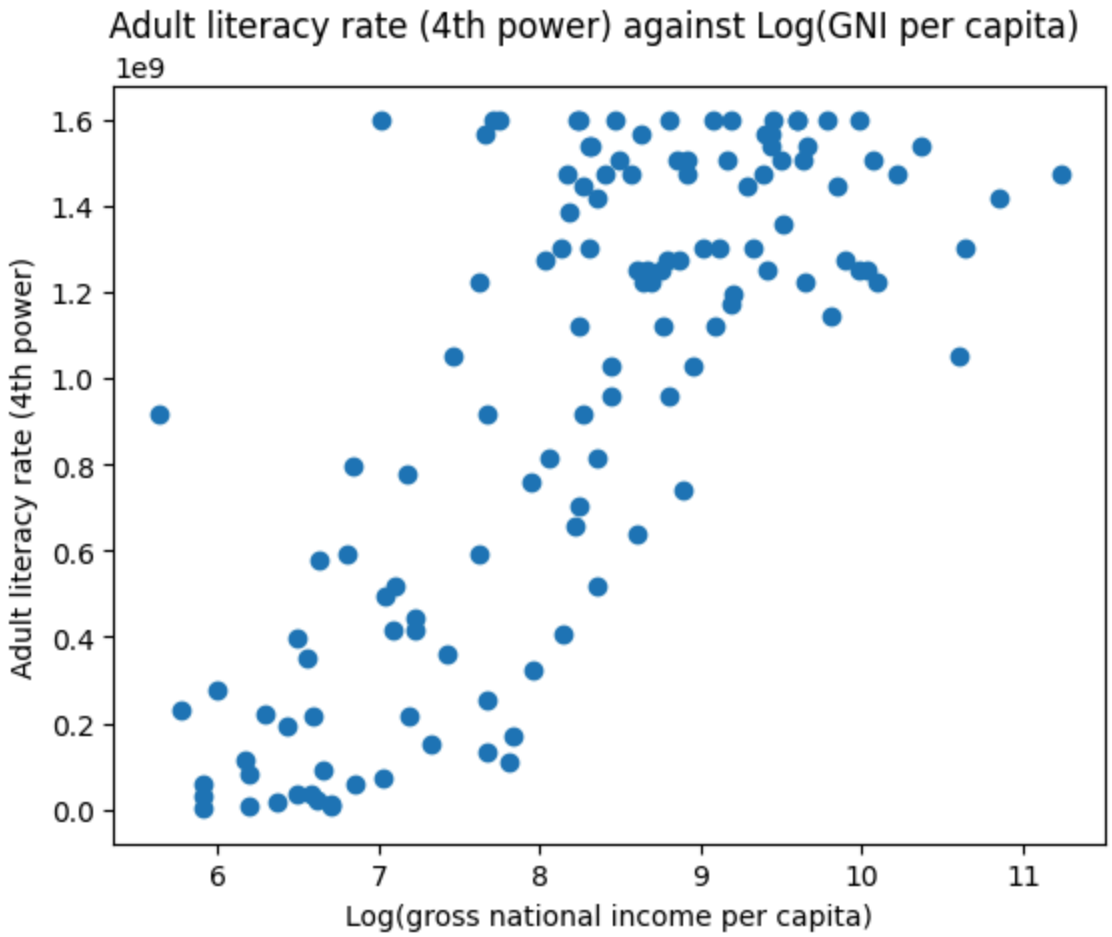
Let's repeat this reasoning for the y values. Considering only the vertical axis of the plot, notice how there are many datapoints concentrated at large y values. Only a few datapoints lie at smaller values of y.

If we were to "spread out" these large values of y more, we would no longer see the dense concentration in one region of the y-axis. We need a transformation that will:

- Increase the magnitude of large values of y so these datapoints are distributed more broadly on the vertical scale,
- Not substantially alter the scaling of small values of y (we do not want to drastically modify the lower end of the y axis, which is already distributed evenly on the vertical scale).

In this case, it is helpful to apply a **power transformation** – that is, raise our y values to a power. Let's try raising our adult literacy rate values to the power of 4. Large values raised to the power of 4 will increase in magnitude proportionally much more than small values raised to the power of 4 (consider the difference between $2^4 = 16$ and $200^4 = 1600000000$).

```
# Apply a log transformation to the x values and a power transformation to the y values
plt.scatter(np.log(df["inc"]), df["lit"]**4)

plt.xlabel("Log(gross national income per capita)")
plt.ylabel("Adult literacy rate (4th power)")
plt.suptitle("Adult literacy rate (4th power) against Log(GNI per capita)")
plt.subplots_adjust(top=0.9);
```



Our scatter plot is looking a lot better! Now, we are plotting the log of our original x values on the horizontal axis, and the 4th power of our original y values on the vertical axis. We start to see an approximate *linear* relationship between our transformed variables.

What can we take away from this? We now know that the log of gross national income and adult literacy to the power of 4 are roughly linearly related. If we denote the original, untransformed gross national income values as x and the original adult literacy rate values as y, we can use the standard form of a linear fit to express this relationship:

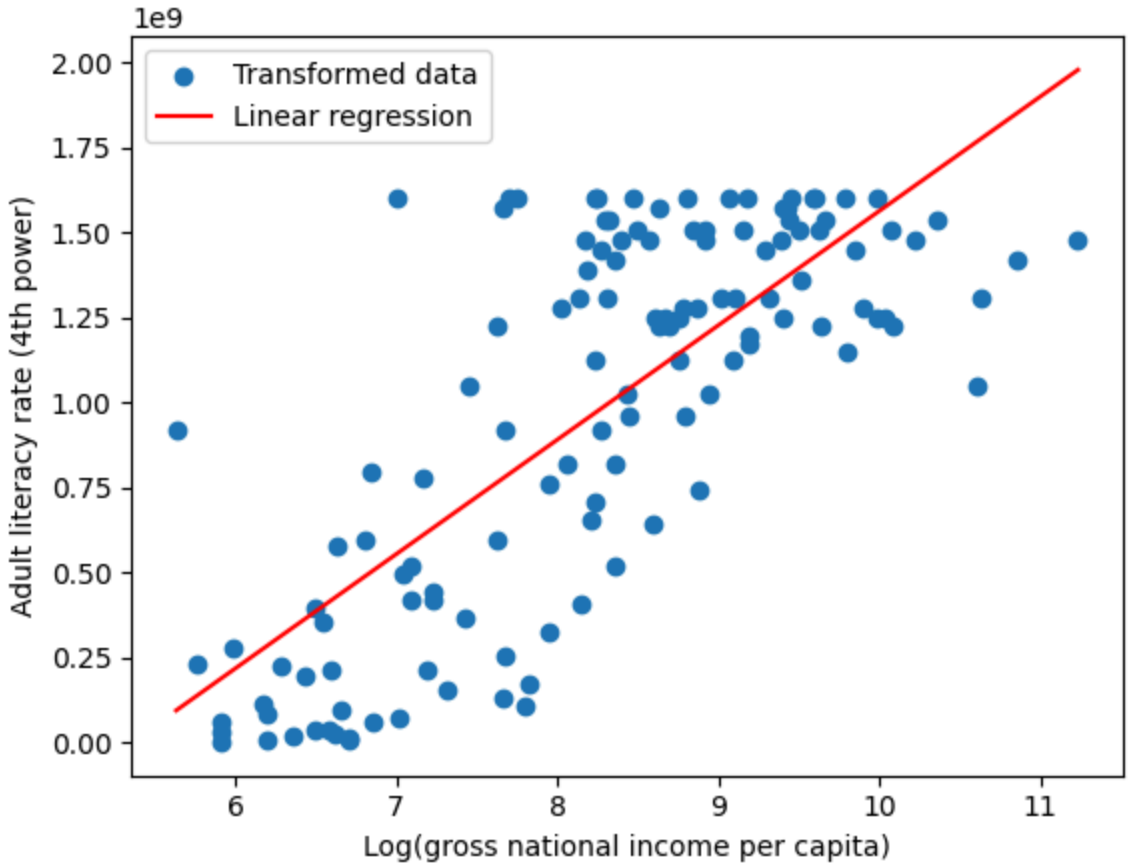
$$y^4 = m(\log x) + b$$

Where m represents the slope of the linear fit, while b represents the intercept.

The cell below computes m and b for our transformed data. We'll discuss how this code was generated in a future lecture.

Click to see the code

```
The slope, m, of the transformed data is: 336400693.43172705
The intercept, b, of the transformed data is: -1802204836.0479987
```



What if we want to understand the *underlying* relationship between our original variables, before they were transformed? We can simply rearrange our linear expression above!

Recall our linear relationship between the transformed variables $\log x$ and y^4.

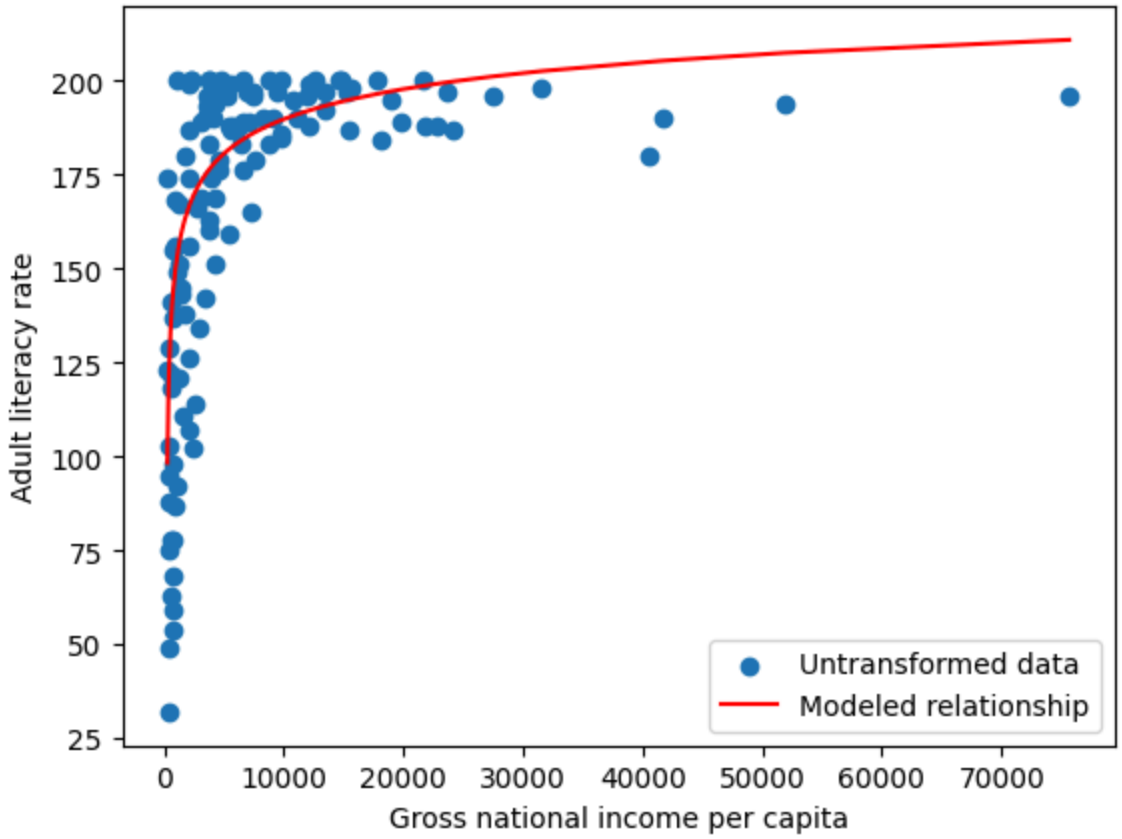
$$y^4 = m(\log x) + b$$

By rearranging the equation, we find a relationship between the untransformed variables x and y.

$$y = [m(\log x) + b]^{(1/4)}$$

When we plug in the values for m and b computed above, something interesting happens.

Click to see the code



We have found a relationship between our original variables – gross national income and adult literacy rate!

Transformations are powerful tools for understanding our data in greater detail. To summarize what we just achieved:

- We identified appropriate transformations to **linearize** the original data.
- We used our knowledge of linear curves to compute the slope and intercept of the transformed data.
- We used this slope and intercept information to derive a relationship in the untransformed data.

Linearization will be an important tool as we begin our work on linear modeling next week.

Additional Remarks

Visualization requires a lot of thought!

- There are many tools for visualizing distributions.

- Distribution of a single variable:
    1. Rugplot
    2. Histogram
    3. Density plot
    4. Box plot
    5. Violin plot
- Joint distribution of two quantitative variables:
    1. Scatter plot
    2. Hex plot
    3. Contour plot

This class primarily uses `seaborn` and `matplotlib`, but `pandas` also has basic built-in plotting methods. Many other visualization libraries exist, and `plotly` is one of them.

- `plotly` makes it very easily creates interactive plots.
- `plotly` will occasionally appear in lecture code, labs, and assignments!

Next, we'll go deeper into the theory behind visualization.

Visualization Theory

This section marks a pivot to the second major topic of this lecture - visualization theory. We'll discuss the abstract nature of visualizations and analyze how they convey information.

Remember, we had two goals for visualizing data. This section is particularly important in:

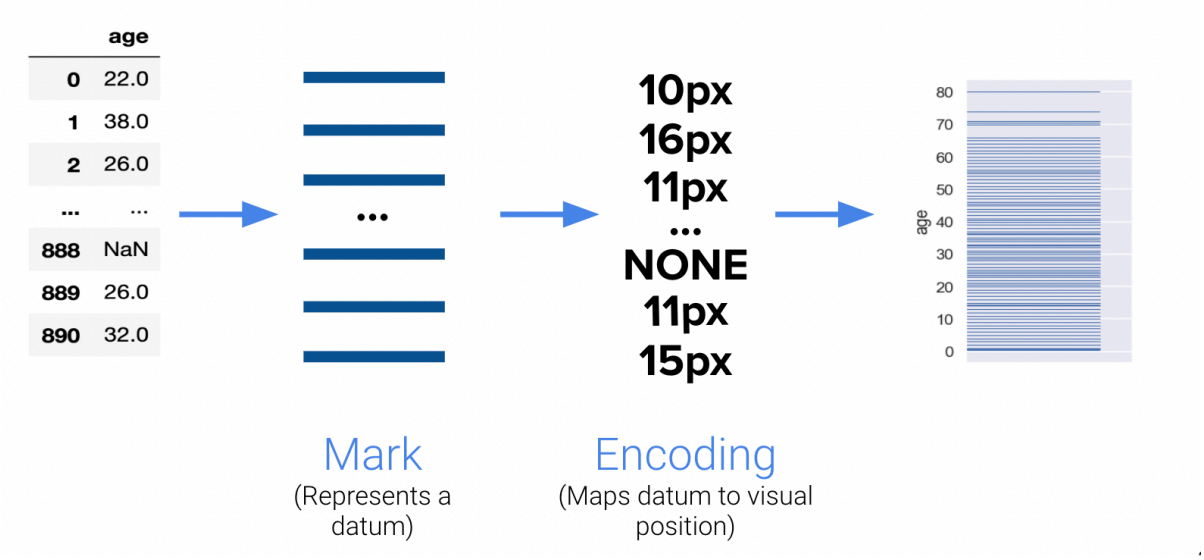
1. Helping us understand the data and results,
2. Communicating our results and conclusions with others.

Information Channels

Visualizations are able to convey information through various encodings. In the remainder of this lecture, we'll look at the use of color, scale, and depth, to name a few.

Encodings in Rugplots

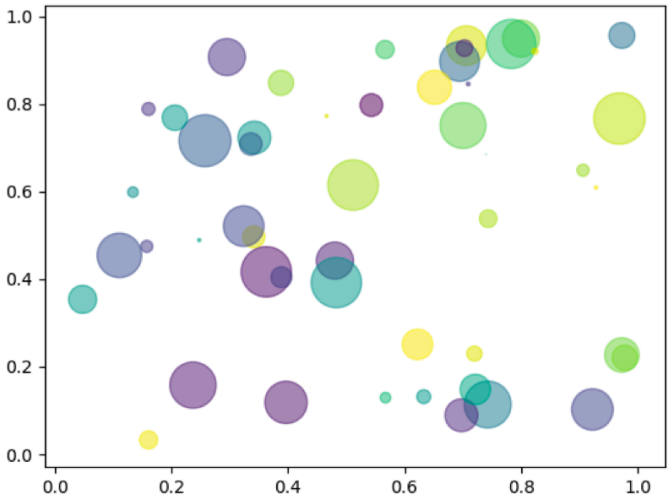
One detail that we may have overlooked in our earlier discussion of rugplots is the importance of encodings. Rugplots are effective visuals because they utilize line thickness to encode frequency. Consider the following diagram:



Multi-Dimensional Encodings

Encodings are also useful for representing multi-dimensional data. Notice how the following visual highlights four distinct "dimensions" of data:

- X-axis
- Y-axis
- Area
- Color

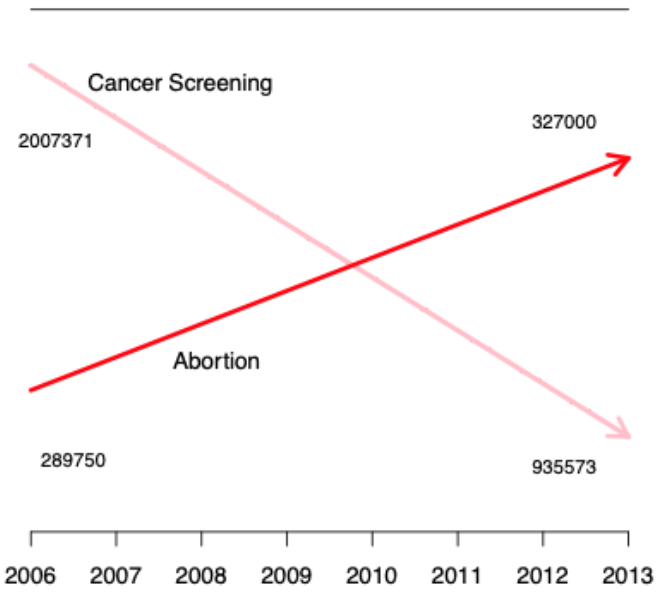


The human visual perception system is only capable of visualizing data in a three-dimensional plane, but as you've seen, we can encode many more channels of information.

Harnessing X|Y

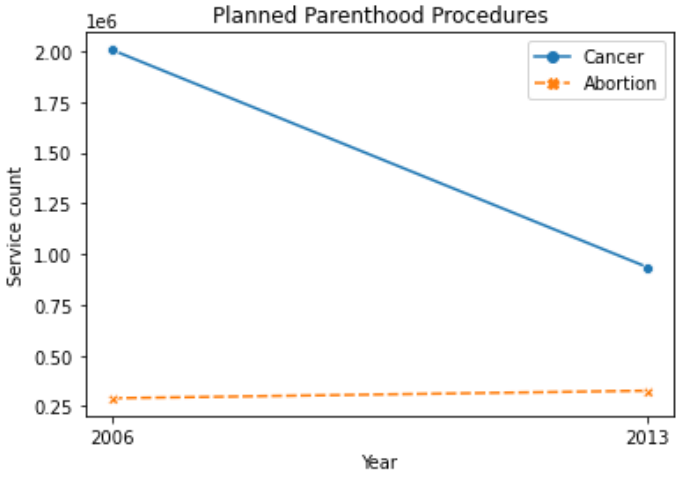
Consider the Scale of the Data

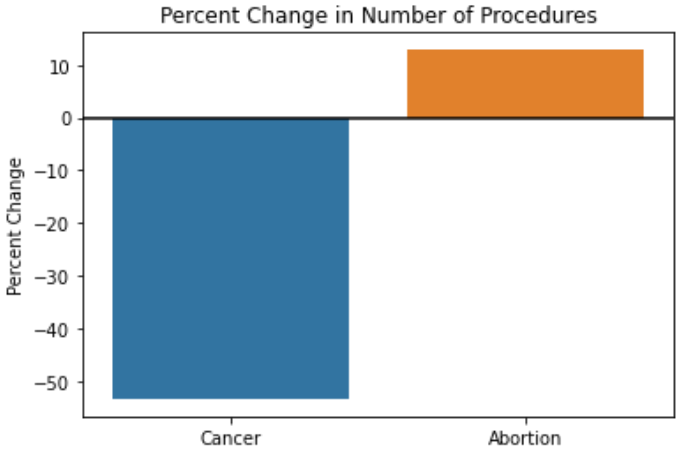
However, we should be careful to not misrepresent relationships in our data by manipulating the scale or axes. The visualization below improperly portrays two seemingly independent relationships on the same plot. The authors have clearly changed the scale of the y-axis to mislead their audience.



Notice how the downwards-facing line segment contains values in the millions, while the upwards-trending segment only contains values near three hundred thousand. These lines should not be intersecting.

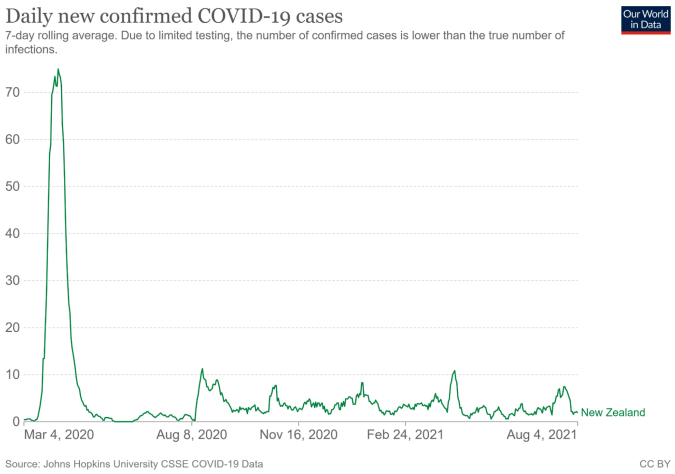
When there is a large difference in the magnitude of the data, it's advised to analyze percentages instead of counts. The following diagrams correctly display the trends in cancer screening and abortion rates.

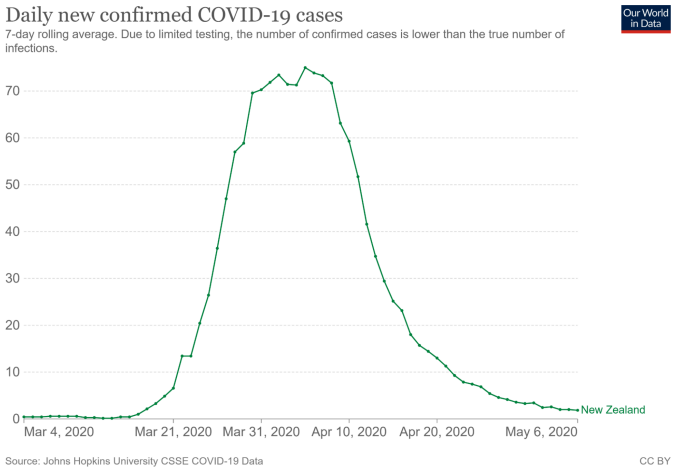




Reveal the Data

Great visualizations not only consider the scale of the data but also utilize the axes in a way that best conveys information. For example, data scientists commonly set certain axes limits to highlight parts of the visualization they are most interested in.





The visualization on the right captures the trend in coronavirus cases during March of 2020. From only looking at the visualization on the left, a viewer may incorrectly believe that coronavirus began to skyrocket on March 4^th^, 2020. However, the second illustration tells a different story - cases rose closer to March 21^th^, 2020.

Harnessing Color

Color is another important feature in visualizations that does more than what meets the eye.

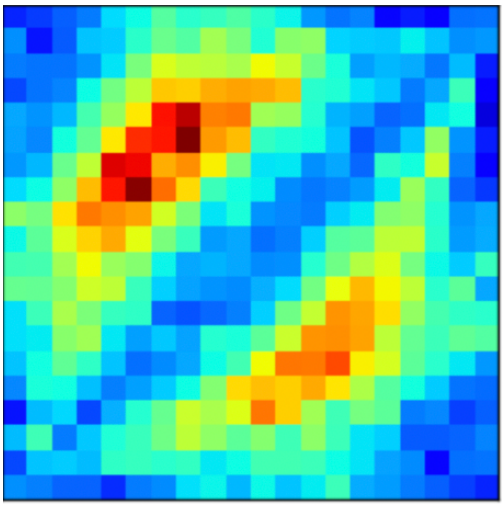
We already explored using color to encode a categorical variable in our scatter plot. Let's now discuss the uses of color in novel visualizations like colormaps and heatmaps.

5-8% of the world is red-green color blind, so we have to be very particular about our color scheme. We want to make these as accessible as possible. Choosing a set of colors that work together is evidently a challenging task!

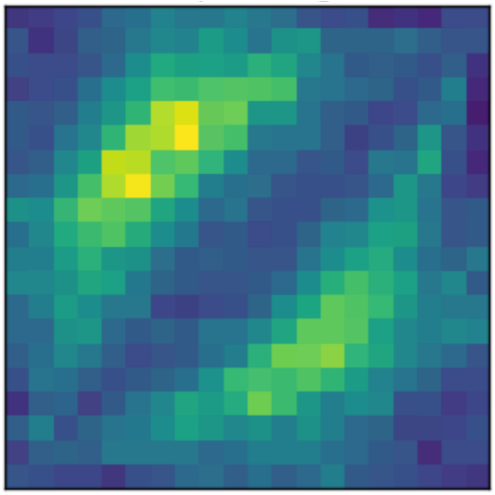
Colormaps

Colormaps are mappings from pixel data to color values, and they're often used to highlight distinct parts of an image. Let's investigate a few properties of colormaps.

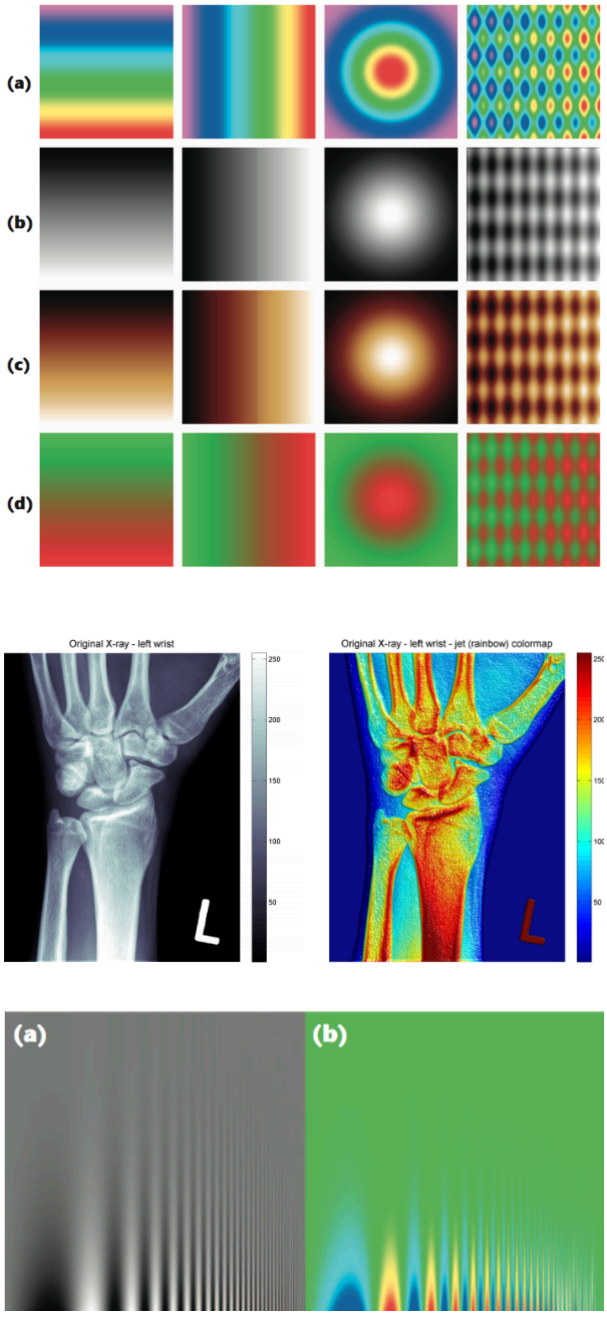
Jet Colormap:



Viridis Colormap



The jet colormap is infamous for being misleading. While it seems more vibrant than viridis, the aggressive colors poorly encode numerical data. To understand why, let's analyze the following images.

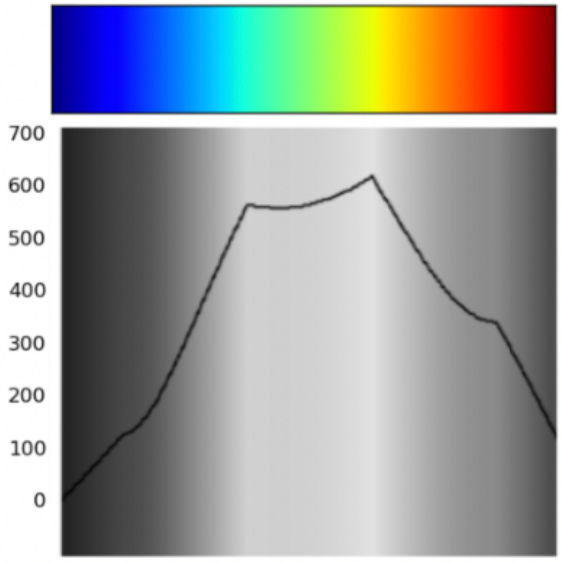


The first diagram compares how a variety of colormaps represent pixel data that transitions from a high to low intensity. These include the jet colormap (row a) and grayscale (row b). Notice how the grayscale images do the best job in smoothly transitioning between pixel data. The jet colormap is the worst at this - the four images in row (a) look like a conglomeration of individual colors.

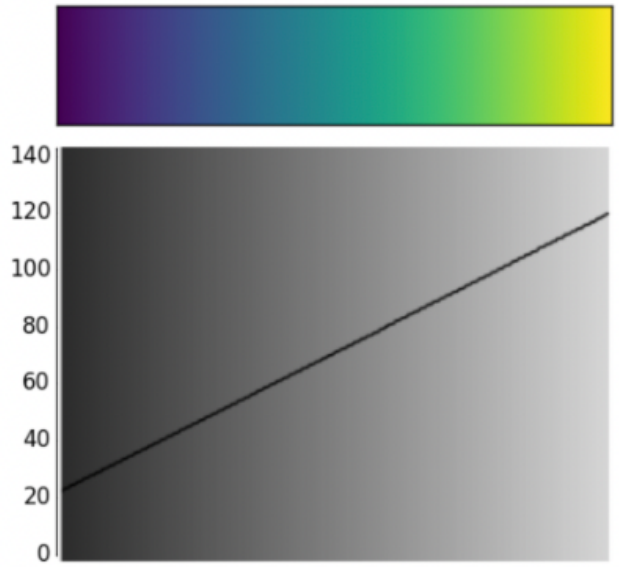
The difference is also evident in the images labeled (a) and (b) in the first diagram. The grayscale image is better at preserving finer detail in the vertical line strokes. Additionally, grayscale is preferred in X-ray scans for being more neutral. The intensity of the dark red color in the jet colormap is frightening and indicates something is wrong.

Why is the jet colormap so much worse? The answer lies in how its color composition is perceived to the human eye.

Jet Colormap Perception



Viridis Colormap Perception

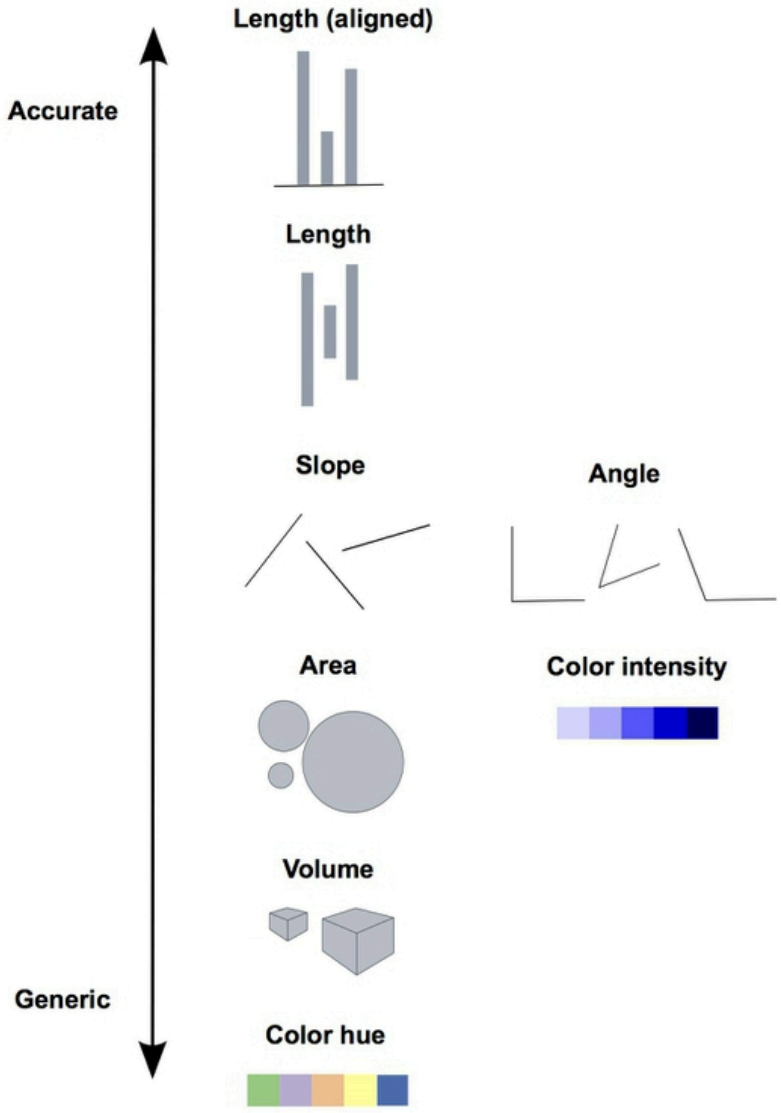


The jet colormap is largely misleading because it is not perceptually uniform. **Perceptually uniform colormaps** have the property that if the pixel data goes from 0.1 to 0.2, the perceptual change is the same as when the data goes from 0.8 to 0.9.

Notice how the said uniformity is present within the linear trend displayed in the viridis colormap. On the other hand, the jet colormap is largely non-linear - this is precisely why it's considered a worse colormap.

Harnessing Markings

In our earlier discussion of multi-dimensional encodings, we analyzed a scatter plot with four pseudo-dimensions: the two axes, area, and color. Were these appropriate to use? The following diagram analyzes how well the human eye can distinguish between these "markings".



There are a few key takeaways from this diagram

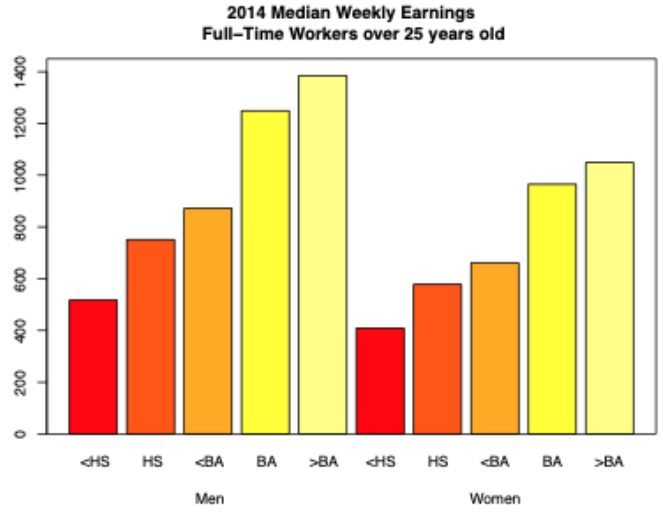
- Lengths are easy to discern. Don't use plots with jiggled baselines - keep everything axis-aligned.

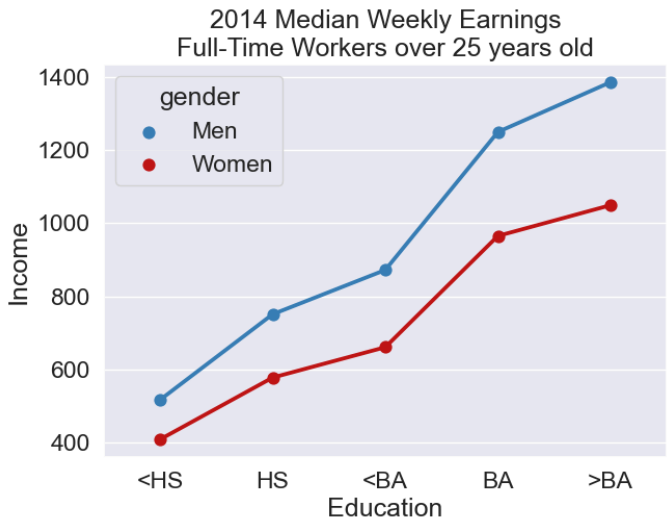
- Avoid pie charts! Angle judgments are inaccurate.
- Areas and volumes are hard to distinguish (area charts, word clouds, etc.).

Harnessing Conditioning

Conditioning is the process of comparing data that belong to separate groups. We've seen this before in overlaid distributions, side-by-side box plots, and scatter plots with categorical encodings. Here, we'll introduce terminology that formalizes these examples.

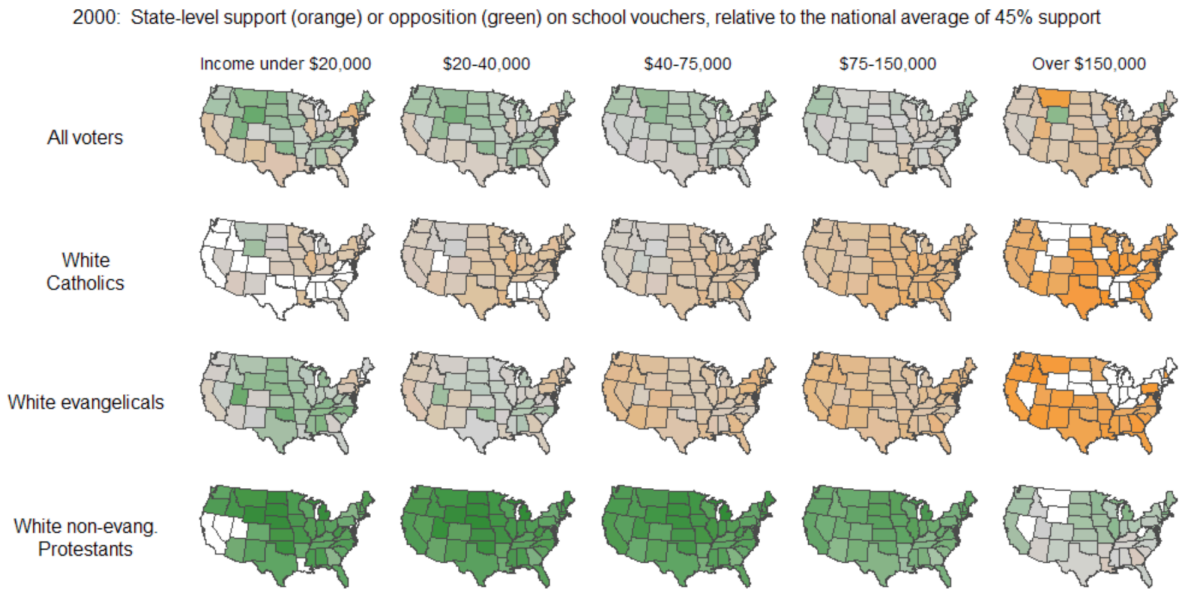
Consider an example where we want to analyze income earnings for males and females with varying levels of education. There are multiple ways to compare this data.





The barplot is an example of **juxtaposition**: placing multiple plots side by side, with the same scale. The scatter plot is an example of **superposition**: placing multiple density curves and scatter plots on top of each other.

Which is better depends on the problem at hand. Here, superposition makes the precise wage difference very clear from a quick glance. However, many sophisticated plots convey information that favors the use of juxtaposition. Below is one example.



Harnessing Context

The last component of a great visualization is perhaps the most critical - the use of context. Adding informative titles, axis labels, and descriptive captions are all best practices that we've heard repeatedly in Data 8.

A publication-ready plot (and every Data 100 plot) needs:

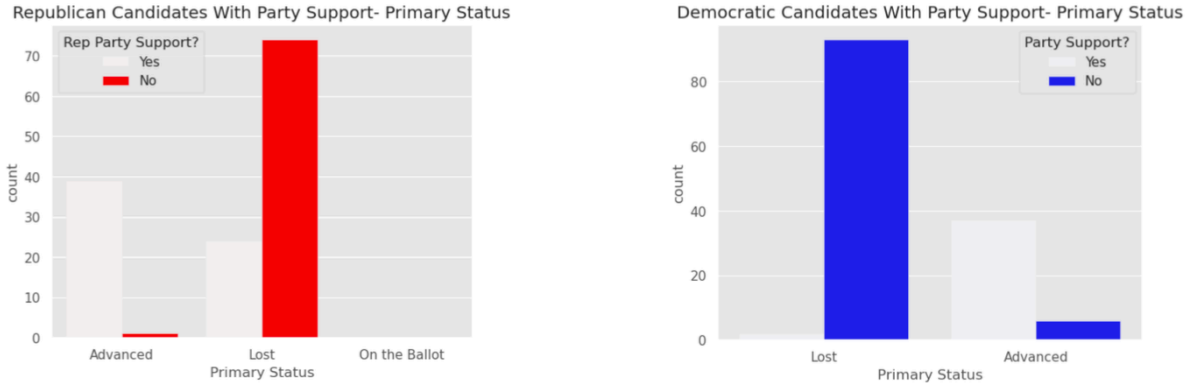
- **Informative title (takeaway, not description)**,
- **Axis labels**,
- Reference lines, markers, etc,
- Legends, if appropriate,
- Captions that describe data,

Captions should:

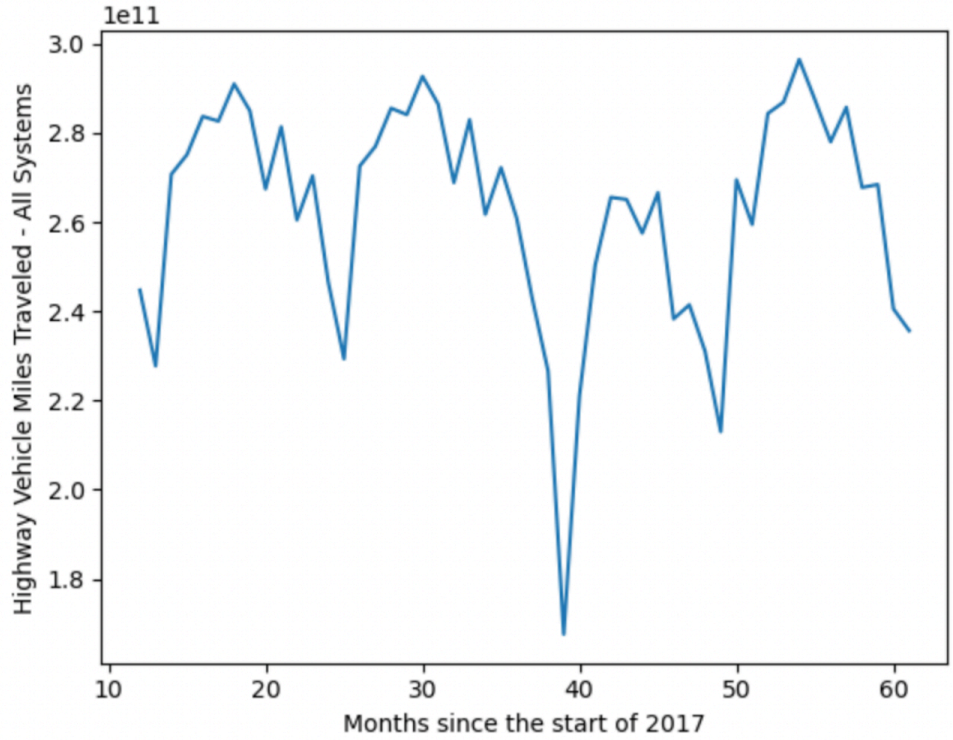
- Be comprehensive and self-contained,
- Describe what is being visualized,
- Draw attention to important features,
- Describe conclusions drawn from graphs.

Examples of Bad Visualization

After reviewing many good visualization practices, it is also helpful to examine examples of how improper use of visualization elements can negatively affect interpretation.

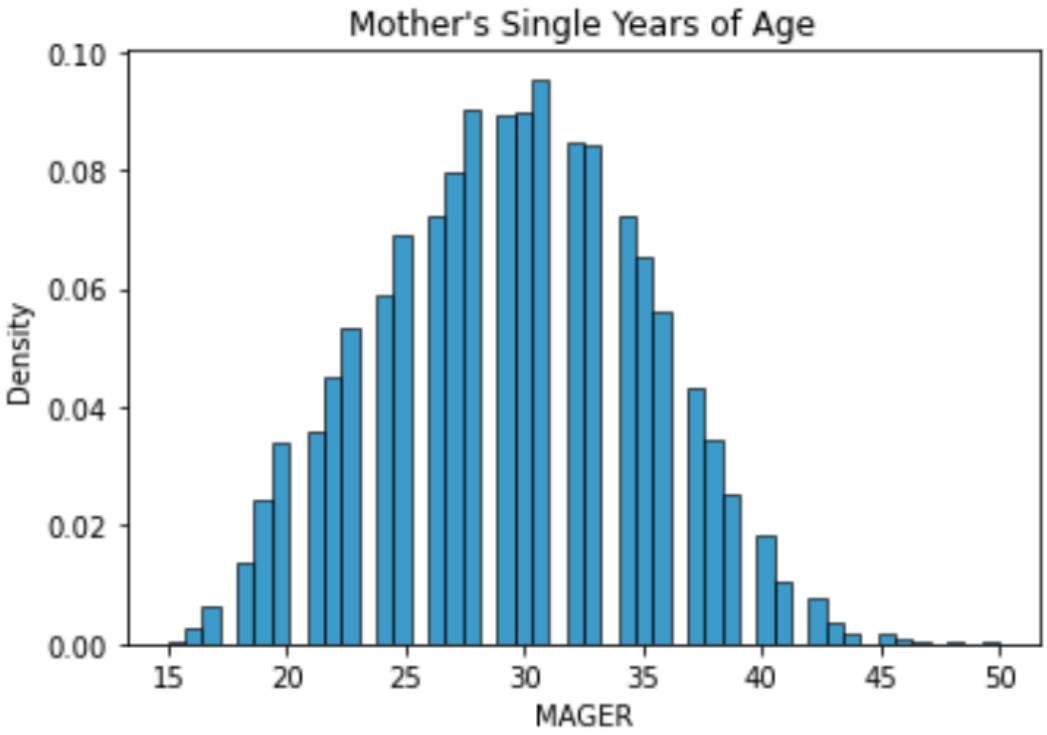


In this example, color is used poorly, making it difficult to distinguish and compare the “Yes” category with the other “No” category. In addition, the category ordering is not clearly aligned, which makes it harder for viewers to quickly understand the results.



Caption: A line plot graphing the number of months since the start of 2017 and the highway vehicle miles traveled for all systems

In this example, “Months since the start of 2017” makes it difficult for readers to interpret the timeline. In addition, the scientific notation used on the y-axis is hard to read and may be confusing for a general audience.



In this example, the data appear to be missing because the histogram bins are set inappropriately, creating the impression that values are missing when this is unlikely to be the case.

Data 100: Principles and Techniques of Data Science
Visualization I

Data 100: Principles and Techniques of Data Science
Sampling

Introduction

CONTENTS

Learning Outcomes

- Acquaint yourself with the overarching goals of Data 100.
- Understand the stages of the data science lifecycle.
- Build familiarity with `pandas` and `pandas` syntax.
- Learn a key data structure: `Series`.

Data science is an interdisciplinary field with a variety of applications and offers great potential to address challenging societal issues. By building data science skills, you can empower yourself to participate in and drive conversations that shape your life and society as a whole, whether that be fighting against climate change, launching diversity initiatives, or more.

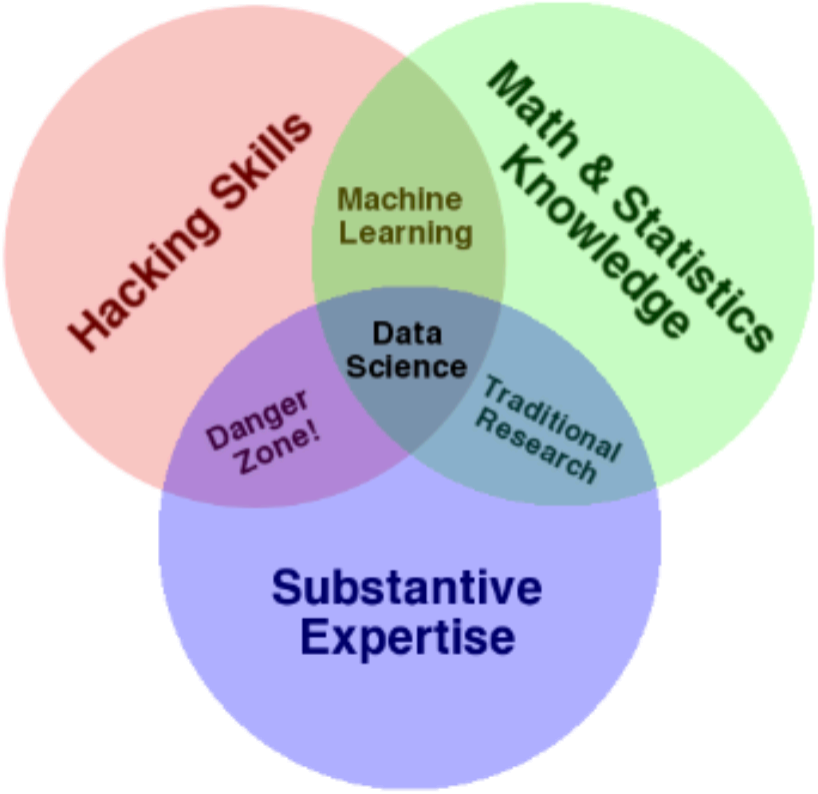
The field of data science is rapidly evolving; many of the key technical underpinnings in modern-day data science have been popularized during the early 21^st^ century, and you will learn them throughout the course. It has a wide range of applications from science and medicine to sports.

While data science has immense potential to address challenging problems facing society by enhancing our critical thinking, it can also be used obscure complex decisions and reinforce historical trends and biases. This course will implore you to consider the ethics of data science within its applications.

Data science is fundamentally human-centered and facilitates decision-making by quantitatively balancing tradeoffs. To quantify things reliably, we must use and analyze data appropriately, apply critical thinking and skepticism at every step of the way, and consider how our decisions affect others.

Ultimately, data science is the application of data-centric, computational, and inferential thinking to:

- Understand the world (science).
- Solve problems (engineering).



A true mastery of data science requires a deep theoretical understanding and strong grasp of domain expertise. This course will help you build on the former -- specifically, the foundation of your technical knowledge, allowing you to take data and produce useful insights on the world's most challenging and ambiguous problems.

Course Goals

Some Topics We'll Cover

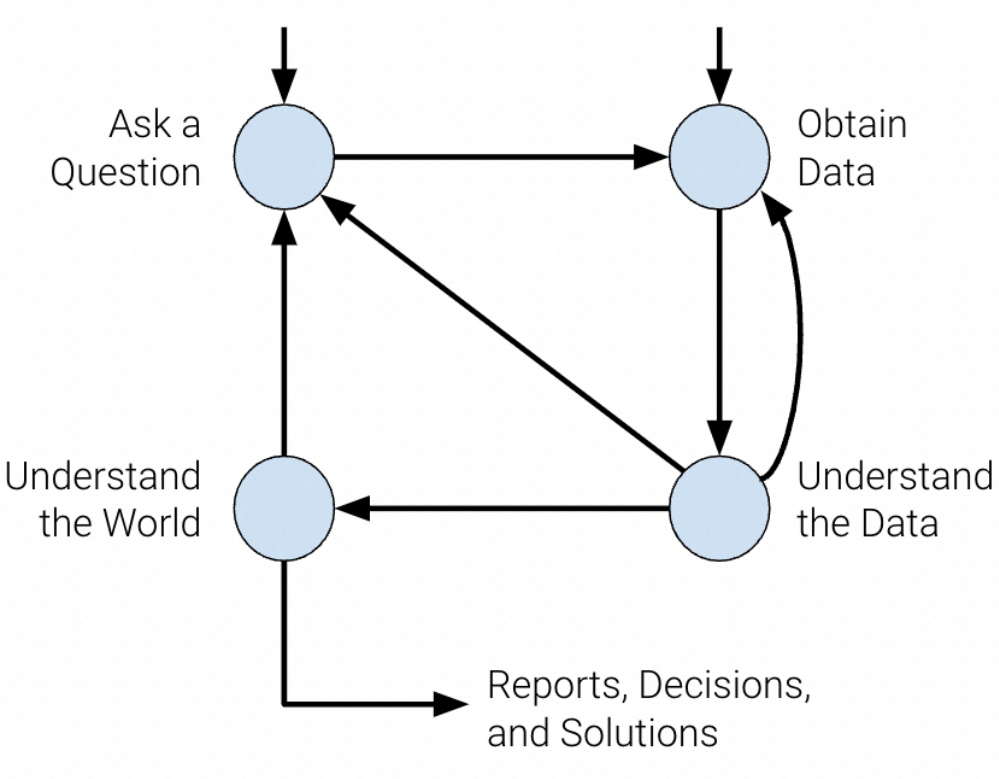
Prerequisites

To set you up for success, we've organized concepts in Data 100 around the **data science lifecycle**: an *iterative* process that encompasses the various statistical and computational building blocks of data science.

Data Science Lifecycle

The data science lifecycle is a *high-level overview* of the data science workflow. It's a cycle of stages that a data scientist should explore as they conduct a thorough analysis of a data-driven problem.

There are many variations of the data science lifecycle but the core ideas are all the same. In Data 100, we visualize the stages of the lifecycle using a flow diagram. Notice how there are two entry points.

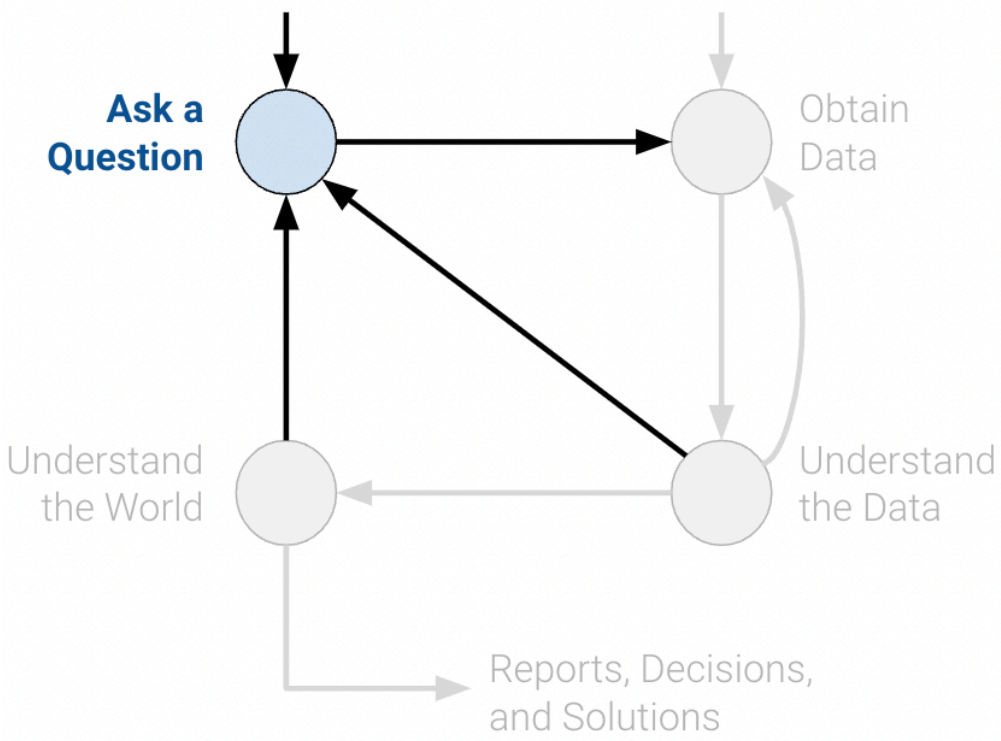


Ask a Question

Whether by curiosity or necessity, data scientists constantly ask questions. For example, in the business world, data scientists may be interested in predicting the profit generated by a certain investment. In the field of medicine, they may ask whether some patients are more likely than others to benefit from a treatment.

Posing questions is one of the primary ways the data science lifecycle begins. It helps to fully define the question. Here are some things you should ask yourself before framing a question.

- What problems are we trying to solve?
    - The goal of asking a question should be clear in order to justify your efforts to stakeholders.
- What are the hypotheses we want to test?
    - This gives a clear perspective from which to analyze final results.
- What are the metrics for our success?
    - This establishes a clear point to know when to conclude the project.

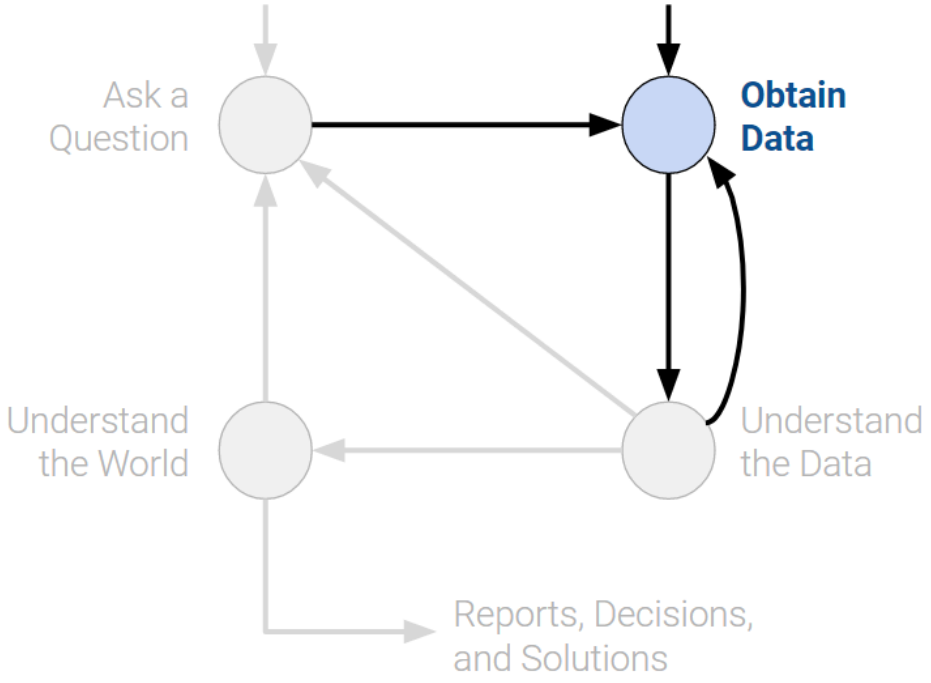


Obtain Data

The second entry point to the lifecycle is by obtaining data. A careful analysis of any problem requires the use of data. Data may be readily available to us, or we may have to embark on a process to collect it. When doing so, it is crucial to ask the following:

- What data do we have, and what data do we need?
    - Define the units of the data (people, cities, points in time, etc.) and what features to measure.
- How can we sample more data?
    - Scrape the web, collect manually, run experiments, etc.
- Is our data representative of the population we want to study?
    - If our data is not representative of our population of interest, then we can come to incorrect conclusions.

Key procedures: *data acquisition*, *data cleaning*

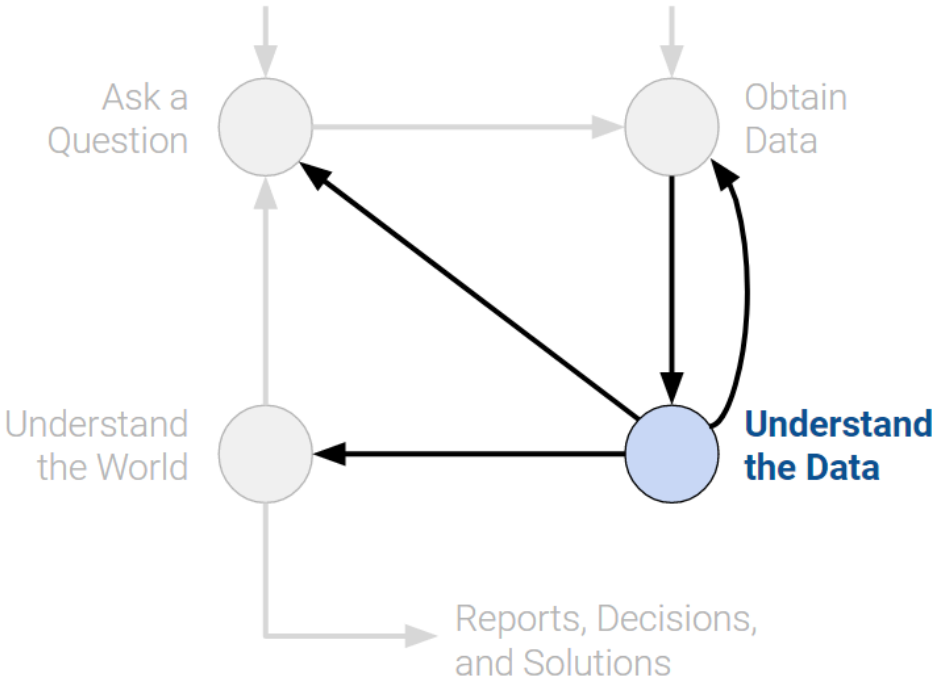


Understand the Data

Raw data itself is not inherently useful. It's impossible to discern all the patterns and relationships between variables without carefully investigating them. Therefore, translating pure data into actionable insights is a key job of a data scientist. For example, we may choose to ask:

- How is our data organized, and what does it contain?
    - Knowing what the data says about the world helps us better understand the world.
- Do we have relevant data?
    - If the data we have collected is not useful to the question at hand, then we must collect more data.
- What are the biases, anomalies, or other issues with the data?
    - These can lead to many false conclusions if ignored, so data scientists must always be aware of these issues.
- How do we transform the data to enable effective analysis?
    - Data is not always easy to interpret at first glance, so a data scientist should strive to reveal the hidden insights.

Key procedures: *exploratory data analysis*, *data visualization*.



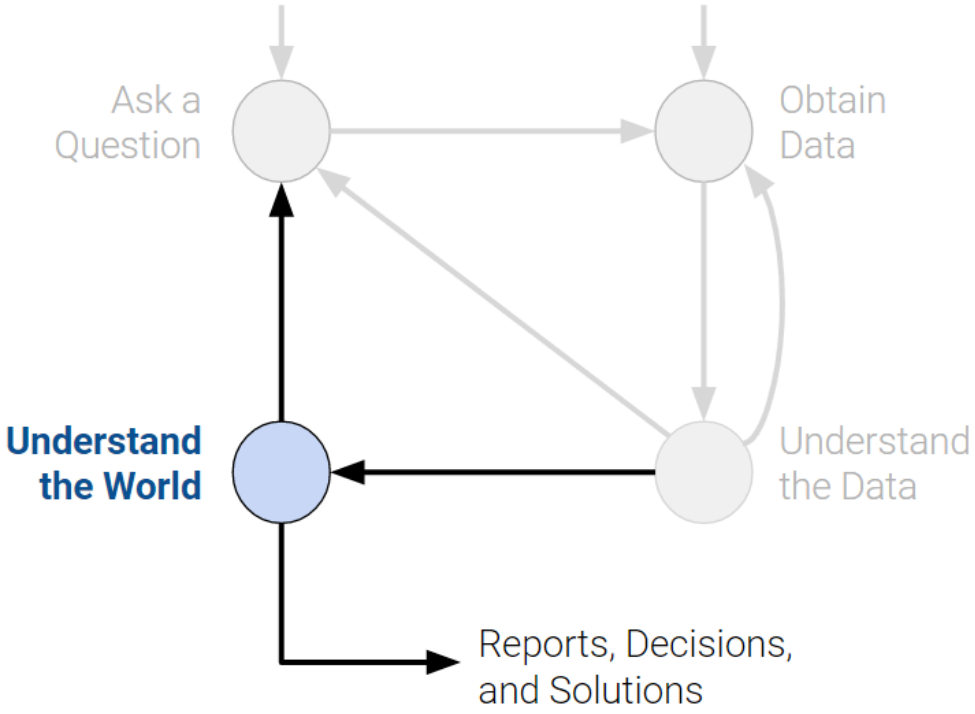
Understand the World

After observing the patterns in our data, we can begin answering our questions. This may require that we predict a quantity (machine learning) or measure the effect of some treatment (inference).

From here, we may choose to report our results, or possibly conduct more analysis. We may not be satisfied with our findings, or our initial exploration may have brought up new questions that require new data.

- What does the data say about the world?
    - Given our models, the data will lead us to certain conclusions about the real world.
- Does it answer our questions or accurately solve the problem?
    - If our model and data can not accomplish our goals, then we must reform our question, model, or both.
- How robust are our conclusions and can we trust the predictions?
    - Inaccurate models can lead to false conclusions.

Key procedures: *model creation*, *prediction*, *inference*.



The data science lifecycle is meant to be a set of general guidelines rather than a hard set of requirements. In our journey exploring the lifecycle, we'll cover both the underlying theory and technologies used in data science. By the end of the course, we hope that you start to see yourself as a data scientist.

With that, we'll begin by introducing one of the most important tools in exploratory data analysis: `pandas`.

In this sequence of lectures, we will dive right into things by having you explore and manipulate real-world data. We'll first introduce `pandas`, a popular Python library for interacting with **tabular data**.

Tabular Data and `pandas`

Data scientists work with data stored in a variety of formats. This class focuses primarily on *tabular data* — data that is stored in a table.

Tabular data is one of the most common systems that data scientists use to organize data. This is in large part due to the simplicity and flexibility of tables. Tables allow us to represent each **observation**, or instance of collecting data from an individual, as its own *row*. We can record each observation's distinct characteristics, or **features**, in separate *columns*.

To see this in action, we'll explore the `elections` dataset, which stores information about political candidates who ran for president of the United States in previous years.

In the `elections` dataset, each row (blue box) represents one instance of a candidate running for president in a particular year. For example, the first row represents Andrew Jackson running for president in the year 1824. Each column (yellow box) represents one characteristic piece of information about each presidential candidate. For example, the column named "Result" stores whether or not the candidate won the election.

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

Your work in Data 8 helped you grow very familiar with using and interpreting data stored in a tabular format. Back then, you used the `Table` class of the `datascience` library, a special programming library created specifically for Data 8 students.

In Data 100, we will be working with the programming library `pandas`, which is generally accepted in the data science community as the industry- and academia-standard tool for manipulating tabular data (as well as the inspiration for Petey, our panda bear mascot).

Using `pandas`, we can

- Arrange data in a tabular format.

- Extract useful information filtered by specific conditions.
- Operate on data to gain new insights.
- Apply `NumPy` functions to our data (our friends from Data 8).
- Perform vectorized computations to speed up our analysis (Lab 1).

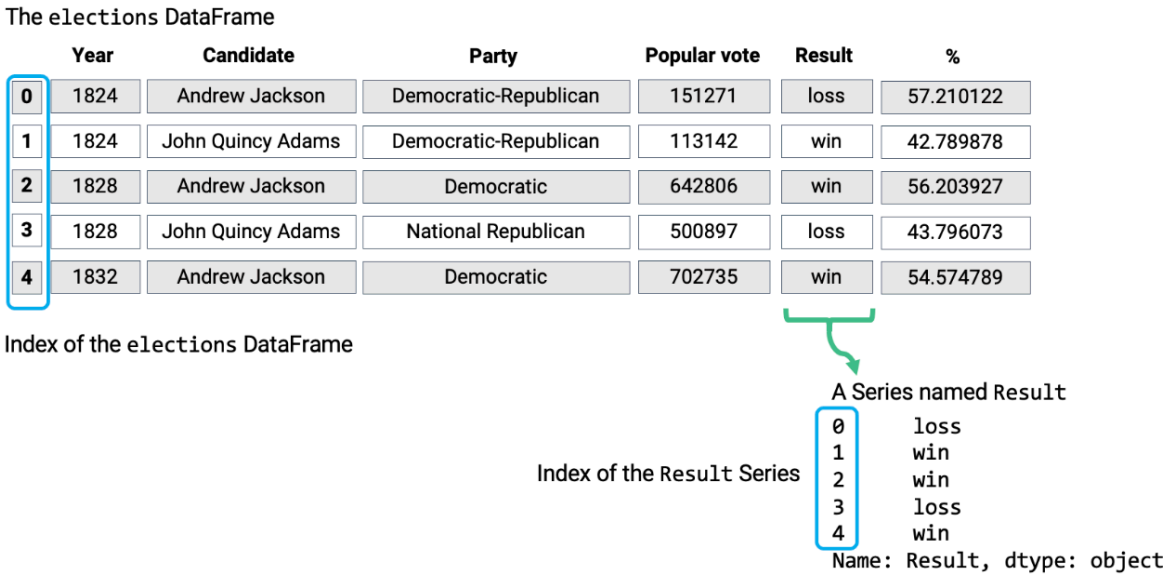
To begin our work in `pandas`, we must first import the library into our Python environment. This will allow us to use `pandas` data structures and methods in our code.

```
# `pd` is the conventional alias for Pandas, as `np` is for NumPy
import pandas as pd
```

There are three fundamental data structures in `pandas`:

1. `Series`: 1D labeled array data; best thought of as columnar data.
2. `DataFrame`: 2D tabular data with rows and columns.
3. `Index`: A sequence of row/column labels.

`DataFrame`s, `Series`, and Indices can be represented visually in the following diagram, which considers the first few rows of the `elections` dataset.



Notice how the **DataFrame** is a two-dimensional object — it contains both rows and columns. The **Series** above is a singular column of this `DataFrame`, namely the `Result` column. Both contain an **Index**, or a shared list of row labels (the integers from 0 to 4, inclusive).

Today, we will start off from `Series` object.

Series

A `Series` represents a column of a `DataFrame`; more generally, it can be any 1-dimensional array-like object. It contains both:

- A sequence of **values** of the same type.
- A sequence of data labels called the **index**.

In the cell below, we create a `Series` named `s`.

```
s = pd.Series(["welcome", "to", "data 100"])
s
```

```
0     welcome
1          to
2    data 100
dtype: object
```

```
# Accessing data values within the Series
s.values
```

```
array(['welcome', 'to', 'data 100'], dtype=object)
```

```
# Accessing the Index of the Series
s.index
```

```
RangeIndex(start=0, stop=3, step=1)
```

By default, the `index` of a `Series` is a sequential list of integers beginning from 0. Optionally, a manually specified list of desired indices can be passed to the `index` argument.

```
s = pd.Series([-1, 10, 2], index = ["a", "b", "c"])
s
```

```
a    -1
b    10
c     2
dtype: int64
```

```
s.index
```

```
Index(['a', 'b', 'c'], dtype='object')
```

Indices can also be changed after initialization.

```
s.index = ["first", "second", "third"]
s
```

```
first     -1
second    10
third      2
dtype: int64
```

```
s.index
```

```
Index(['first', 'second', 'third'], dtype='object')
```

Selection in `Series`

Much like when working with `NumPy` arrays, we can select a single value or a set of values from a `Series`. To do so, there are three primary methods:

1. A single label.

2. A list of labels.
3. A filtering condition.

To demonstrate this, let's define a new Series `s`.

```
s = pd.Series([4, -2, 0, 6], index = ["a", "b", "c", "d"])
s
```

```
a    4
b   -2
c    0
d    6
dtype: int64
```

A Single Label

```
# We return the value stored at the index label "a"
s["a"]
```

```
4
```

A List of Labels

```
# We return a Series of the values stored at the index labels "a" and "c"
s[["a", "c"]]
```

```
a    4
c    0
dtype: int64
```

A Filtering Condition

Perhaps the most interesting (and useful) method of selecting data from a `Series` is by using a filtering condition.

First, we apply a boolean operation to the `Series`. This creates **a new `Series` of boolean values**.

```
# Filter condition: select all elements greater than 0
s > 0
```

```
a     True
b    False
c    False
d     True
dtype: bool
```

We then use this boolean condition to index into our original `Series`. `pandas` will select only the entries in the original `Series` that satisfy the condition.

```
s[s > 0]
```

```
a    4
d    6
dtype: int64
```

Data 100: Principles and Techniques of Data Science
← Data 100: Principles and Techniques of Data Science

Data 100: Principles and Techniques of Data Science
Pandas I →

Pandas III

CONTENTS

Aggregating Data with `.groupby`
Aggregation Functions
Plotting Birth Counts
Summary of the `.groupby()` Function
Revisiting the `.agg()` Function
Nuisance Columns
Renaming Columns After Grouping
Some Data Science Payoff
groupby() Continued

Learning Outcomes

- Perform advanced aggregation using `.groupby()`
- Use the `pd.pivot_table` method to construct a pivot table

- Perform simple merges between DataFrames using `pd.merge()`

We will introduce the concept of aggregating data – we will familiarize ourselves with `GroupBy` objects and used them as tools to consolidate and summarize a `DataFrame`. In this lecture, we will explore working with the different aggregation functions and dive into some advanced `.groupby` methods to show just how powerful of a resource they can be for understanding our data. We will also introduce other techniques for data aggregation to provide flexibility in how we manipulate our tables.

First, let's load `babynames` dataset.

Click to see the code

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

Aggregating Data with `.groupby`

Up until this point, we have been working with individual rows of `DataFrame`s. As data scientists, we often wish to investigate trends across a larger *subset* of our data. For example, we may want to compute some summary statistic (the mean, median, sum, etc.) for a group of rows in our `DataFrame`. To do this, we'll use `pandas GroupBy` objects. Our goal is to group together rows that fall under the same category and perform an operation that aggregates across all rows in the category.

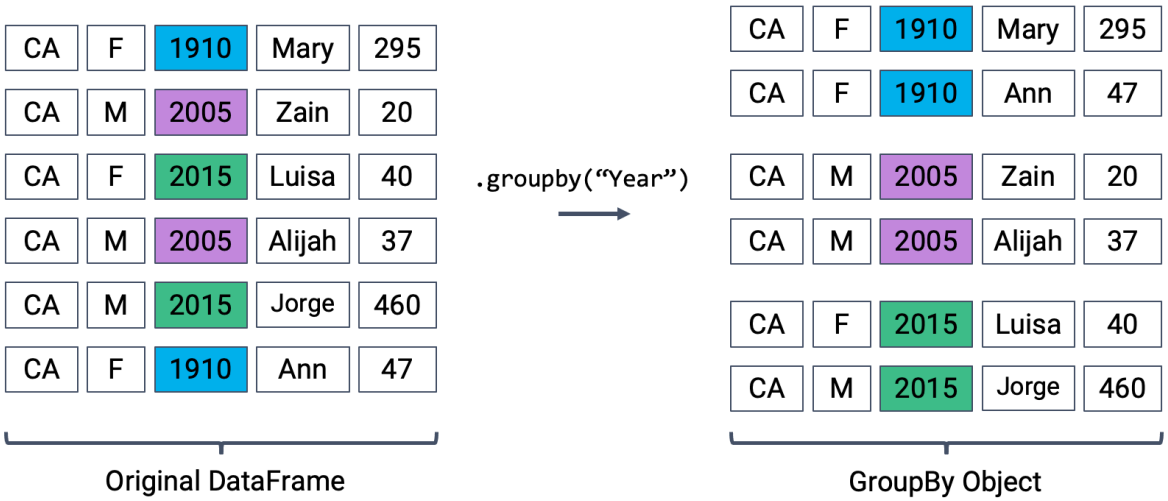
Let's say we wanted to aggregate all rows in `babynames` for a given year.

```
babynames.groupby("Year")
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x12fc94290>
```

What does this strange output mean? Calling `.groupby` (documentation) has generated a `GroupBy` object. You can imagine this as a set of "mini", grouped sub-`DataFrame`s, where each group/sub-`DataFrame` contains all of the rows from `babynames` that correspond to a particular year.

The diagram below shows a simplified view of `babynames` to help illustrate this idea.



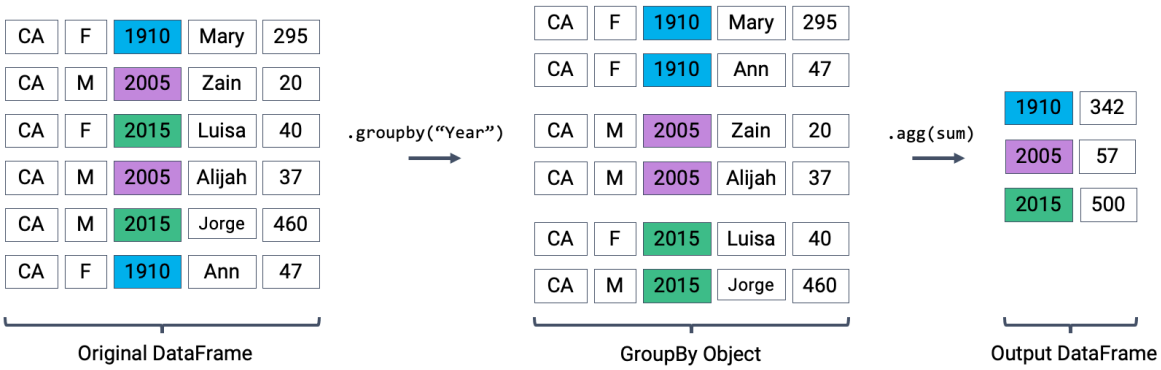
We can't work with a `GroupBy` object directly – that is why you saw that strange output earlier rather than a standard view of a `DataFrame`. To actually manipulate values within these groups/sub-`DataFrame`s, we'll need to call an *aggregation method*. This is a method that tells `pandas` how to aggregate the values within the `GroupBy` object. Once the aggregation is applied, `pandas` will return a normal (now grouped) `DataFrame`.

The first aggregation method we'll consider is `.agg`. The `.agg` method takes in a function as its argument; this function is then applied to each column of a group/sub-`DataFrame`. We end up with a new `DataFrame` with one aggregated row per subframe. Let's see this in action by finding the `sum` of all counts for each year in `babynames` – this is equivalent to finding the number of babies born in each year.

```
babynames[["Year", "Count"]].groupby("Year").agg("sum").head(5)
```

Year	Count
1910	9163
1911	9983
1912	17946
1913	22094
1914	26926

We can relate this back to the diagram we used above. Remember that the diagram uses a simplified version of `babynames`, which is why we see smaller values for the summed counts.



Calling `.agg` has condensed each subframe (i.e. group/sub-`DataFrame`) back into a single row. This gives us our final output: a `DataFrame` that is now indexed by `"Year"`, with a single row for each unique year in the original `babynames` DataFrame.

There are many different aggregation functions we can use, all of which are useful in different applications.

```
babynames[["Year", "Count"]].groupby("Year").agg("min").head(5)
```

Year	Count
1910	5
1911	5

Year	Count
1912	5
1913	5
1914	5

```
babynames[["Year", "Count"]].groupby("Year").agg("max").head(5)
```

Year	Count
1910	295
1911	390
1912	534
1913	614
1914	773

```
# Same result, but now we explicitly tell pandas to only consider the "Count" column when
babynames.groupby("Year")[["Count"]].agg("sum").head(5)
```

Year	Count
1910	9163
1911	9983
1912	17946
1913	22094
1914	26926

There are many different aggregations that can be applied to the grouped data. The primary requirement is that an aggregation function must:

- Take in a `Series` of data (a single column of the grouped subframe).
- Return a single value that aggregates this `Series`.

Aggregation Functions

Because of this fairly broad requirement, `pandas` offers many ways of computing an aggregation.

In-built Python operations – such as `sum`, `max`, and `min` – are automatically recognized by `pandas`.

```
# What is the minimum count for each name in any year?
babynames.groupby("Name")[["Count"]].agg("min").head()
```

Name	Count
Aadan	5
Aadarsh	6
Aaden	10
Aadhav	6
Aadhini	6

```
# What is the largest single-year count of each name?
babynames.groupby("Name")[["Count"]].agg("max").head()
```

Name	Count
Aadan	7
Aadarsh	6
Aaden	158
Aadhav	8
Aadhini	6

As mentioned previously, functions from the `NumPy` library, such as `np.mean`, `np.max`, `np.min`, and `np.sum`, are also fair game in `pandas`.

```
# What is the average count for each name across all years?
babynames.groupby("Name")[["Count"]].agg("mean").head()
```

	Count
Name	
Aadan	6.000000
Aadarsh	6.000000
Aaden	46.214286
Aadhav	6.750000
Aadhini	6.000000

`pandas` also offers a number of in-built functions. Functions that are native to `pandas` can be referenced using their string name within a call to `.agg`. Some examples include:

- `.agg("sum")`
- `.agg("max")`
- `.agg("min")`
- `.agg("mean")`
- `.agg("first")`
- `.agg("last")`

The latter two entries in this list – `"first"` and `"last"` – are unique to `pandas`. They return the first or last entry in a subframe column. Why might this be useful? Consider a case where *multiple* columns in a group share identical information. To represent this information in the grouped output, we can simply grab the first or last entry, which we know will be identical to all other entries.

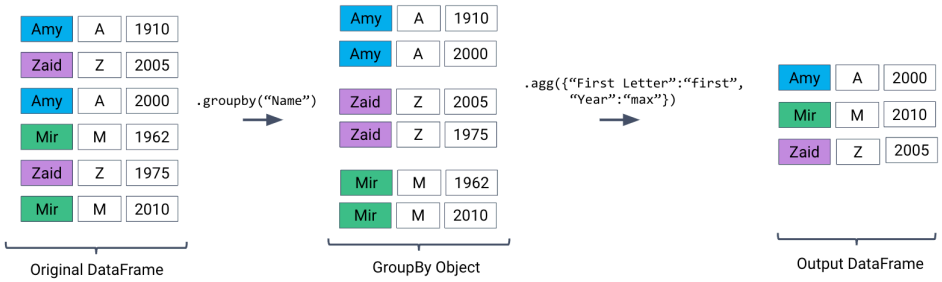
Let's illustrate this with an example. Say we add a new column to `babynames` that contains the first letter of each name.

```
# Imagine we had an additional column, "First Letter". We'll explain this code next week
babynames["First Letter"] = babynames["Name"].str[0]

# We construct a simplified DataFrame containing just a subset of columns
babynames_new = babynames[["Name", "First Letter", "Year"]]
babynames_new.head()
```

	Name	First Letter	Year
0	Mary	M	1910
1	Helen	H	1910
2	Dorothy	D	1910
3	Margaret	M	1910
4	Frances	F	1910

If we form groups for each name in the dataset, `"First Letter"` will be the same for all members of the group. This means that if we simply select the first entry for `"First Letter"` in the group, we'll represent all data in that group.

We can use a dictionary to apply different aggregation functions to each column during grouping.




```
babynames_new.groupby("Name").agg({"First Letter":"first", "Year":"max"}).head()
```

Name	First Letter	Year
Aadan	A	2014
Aadarsh	A	2019
Aaden	A	2020

Name	First Letter	Year
Aadhav	A	2019
Aadhini	A	2022

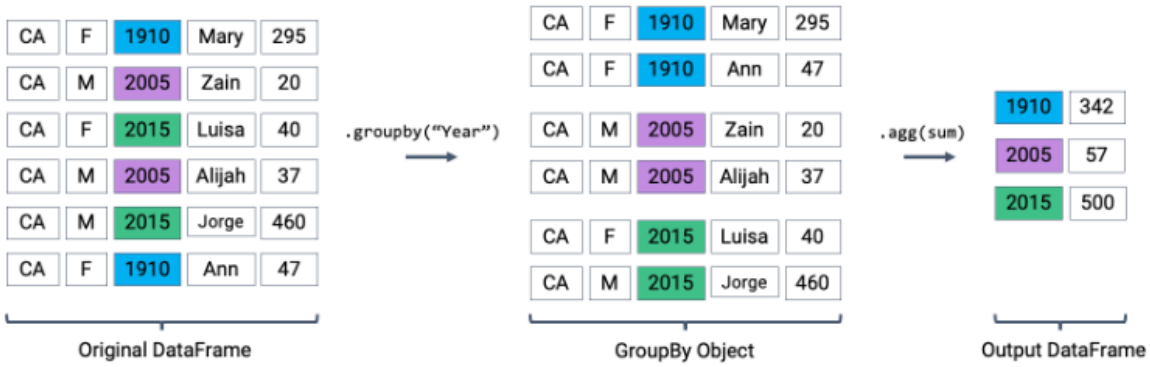
Plotting Birth Counts

Let's use `.agg` to find the total number of babies born in each year. Recall that using `.agg` with `.groupby()` follows the format: `df.groupby(column_name).agg(aggregation_function)`. The line of code below gives us the total number of babies born in each year.

Click to see the code

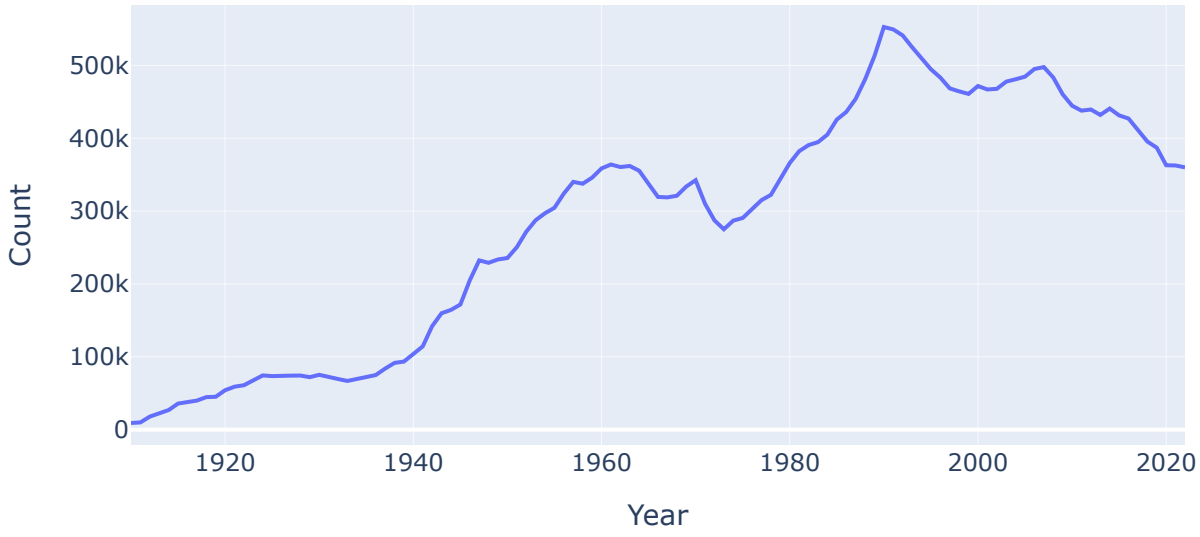
Year	Count
1910	9163
1911	9983
1912	17946
1913	22094
1914	26926

Here's an illustration of the process:



Plotting the `DataFrame` we obtain tells an interesting story.

Click to see the code



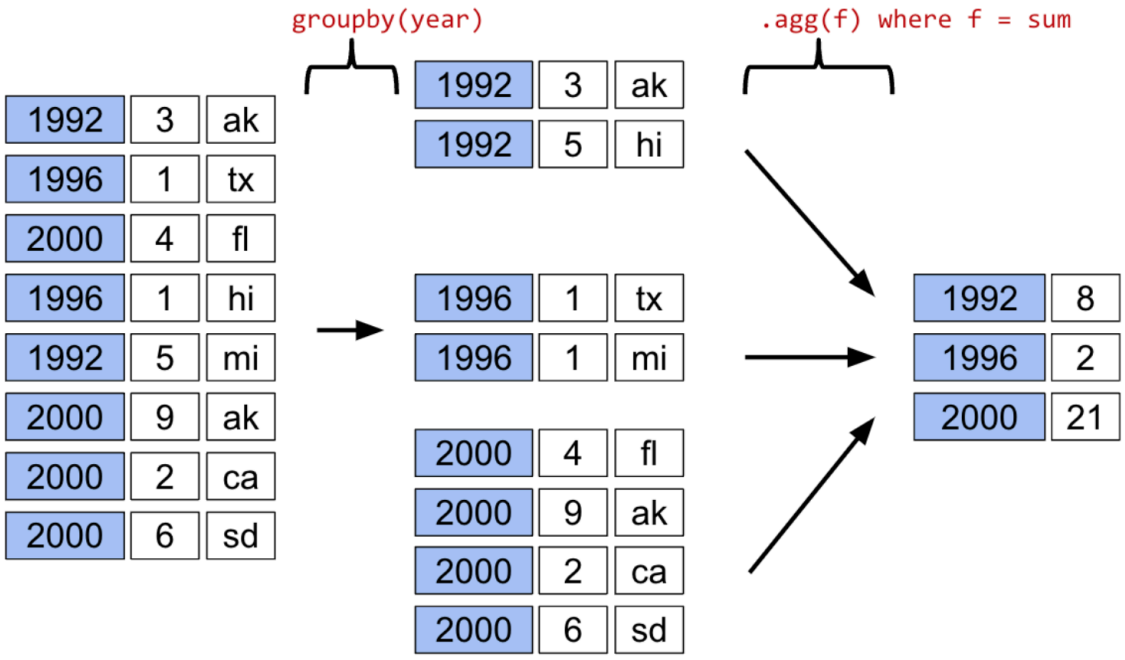
A word of warning: we made an enormous assumption when we decided to use this dataset to estimate birth rate. According to this article from the Legislative Analyst's Office, the true number of babies born in California in 2020 was 421,275. However, our plot shows 362,882 babies —— what happened?

Summary of the `.groupby()` Function

A `groupby` operation involves some combination of **splitting a `DataFrame` into grouped subframes**, **applying a function**, and **combining the results**.

For some arbitrary `DataFrame df` below, the code `df.groupby("year").agg(sum)` does the following:

- **Splits** the `DataFrame` into sub-`DataFrame`s with rows belonging to the same year.
- **Applies** the `sum` function to each column of each sub-`DataFrame`.
- **Combines** the results of `sum` into a single `DataFrame`, indexed by `year`.



Revisiting the `.agg()` Function

`.agg()` can take in any function that aggregates several values into one summary value. Some commonly-used aggregation functions can even be called directly, without explicit use of `.agg()`. For example, we can call `.mean()` on `.groupby()`:

```
babynames.groupby("Year").mean().head()
```

We can now put this all into practice. Say we want to find the baby name with sex "F" that has fallen in popularity the most in California. To calculate this, we can first create a metric: "Ratio to Peak" (RTP). The RTP is the ratio of babies born with a given name in 2022 to the *maximum* number of babies born with the name in *any* year.

Let's start with calculating this for one baby, "Jennifer".

```
# We filter by babies with sex "F" and sort by "Year"
f_babynames = babynames[babynames["Sex"] == "F"]
f_babynames = f_babynames.sort_values(["Year"])

# Determine how many Jennifers were born in CA per year
jenn_counts_series = f_babynames[f_babynames["Name"] == "Jennifer"]["Count"]

# Determine the max number of Jennifers born in a year and the number born in 2022
# to calculate RTP
max_jenn = max(f_babynames[f_babynames["Name"] == "Jennifer"]["Count"])
```

```
curr_jenn = f_babynames[f_babynames["Name"] == "Jennifer"]["Count"].iloc[-1]
rtp = curr_jenn / max_jenn
rtp
```

```
0.018796372629843364
```

By creating a function to calculate RTP and applying it to our `DataFrame` by using `.groupby()`, we can easily compute the RTP for all names at once!

```
def ratio_to_peak(series):
    return series.iloc[-1] / max(series)

#Using .groupby() to apply the function
rtp_table = f_babynames.groupby("Name")[["Year", "Count"]].agg(ratio_to_peak)
rtp_table.head()
```

Name	Year	Count
Aadhini	1.0	1.000000
Aadhira	1.0	0.500000
Aadhya	1.0	0.660000
Aadya	1.0	0.586207
Aahana	1.0	0.269231

In the rows shown above, we can see that every row shown has a `Year` value of `1.0`.

This is the "`pandas`-ification" of logic you saw in Data 8. Much of the logic you've learned in Data 8 will serve you well in Data 100.

Nuisance Columns

Note that you must be careful with which columns you apply the `.agg()` function to. If we were to apply our function to the table as a whole by doing `f_babynames.groupby("Name").agg(ratio_to_peak)`, executing our `.agg()` call would result in a `TypeError`.

```
Cell In[110], line 5, in ratio_to_peak(series)
      1 def ratio_to_peak(series):
      2     """
      3     Compute the RTP for a Series containing the counts per year for a single name
      4     """
----> 5     return series.iloc[-1] / np.max(series)

TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

We can avoid this issue (and prevent unintentional loss of data) by explicitly selecting column(s) we want to apply our aggregation function to **BEFORE** calling `.agg()`,

Renaming Columns After Grouping

By default, `.groupby` will not rename any aggregated columns. As we can see in the table above, the aggregated column is still named `Count` even though it now represents the RTP. For better readability, we can rename `Count` to `Count RTP`

```
rtp_table = rtp_table.rename(columns = {"Count": "Count RTP"})
rtp_table
```

Name	Year	Count RTP
Aadhini	1.0	1.000000
Aadhira	1.0	0.500000
Aadhya	1.0	0.660000
Aadya	1.0	0.586207
Aahana	1.0	0.269231
...	...	...
Zyanya	1.0	0.466667
Zyla	1.0	1.000000
Zylah	1.0	1.000000
Zyra	1.0	1.000000
Zyrah	1.0	0.833333

13782 rows × 2 columns

Some Data Science Payoff

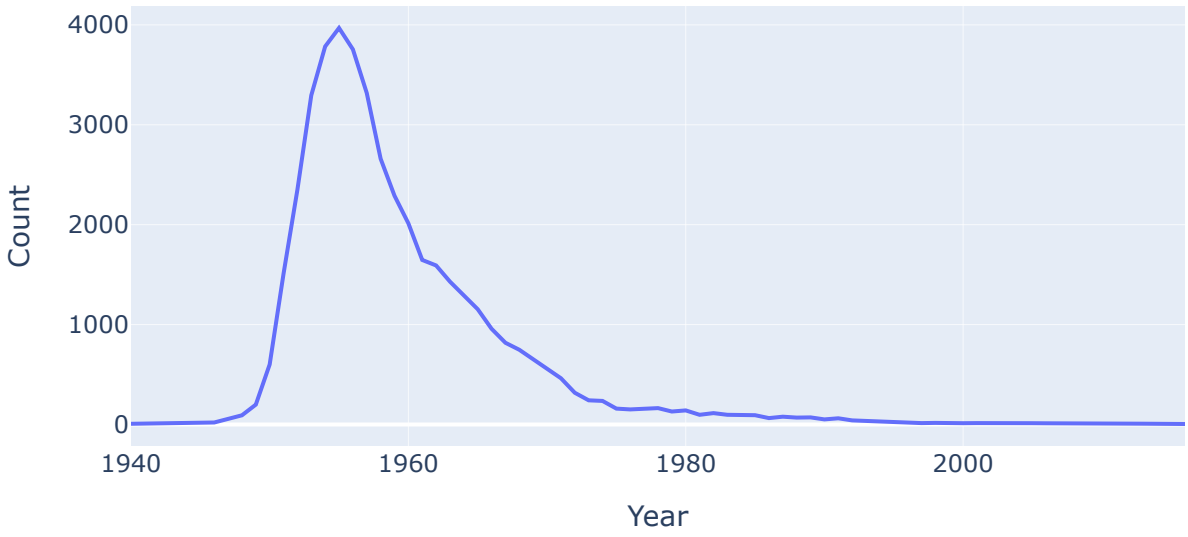
By sorting `rtp_table`, we can see the names whose popularity has decreased the most.

```
rtp_table = rtp_table.rename(columns = {"Count": "Count RTP"})
rtp_table.sort_values("Count RTP").head()
```

Name	Year	Count RTP
Debra	1.0	0.001260
Debbie	1.0	0.002815
Carol	1.0	0.003180
Tammy	1.0	0.003249
Susan	1.0	0.003305

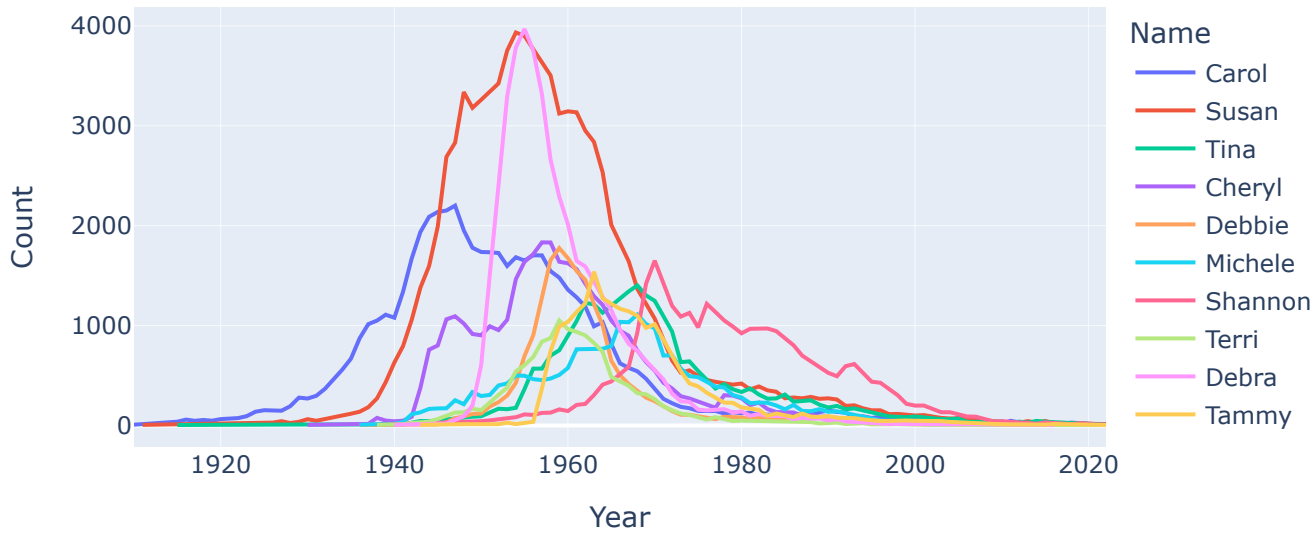
To visualize the above `DataFrame`, let's look at the line plot below:

Click to see the code



We can get the list of the top 10 names and then plot popularity with the following code:

```
top10 = rtp_table.sort_values("Count RTP").head(10).index
px.line(
    f_babynames[f_babynames["Name"].isin(top10)],
    x = "Year",
    y = "Count",
    color = "Name"
)
```



As a quick exercise, consider what code would compute the total number of babies with each name.

Click to see the code

Name	Count
Aadan	18
Aadarsh	6
Aaden	647

Name	Count
Aadhav	27
Aadhini	6

`.groupby()`, Continued

We'll work with the `elections` `DataFrame` again.

Click to see the code

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789

Raw `GroupBy` Objects

The result of `groupby` applied to a `DataFrame` is a `DataFrameGroupBy` object, **not** a `DataFrame`.

```
grouped_by_year = elections.groupby("Year")
type(grouped_by_year)
```

```
pandas.core.groupby.generic.DataFrameGroupBy
```

There are several ways to look into `DataFrameGroupBy` objects:

```
grouped_by_party = elections.groupby("Party")
grouped_by_party.groups
```

```
{'American': [22, 126], 'American Independent': [115, 119, 124], 'Anti-Masonic': [6],
'Anti-Monopoly': [38], 'Citizens': [127], 'Communist': [89], 'Constitution': [160, 164,
172], 'Constitutional Union': [24], 'Democratic': [2, 4, 8, 10, 13, 14, 17, 20, 28, 29, 34,
37, 39, 45, 47, 52, 55, 57, 64, 70, 74, 77, 81, 83, 86, 91, 94, 97, 100, 105, 108, 111,
114, 116, 118, 123, 129, 134, 137, 140, 144, 151, 158, 162, 168, 176, 178, 183],
'Democratic-Republican': [0, 1], 'Dixiecrat': [103], 'Farmer–Labor': [78], 'Free Soil':
[15, 18], 'Green': [149, 155, 156, 165, 170, 177, 181, 184], 'Greenback': [35],
'Independent': [121, 130, 143, 161, 167, 174, 185], 'Liberal Republican': [31],
'Libertarian': [125, 128, 132, 138, 139, 146, 153, 159, 163, 169, 175, 180], 'Libertarian
Party': [186], 'National Democratic': [50], 'National Republican': [3, 5], 'National
Union': [27], 'Natural Law': [148], 'New Alliance': [136], 'Northern Democratic': [26],
'Populist': [48, 61, 141], 'Progressive': [68, 82, 101, 107], 'Prohibition': [41, 44, 49,
51, 54, 59, 63, 67, 73, 75, 99], 'Reform': [150, 154], 'Republican': [21, 23, 30, 32, 33,
36, 40, 43, 46, 53, 56, 60, 65, 69, 72, 79, 80, 84, 87, 90, 96, 98, 104, 106, 109, 112,
113, 117, 120, 122, 131, 133, 135, 142, 145, 152, 157, 166, 171, 173, 179, 182],
'Socialist': [58, 62, 66, 71, 76, 85, 88, 92, 95, 102], 'Southern Democratic': [25],
'States' Rights': [110], 'Taxpayers': [147], 'Union': [93], 'Union Labor': [42], 'Whig':
[7, 9, 11, 12, 16, 19]}
```

```
grouped_by_party.get_group("Socialist")
```

	Year	Candidate	Party	Popular vote	Result	%
58	1904	Eugene V. Debs	Socialist	402810	loss	2.985897
62	1908	Eugene V. Debs	Socialist	420852	loss	2.850866
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
71	1916	Allan L. Benson	Socialist	590524	loss	3.194193
76	1920	Eugene V. Debs	Socialist	913693	loss	3.428282
85	1928	Norman Thomas	Socialist	267478	loss	0.728623
88	1932	Norman Thomas	Socialist	884885	loss	2.236211
92	1936	Norman Thomas	Socialist	187910	loss	0.412876
95	1940	Norman Thomas	Socialist	116599	loss	0.234237
102	1948	Norman Thomas	Socialist	139569	loss	0.286312

Other `GroupBy` Methods

There are many aggregation methods we can use with `.agg`. Some useful options are:

- `.mean()` documentation: creates a new `DataFrame` with the mean value of each group
- `.sum()` documentation: creates a new `DataFrame` with the sum of each group
- `.max()` documentation and `.min()` documentation: creates a new `DataFrame` with the maximum/minimum value of each group
- `.first()` documentation and `.last()` documentation: creates a new `DataFrame` with the first/last row in each group
- `.head(n)` documentation and `.tail(n)` documentation: creates a new `DataFrame` with the first/last `n` rows in each group
- `.size()` documentation: creates a new `Series` with the number of entries in each group
- `.count()` documentation: creates a new `DataFrame` with the number of entries, excluding missing values.

Let's illustrate some examples by creating a `DataFrame` called `df`.

```
df = pd.DataFrame({'letter':['A','A','B','C','C','C'],
                   'num':[1,2,3,4,np.nan,4],
                   'state':[np.nan, 'tx', 'fl', 'hi', np.nan, 'ak']})
df
```

	letter	num	state
0	A	1.0	NaN
1	A	2.0	tx
2	B	3.0	fl
3	C	4.0	hi
4	C	NaN	NaN
5	C	4.0	ak

Note the slight difference between `.size()` and `.count()`: while `.size()` returns a `Series` and counts the number of entries including the missing values, `.count()` returns a `DataFrame` and counts the number of entries in each column *excluding missing values*.

```
df.groupby("letter").size()
```

```
letter
A    2
B    1
C    3
dtype: int64
```

```
df.groupby("letter").count()
```

letter	num	state
A	2	1
B	1	1
C	2	2

You might recall that the `value_counts()` function in the previous note does something similar. It turns out `value_counts()` and `groupby.size()` are the same, except `value_counts()` sorts the resulting `Series` in descending order automatically.

```
df["letter"].value_counts()
```

```
letter
C    3
A    2
B    1
Name: count, dtype: int64
```

These (and other) aggregation functions are so common that `pandas` allows for writing shorthand. Instead of explicitly stating the use of `.agg`, we can call the function directly on the `GroupBy` object.

For example, the following are equivalent:

- `elections.groupby("Candidate").agg(mean)`
- `elections.groupby("Candidate").mean()`

There are many other methods that `pandas` supports. You can check them out on the `pandas` documentation.

Filtering by Group

Another common use for `GroupBy` objects is to filter data by group.

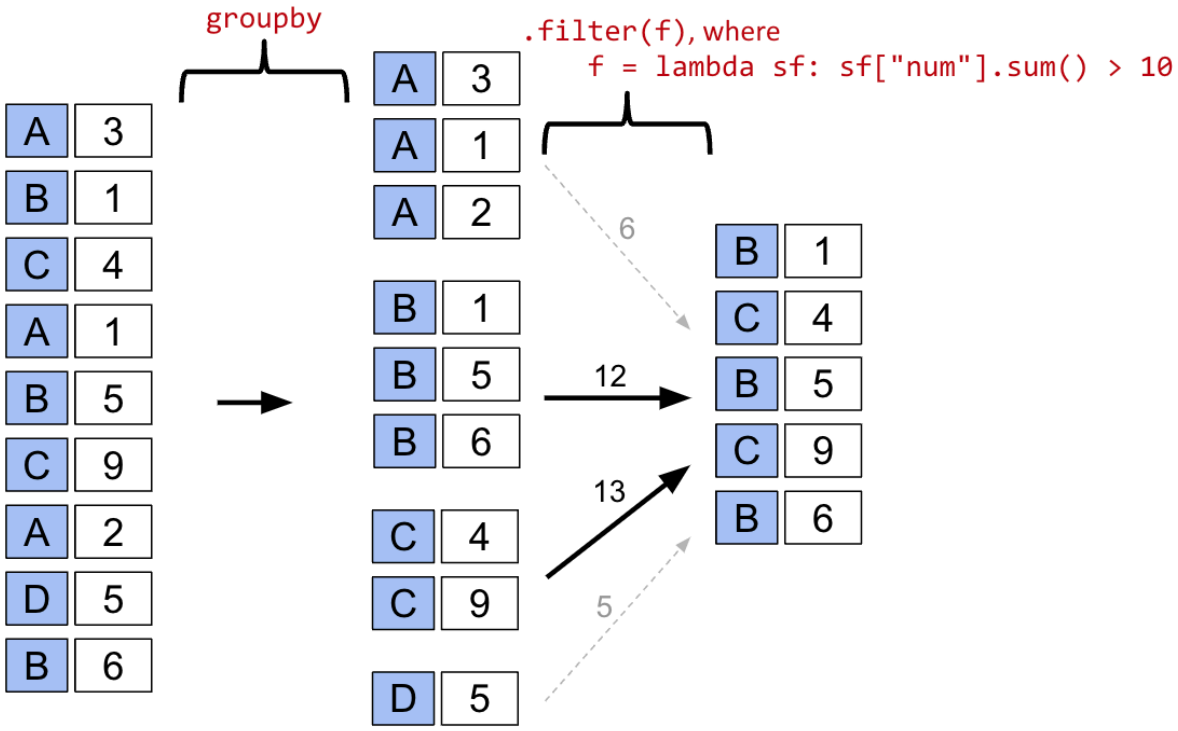
`groupby.filter` takes an argument `func`, where `func` is a function that:

- Takes a `DataFrame` object as input
- Returns a single `True` or `False`.

`groupby.filter` applies `func` to each group/sub-`DataFrame`:

- If `func` returns `True` for a group, then all rows belonging to the group are preserved.
- If `func` returns `False` for a group, then all rows belonging to that group are filtered out.

In other words, sub-`DataFrame`s that correspond to `True` are returned in the final result, whereas those with a `False` value are not. Importantly, `groupby.filter` is different from `groupby.agg` in that an *entire* sub-`DataFrame` is returned in the final `DataFrame`, not just a single row. As a result, `groupby.filter` preserves the original indices and the column we grouped on does **NOT** become the index!



`sf` refers to subframe or sub-`DataFrame` which are the "mini", grouped sub-`DataFrame`s.

To illustrate how this happens, let's go back to the `elections` dataset. Say we want to identify "tight" election years – that is, we want to find all rows that correspond to election years where all candidates in that year won a similar portion of the total vote. Specifically, let's find all rows corresponding to a year where no candidate won more than 45% of the total vote.

In other words, we want to:

- Find the years where the maximum `%` in that year is less than 45%
- Return all `DataFrame` rows that correspond to these years

For each year, we need to find the maximum `%` among *all* rows for that year. If this maximum `%` is lower than 45%, we will tell `pandas` to keep all rows corresponding to that year.

```
elections.groupby("Year").filter(lambda sf: sf["%"].max() < 45).head(9)
```

	Year	Candidate	Party	Popular vote	Result	%
23	1860	Abraham Lincoln	Republican	1855993	win	39.699408
24	1860	John Bell	Constitutional Union	590901	loss	12.639283

	Year	Candidate	Party	Popular vote	Result	%
25	1860	John C. Breckinridge	Southern Democratic	848019	loss	18.138998
26	1860	Stephen A. Douglas	Northern Democratic	1380202	loss	29.522311
66	1912	Eugene V. Debs	Socialist	901551	loss	6.004354
67	1912	Eugene W. Chafin	Prohibition	208156	loss	1.386325
68	1912	Theodore Roosevelt	Progressive	4122721	loss	27.457433
69	1912	William Taft	Republican	3486242	loss	23.218466
70	1912	Woodrow Wilson	Democratic	6296284	win	41.933422

What's going on here? In this example, we've defined our filtering function, `func`, to be `lambda sf: sf["%"].max() < 45`. This filtering function will find the maximum `"%"` value among all entries in the grouped sub-`DataFrame`, which we call `sf`. If the maximum value is less than 45, then the filter function will return `True` and all rows in that grouped sub-`DataFrame` will appear in the final output `DataFrame`.

Examine the `DataFrame` above. Notice how, in this preview of the first 9 rows, all entries from the years 1860 and 1912 appear. This means that in 1860 and 1912, no candidate in that year won more than 45% of the total vote.

You may ask: how is the `groupby.filter` procedure different to the boolean filtering we've seen previously? Boolean filtering considers *individual* rows when applying a boolean condition. For example, the code `elections[elections["%"] < 45]` will check the `"%"` value of every single row in `elections`; if it is less than 45, then that row will be kept in the output. `groupby.filter`, in contrast, applies a boolean condition *across* all rows in a group. If not all rows in that group satisfy the condition specified by the filter, the entire group will be discarded in the output.

Aggregation with `lambda` Functions

What if we wish to aggregate our `DataFrame` using a non-standard function – for example, a function of our own design? We can do so by combining `.agg` with `lambda` expressions.

Let's first consider a puzzle to jog our memory. We will attempt to find the `Candidate` from each `Party` with the highest `%` of votes.

A naive approach may be to group by the `Party` column and aggregate by the maximum.

```
elections.groupby("Party").agg("max").head(10)
```

Party	Year	Candidate	Popular vote	Result	%	First Name
American	1976	Thomas J. Anderson	873053	loss	21.554001	Thomas
American Independent	1976	Lester Maddox	9901118	loss	13.571218	Lester
Anti-Masonic	1832	William Wirt	100715	loss	7.821583	William
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838	Benjamin
Citizens	1980	Barry Commoner	233052	loss	0.270182	Barry
Communist	1932	William Z. Foster	103307	loss	0.261069	William
Constitution	2016	Michael Peroutka	203091	loss	0.152398	Michael
Constitutional Union	1860	John Bell	590901	loss	12.639283	John
Democratic	2024	Woodrow Wilson	81268924	win	61.344703	Woodrow
Democratic-Republican	1824	John Quincy Adams	151271	win	57.210122	John

This approach is clearly wrong – the `DataFrame` claims that Woodrow Wilson won the presidency in 2024.

Why is this happening? Here, the `max` aggregation function is taken over every column *independently*. Among Democrats, `max` is computing:

- The most recent `Year` a Democratic candidate ran for president (2024)
- The `Candidate` with the alphabetically "largest" name ("Woodrow Wilson")
- The `Result` with the alphabetically "largest" outcome ("win")

Instead, let's try a different approach. We will:

1. Sort the `DataFrame` so that rows are in descending order of `%`
2. Group by `Party` and select the first row of each sub-`DataFrame`

While it may seem unintuitive, sorting `elections` by descending order of `%` is extremely helpful. If we then group by `Party`, the first row of each `GroupBy` object will contain information about the `Candidate` with the highest voter `%`.

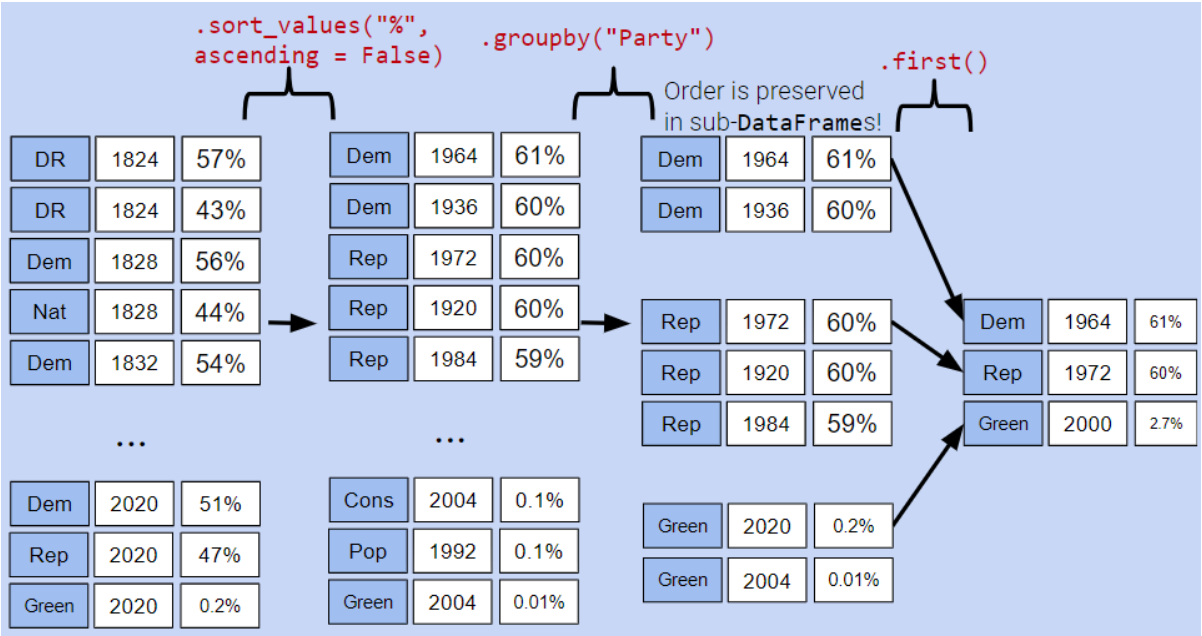
```
elections_sorted_by_percent = elections.sort_values("%", ascending=False)
elections_sorted_by_percent.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%
114	1964	Lyndon Johnson	Democratic	43127041	win	61.344703
91	1936	Franklin Roosevelt	Democratic	27752648	win	60.978107
120	1972	Richard Nixon	Republican	47168710	win	60.907806
79	1920	Warren Harding	Republican	16144093	win	60.574501
133	1984	Ronald Reagan	Republican	54455472	win	59.023326

```
elections_sorted_by_percent.groupby("Party").agg(lambda x : x.iloc[0]).head(10)

# Equivalent to the below code
# elections_sorted_by_percent.groupby("Party").agg('first').head(10)
```

Party	Year	Candidate	Popular vote	Result	%
American	1856	Millard Fillmore	873053	loss	21.554001
American Independent	1968	George Wallace	9901118	loss	13.571218
Anti-Masonic	1832	William Wirt	100715	loss	7.821583
Anti-Monopoly	1884	Benjamin Butler	134294	loss	1.335838
Citizens	1980	Barry Commoner	233052	loss	0.270182
Communist	1932	William Z. Foster	103307	loss	0.261069
Constitution	2008	Chuck Baldwin	199750	loss	0.152398
Constitutional Union	1860	John Bell	590901	loss	12.639283
Democratic	1964	Lyndon Johnson	43127041	win	61.344703
Democratic-Republican	1824	Andrew Jackson	151271	loss	57.210122

Here's an illustration of the process:



Notice how our code correctly determines that Lyndon Johnson from the Democratic Party has the highest voter `%`.

More generally, `lambda` functions are used to design custom aggregation functions that aren't pre-defined by Python. The input parameter `x` to the `lambda` function is a `GroupBy` object. Therefore, it should make sense why `lambda x : x.iloc[0]` selects the first row in each groupby object.

In fact, there's a few different ways to approach this problem. Each approach has different tradeoffs in terms of readability, performance, memory consumption, complexity, etc. We've given a few examples below.

Note: Understanding these alternative solutions is not required. They are given to demonstrate the vast number of problem-solving approaches in `pandas`.

```
# Using the idxmax function
best_per_party = elections.loc[elections.groupby('Party')['%'].idxmax()]
best_per_party.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%
22	1856	Millard Fillmore	American	873053	loss	21.554001

	Year	Candidate	Party	Popular vote	Result	%
115	1968	George Wallace	American Independent	9901118	loss	13.571218
6	1832	William Wirt	Anti-Masonic	100715	loss	7.821583
38	1884	Benjamin Butler	Anti-Monopoly	134294	loss	1.335838
127	1980	Barry Commoner	Citizens	233052	loss	0.270182

```
# Using the .drop_duplicates function
best_per_party2 = elections.sort_values('%').drop_duplicates(['Party'], keep='last')
best_per_party2.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%
148	1996	John Hagelin	Natural Law	113670	loss	0.118219
164	2008	Chuck Baldwin	Constitution	199750	loss	0.152398
110	1956	T. Coleman Andrews	States' Rights	107929	loss	0.174883
147	1996	Howard Phillips	Taxpayers	184656	loss	0.192045
136	1988	Lenora Fulani	New Alliance	217221	loss	0.237804

Aggregating Data with Pivot Tables

We know now that `.groupby` gives us the ability to group and aggregate data across our `DataFrame`. The examples above formed groups using just one column in the `DataFrame`. It's possible to group by multiple columns at once by passing in a list of column names to `.groupby`.

Let's consider the `babynames` dataset again. In this problem, we will find the total number of baby names associated with each sex for each year. To do this, we'll group by *both* the `"Year"` and `"Sex"` columns.

```
babynames.head()
```

	State	Sex	Year	Name	Count
0	CA	F	1910	Mary	295
1	CA	F	1910	Helen	239
2	CA	F	1910	Dorothy	220
3	CA	F	1910	Margaret	163
4	CA	F	1910	Frances	134

```
# Find the total number of baby names associated with each sex for each
# year in the data
babynames.groupby(["Year", "Sex"])[["Count"]].agg("sum").head(6)
```

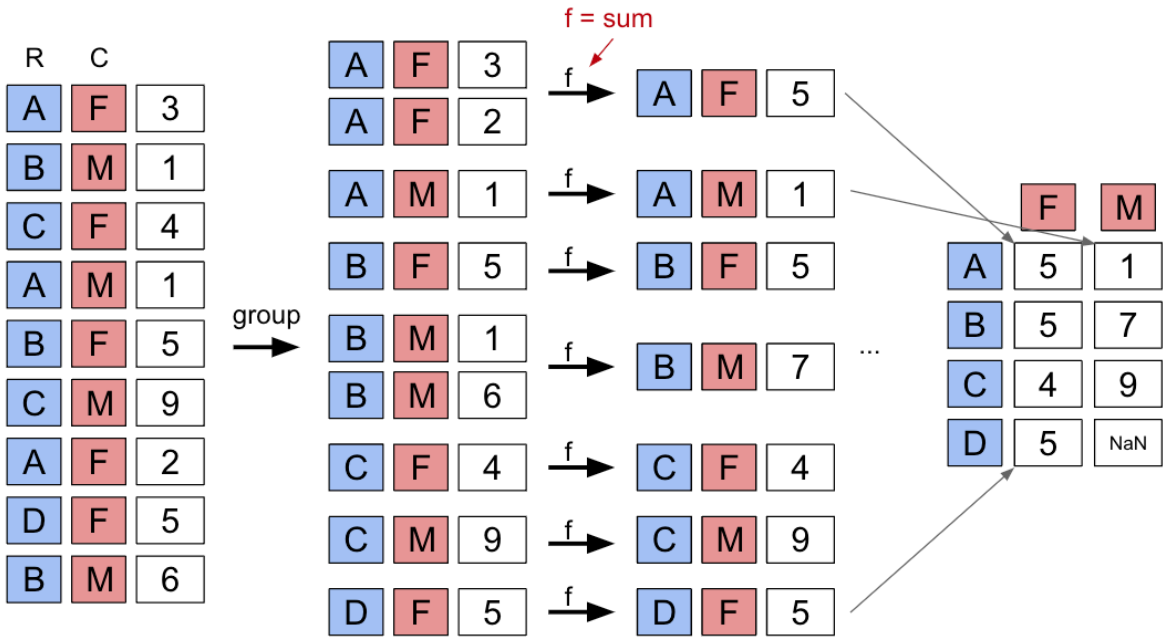
		Count
Year	**Sex**	
1910	**F**	5950
	M	3213
1911	**F**	6602
	M	3381
1912	**F**	9804
	M	8142

Notice that both `"Year"` and `"Sex"` serve as the index of the `DataFrame` (they are both rendered in bold). We've created a *multi-index* `DataFrame` where two different index values, the year and sex, are used to uniquely identify each row.

This isn't the most intuitive way of representing this data – and, because multi-indexed DataFrames have multiple dimensions in their index, they can often be difficult to use.

Another strategy to aggregate across two columns is to create a pivot table. You saw these back in Data 8. One set of values is used to create the index of the pivot table; another set is used to define the column names. The values contained in each cell of the table correspond to the aggregated data for each index-column pair.

Here's an illustration of the process:



The best way to understand pivot tables is to see one in action. Let's return to our original goal of summing the total number of names associated with each combination of year and sex. We'll call the `pandas .pivot_table` documentation method to create a new table.

```
# The `pivot_table` method is used to generate a Pandas pivot table
import numpy as np
babynames.pivot_table(
    index = "Year",
    columns = "Sex",
    values = "Count",
    aggfunc = "sum",
).head(5)
```

Sex	F	M
Year		
1910	5950	3213
1911	6602	3381
1912	9804	8142
1913	11860	10234
1914	13815	13111

Looks a lot better! Now, our `DataFrame` is structured with clear index-column combinations. Each entry in the pivot table represents the summed count of names for a given combination of `"Year"` and `"Sex"`.

Let's take a closer look at the code implemented above.

- `index = "Year"` specifies the column name in the original `DataFrame` that should be used as the index of the pivot table
- `columns = "Sex"` specifies the column name in the original `DataFrame` that should be used to generate the columns of the pivot table
- `values = "Count"` indicates what values from the original `DataFrame` should be used to populate the entry for each index-column combination
- `aggfunc = np.sum` tells `pandas` what function to use when aggregating the data specified by `values`. Here, we are summing the name counts for each pair of `"Year"` and `"Sex"`

We can even include multiple values in the index or columns of our pivot tables.

```
babynames_pivot = babynames.pivot_table(
    index="Year",     # the rows (turned into index)
    columns="Sex",    # the column values
    values=["Count", "Name"],
    aggfunc="max",      # group operation
)
babynames_pivot.head(6)
```

	Count		Name	
Sex	**F**	**M**	**F**	**M**
Year				
1910	295	237	Yvonne	William
1911	390	214	Zelma	Willis
1912	534	501	Yvonne	Woodrow
1913	584	614	Zelma	Yoshio
1914	773	769	Zelma	Yoshio
1915	998	1033	Zita	Yukio

Note that each row provides the number of girls and number of boys having that year's most common name, and also lists the alphabetically largest girl name and boy name. The counts for number of girls/boys in the resulting `DataFrame` do not correspond to the names listed. For example, in 1910, the most popular girl name is given to 295 girls, but that name was likely not Yvonne.

Joining Tables

When working on data science projects, we're unlikely to have absolutely all the data we want contained in a single `DataFrame` – a real-world data scientist needs to grapple with data coming from multiple sources. If we have access to multiple datasets with related information, we can join two or more tables into a single `DataFrame`.

To put this into practice, we'll revisit the `elections` dataset.

```
elections.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789

Say we want to understand the popularity of the names of each presidential candidate in 2022. To do this, we'll need the combined data of `babynames` *and* `elections`.

We'll start by creating a new column containing the first name of each presidential candidate. This will help us join each name in `elections` to the corresponding name data in `babynames`.

```
# This `str` operation splits each candidate's full name at each
# blank space, then takes just the candidate's first name
elections["First Name"] = elections["Candidate"].str.split().str[0]
elections.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%	First Name
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	Andrew
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	John
2	1828	Andrew Jackson	Democratic	642806	win	56.203927	Andrew
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	John
4	1832	Andrew Jackson	Democratic	702735	win	54.574789	Andrew

```
# Here, we'll only consider `babynames` data from 2022
babynames_2022 = babynames[babynames["Year"]==2022]
babynames_2022.head()
```

	State	Sex	Year	Name	Count
235835	CA	F	2022	Olivia	2178
235836	CA	F	2022	Emma	2080
235837	CA	F	2022	Camila	2046
235838	CA	F	2022	Mia	1882
235839	CA	F	2022	Sophia	1762

Now, we're ready to join the two tables. `pd.merge` (documentation) is the `pandas` method used to join `DataFrame`s together.

```
merged = pd.merge(left = elections, right = babynames_2022, \
                  left_on = "First Name", right_on = "Name")
merged.head()
# Notice that pandas automatically specifies `Year_x` and `Year_y`
# when both merged DataFrames have the same column name to avoid confusion

# Second option
# merged = elections.merge(right = babynames_2022, \
    # left_on = "First Name", right_on = "Name")
```

	Year_x	Candidate	Party	Popular vote	Result	%	First Name	State	Sex	Year_y	Nan
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122	Andrew	CA	M	2022	Andre
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878	John	CA	M	2022	Joh
2	1828	Andrew Jackson	Democratic	642806	win	56.203927	Andrew	CA	M	2022	Andre
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073	John	CA	M	2022	Joh
4	1832	Andrew Jackson	Democratic	702735	win	54.574789	Andrew	CA	M	2022	Andre

Let's take a closer look at the parameters:

- `left` and `right` parameters are used to specify the `DataFrame`s to be joined.
- `left_on` and `right_on` parameters are assigned to the string names of the columns to be used when performing the join. These two `on` parameters tell `pandas` what values should act as pairing keys to determine which rows to merge across the `DataFrame`s. We'll talk more about this idea of a pairing key next lecture.

Parting Note

Congratulations! We finally tackled `pandas`. Don't worry if you are still not feeling very comfortable with it—you will have plenty of chances to practice over the next few weeks.

Next, we will get our hands dirty with some real-world datasets and use our `pandas` knowledge to conduct some exploratory data analysis.

← Data 100: Principles and Techniques of Data Science
Pandas II

Data 100: Principles and Techniques of Data Science
Data Cleaning and EDA →

Pandas I

CONTENTS

- DataFrames and Indices
  - DataFrame
    - Using a List and Column Name(s)
    - From a Dictionary
    - From a Series
  - Indices
- DataFrame Attributes: Index, Columns, and Shape
- Slicing in DataFrames

Learning Outcomes

- Learn more key data structures: `DataFrame` and `Index`.
- Understand methods for extracting data: `.loc`, `.iloc`, and `[]`.

We continue learning tabular data abstraction, going over `DataFrame` and `Index` objects.

`DataFrame`s and Indices

`DataFrame`

```
import pandas as pd
elections = pd.read_csv("data/elections.csv")
elections
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

187 rows × 6 columns

The code above code stores our `DataFrame` object in the `elections` variable. Upon inspection, our `elections` `DataFrame` has 182 rows and 6 columns (`Year`, `Candidate`, `Party`, `Popular Vote`, `Result`, `%`). Each row represents a single record — in our example, a presidential candidate from some particular year. Each column represents a single attribute or feature of the record.

Using a List and Column Name(s)

We'll now explore creating a `DataFrame` with data of our own.

Consider the following examples. The first code cell creates a `DataFrame` with a single column `Numbers`.

```
df_list = pd.DataFrame([1, 2, 3], columns=["Numbers"])
df_list
```

	Numbers
0	1
1	2
2	3

The second creates a `DataFrame` with the columns `Numbers` and `Description`. Notice how a 2D list of values is required to initialize the second `DataFrame` — each nested list represents a single row of data.

```
df_list = pd.DataFrame([[1, "one"], [2, "two"]], columns = ["Number", "Description"])
df_list
```

	Number	Description
0	1	one
1	2	two

From a Dictionary

A third (and more common) way to create a `DataFrame` is with a dictionary. The dictionary keys represent the column names, and the dictionary values represent the column values.

Below are two ways of implementing this approach. The first is based on specifying the columns of the `DataFrame`, whereas the second is based on specifying the rows of the `DataFrame`.

```
df_dict = pd.DataFrame({
    "Fruit": ["Strawberry", "Orange"],
    "Price": [5.49, 3.99]
```

```
})
df_dict
```

	Fruit	Price
0	Strawberry	5.49
1	Orange	3.99

```
df_dict = pd.DataFrame(
    [
        {"Fruit":"Strawberry", "Price":5.49},
        {"Fruit": "Orange", "Price":3.99}
    ]
)
df_dict
```

	Fruit	Price
0	Strawberry	5.49
1	Orange	3.99

From a `Series`

Earlier, we explained how a `Series` was synonymous to a column in a `DataFrame`. It follows, then, that a `DataFrame` is equivalent to a collection of `Series`, which all share the same `Index`.

In fact, we can initialize a `DataFrame` by merging two or more `Series`. Consider the `Series` `s_a` and `s_b`.

```
# Notice how our indices, or row labels, are the same

s_a = pd.Series(["a1", "a2", "a3"], index = ["r1", "r2", "r3"])
s_b = pd.Series(["b1", "b2", "b3"], index = ["r1", "r2", "r3"])
```

We can turn individual `Series` into a `DataFrame` using two common methods (shown below):

```
pd.DataFrame(s_a)
```

	0
r1	a1
r2	a2
r3	a3

```
s_b.to_frame()
```

	0
r1	b1
r2	b2
r3	b3

To merge the two `Series` and specify their column names, we use the following syntax:

```
pd.DataFrame({
    "A-column": s_a,
    "B-column": s_b
})
```

	A-column	B-column
r1	a1	b1
r2	a2	b2
r3	a3	b3

Indices

On a more technical note, an index doesn't have to be an integer, nor does it have to be unique. For example, we can set the index of the `elections DataFrame` to be the name of presidential candidates.

```
# Creating a DataFrame from a CSV file and specifying the index column
elections = pd.read_csv("data/elections.csv", index_col = "Candidate")
elections
```

Candidate	Year	Party	Popular vote	Result	%
Andrew Jackson	1824	Democratic-Republican	151271	loss	57.210122
John Quincy Adams	1824	Democratic-Republican	113142	win	42.789878
Andrew Jackson	1828	Democratic	642806	win	56.203927
John Quincy Adams	1828	National Republican	500897	loss	43.796073
Andrew Jackson	1832	Democratic	702735	win	54.574789
...	...	...	...	...	...
Donald Trump	2024	Republican	77303568	win	49.808629
Kamala Harris	2024	Democratic	75019230	loss	48.336772
Jill Stein	2024	Green	861155	loss	0.554864
Robert Kennedy	2024	Independent	756383	loss	0.487357
Chase Oliver	2024	Libertarian Party	650130	loss	0.418895

187 rows × 5 columns

We can also select a new column and set it as the index of the `DataFrame`. For example, we can set the index of the `elections` `DataFrame` to represent the candidate's party.

```
elections.reset_index(inplace = True) # Resetting the index so we can set it again
# This sets the index to the "Party" column
elections.set_index("Party")
```

Party	Candidate	Year	Popular vote	Result	%
Democratic-Republican	Andrew Jackson	1824	151271	loss	57.210122
Democratic-Republican	John Quincy Adams	1824	113142	win	42.789878
Democratic	Andrew Jackson	1828	642806	win	56.203927
National Republican	John Quincy Adams	1828	500897	loss	43.796073

Party	Candidate	Year	Popular vote	Result	%
Democratic	Andrew Jackson	1832	702735	win	54.574789
...	...	...	...	...	...
Republican	Donald Trump	2024	77303568	win	49.808629
Democratic	Kamala Harris	2024	75019230	loss	48.336772
Green	Jill Stein	2024	861155	loss	0.554864
Independent	Robert Kennedy	2024	756383	loss	0.487357
Libertarian Party	Chase Oliver	2024	650130	loss	0.418895

187 rows × 5 columns

And, if we'd like, we can revert the index back to the default list of integers.

```
# This resets the index to be the default list of integer
elections.reset_index(inplace=True)
elections.index
```

```
RangeIndex(start=0, stop=187, step=1)
```

It is also important to note that the row labels that constitute an index don't have to be unique. While index values can be unique and numeric, acting as a row number, they can also be named and non-unique.

Here we see unique and numeric index values.

	Candidate	Party	%	Year	Result
0	Obama	Democratic	52.9	2008	win
1	McCain	Republican	45.7	2008	loss
2	Obama	Democratic	51.1	2012	win
3	Romney	Republican	47.2	2012	loss
4	Clinton	Democratic	48.2	2016	loss
5	Trump	Republican	46.1	2016	win

However, here the index values are not unique.

	Candidate	Party	%	Result
Year				
2008	Obama	Democratic	52.9	win
2008	McCain	Republican	45.7	loss
2012	Obama	Democratic	51.1	win
2012	Romney	Republican	47.2	loss
2016	Clinton	Democratic	48.2	loss
2016	Trump	Republican	46.1	win

`DataFrame` Attributes: Index, Columns, and Shape

On the other hand, column names in a `DataFrame` are almost always unique. Looking back to the `elections` dataset, it wouldn't make sense to have two columns named `"Candidate"`. Sometimes, you'll want to extract these different values, in particular, the list of row and column labels.

For index/row labels, use `DataFrame.index`:

```
elections.set_index("Party", inplace = True)
elections.index
```

```
Index(['Democratic-Republican', 'Democratic-Republican', 'Democratic',
       'National Republican', 'Democratic', 'National Republican',
       'Anti-Masonic', 'Whig', 'Democratic', 'Whig',
       ...
       'Green', 'Democratic', 'Republican', 'Libertarian', 'Green',
       'Republican', 'Democratic', 'Green', 'Independent',
       'Libertarian Party'],
      dtype='object', name='Party', length=187)
```

For column labels, use `DataFrame.columns`:

```
elections.columns
```

```
Index(['index', 'Candidate', 'Year', 'Popular vote', 'Result', '%'], dtype='object')
```

And for the shape of the `DataFrame`, we can use `DataFrame.shape` to get the number of rows followed by the number of columns:

```
elections.shape
```

```
(187, 6)
```

Slicing in `DataFrame`s

Now that we've learned more about `DataFrame`s, let's dive deeper into their capabilities.

The API (Application Programming Interface) for the `DataFrame` class is enormous. In this section, we'll discuss several methods of the `DataFrame` API that allow us to extract subsets of data.

The simplest way to manipulate a `DataFrame` is to extract a subset of rows and columns, known as **slicing**.

Common ways we may want to extract data are grabbing:

- The first or last `n` rows in the `DataFrame`.
- Data with a certain label.
- Data at a certain position.

We will do so with four primary methods of the `DataFrame` class:

1. `.head` and `.tail`
2. `.loc`
3. `.iloc`
4. `[]`

Extracting data with `.head` and `.tail`

The simplest scenario in which we want to extract data is when we simply want to select the first or last few rows of the `DataFrame`.

To extract the first `n` rows of a `DataFrame` `df`, we use the syntax `df.head(n)`.

```
elections = pd.read_csv("data/elections.csv")
```

```
# Extract the first 5 rows of the DataFrame
elections.head(5)
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789

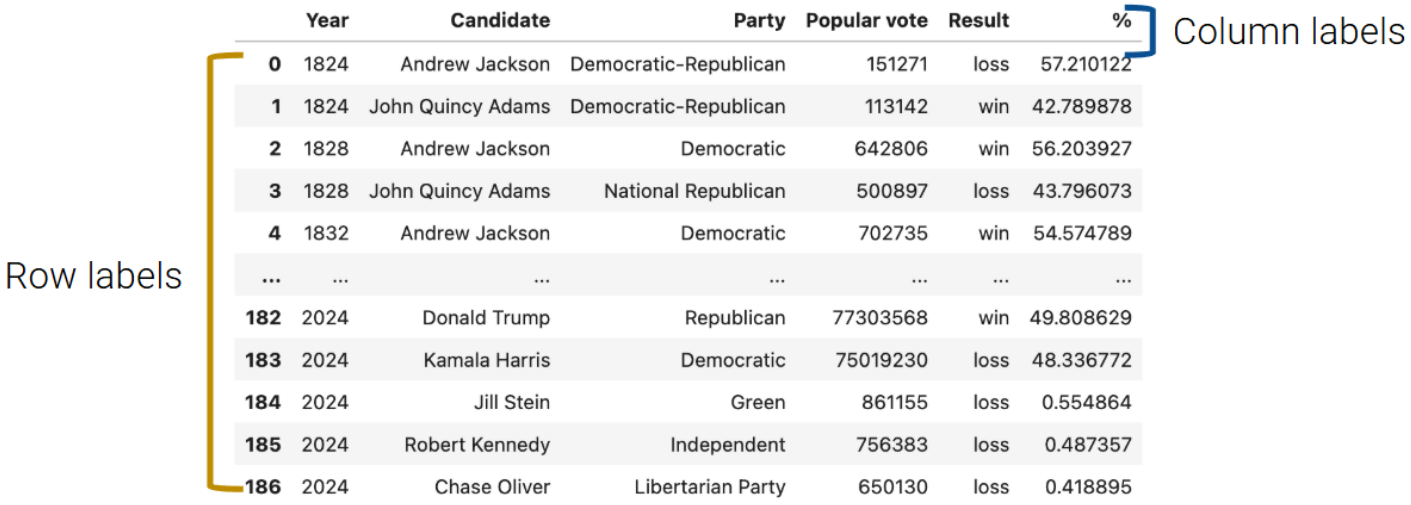
Similarly, calling `df.tail(n)` allows us to extract the last `n` rows of the `DataFrame`.

```
# Extract the last 5 rows of the DataFrame
elections.tail(5)
```

	Year	Candidate	Party	Popular vote	Result	%
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

Label-based Extraction: Indexing with `.loc`

For the more complex task of extracting data with specific column or index labels, we can use `.loc`. The `.loc` accessor allows us to specify the ***labels*** of rows and columns we wish to extract. The **labels** (commonly referred to as the **indices**) are the bold text on the far *left* of a `DataFrame`, while the **column labels** are the column names found at the *top* of a `DataFrame`.



	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073
4	1832	Andrew Jackson	Democratic	702735	win	54.574789
...	...	...	...	...	...	...
182	2024	Donald Trump	Republican	77303568	win	49.808629
183	2024	Kamala Harris	Democratic	75019230	loss	48.336772
184	2024	Jill Stein	Green	861155	loss	0.554864
185	2024	Robert Kennedy	Independent	756383	loss	0.487357
186	2024	Chase Oliver	Libertarian Party	650130	loss	0.418895

To grab data with `.loc`, we must specify the row and column label(s) where the data exists. The row labels are the first argument to the `.loc` function; the column labels are the second.

Arguments to `.loc` can be:

- A single value.
- A slice.
- A list.

For example, to select a single value, we can select the row labeled `0` and the column labeled `Candidate` from the `elections DataFrame`.

```
elections.loc[0, 'Candidate']
```

```
'Andrew Jackson'
```

Keep in mind that passing in just one argument as a single value will produce a `Series`. Below, we've extracted a subset of the `"Popular vote"` column as a `Series`.

```
elections.loc[[87, 25, 179], "Popular vote"]
```

```
87     15761254
25       848019
179    74216154
Name: Popular vote, dtype: int64
```

Note that if we pass `"Popular vote"` as a list, the output will be a `DataFrame`.

```
elections.loc[[87, 25, 179], ["Popular vote"]]
```

	Popular vote
87	15761254
25	848019
179	74216154

To select *multiple* rows and columns, we can use Python slice notation. Here, we select the rows from labels `0` to `3` and the columns from labels `"Year"` to `"Popular vote"`. Notice that unlike

Python slicing, `.loc` is *inclusive* of the right upper bound.

```
elections.loc[0:3, 'Year':'Popular vote']
```

	Year	Candidate	Party	Popular vote
0	1824	Andrew Jackson	Democratic-Republican	151271
1	1824	John Quincy Adams	Democratic-Republican	113142
2	1828	Andrew Jackson	Democratic	642806
3	1828	John Quincy Adams	National Republican	500897

Suppose that instead, we want to extract *all* column values for the first four rows in the `elections DataFrame`. The shorthand `:` is useful for this.

```
elections.loc[0:3, :]
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073

We can use the same shorthand to extract all rows.

```
elections.loc[:, ["Year", "Candidate", "Result"]]
```

	Year	Candidate	Result
0	1824	Andrew Jackson	loss
1	1824	John Quincy Adams	win
2	1828	Andrew Jackson	win

	Year	Candidate	Result
3	1828	John Quincy Adams	loss
4	1832	Andrew Jackson	win
...	...	...	...
182	2024	Donald Trump	win
183	2024	Kamala Harris	loss
184	2024	Jill Stein	loss
185	2024	Robert Kennedy	loss
186	2024	Chase Oliver	loss

187 rows × 3 columns

There are a couple of things we should note. Firstly, unlike conventional Python, `pandas` allows us to slice string values (in our example, the column labels). Secondly, slicing with `.loc` is *inclusive*. Notice how our resulting `DataFrame` includes every row and column between and including the slice labels we specified.

Equivalently, we can use a list to obtain multiple rows and columns in our `elections` `DataFrame`.

```
elections.loc[[0, 1, 2, 3], ['Year', 'Candidate', 'Party', 'Popular vote']]
```

	Year	Candidate	Party	Popular vote
0	1824	Andrew Jackson	Democratic-Republican	151271
1	1824	John Quincy Adams	Democratic-Republican	113142
2	1828	Andrew Jackson	Democratic	642806
3	1828	John Quincy Adams	National Republican	500897

Lastly, we can interchange list and slicing notation.

```
elections.loc[[0, 1, 2, 3], :]
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073

Integer-based Extraction: Indexing with `.iloc`

Slicing with `.iloc` works similarly to `.loc`. However, `.iloc` uses the *index positions* of rows and columns rather than the labels (think to yourself: **l**oc uses **l**ables; **i**loc uses **i**ndices). The arguments to the `.iloc` function also behave similarly — single values, lists, indices, and any combination of these are permitted.

Let's begin reproducing our results from above. We'll begin by selecting the first presidential candidate in our `elections DataFrame`:

```
# elections.loc[0, "Candidate"] - Previous approach
elections.iloc[0, 1]
```

Notice how the first argument to both `.loc` and `.iloc` are the same. This is because the row with a label of `0` is conveniently in the 0^{th} (equivalently, the first position) of the `elections DataFrame`. Generally, this is true of any `DataFrame` where the row labels are incremented in ascending order from 0.

And, as before, if we were to pass in only one single value argument, our result would be a `Series`.

```
elections.iloc[[1,2,3],1]
```

```
1    John Quincy Adams
2       Andrew Jackson
3    John Quincy Adams
Name: Candidate, dtype: object
```

However, when we select the first four rows and columns using `.iloc`, we notice something.

```
# elections.loc[0:3, 'Year':'Popular vote'] - Previous approach
elections.iloc[0:4, 0:4]
```

	Year	Candidate	Party	Popular vote
0	1824	Andrew Jackson	Democratic-Republican	151271
1	1824	John Quincy Adams	Democratic-Republican	113142
2	1828	Andrew Jackson	Democratic	642806
3	1828	John Quincy Adams	National Republican	500897

Slicing is no longer inclusive in `.iloc` — it's *exclusive*. In other words, the right end of a slice is not included when using `.iloc`. This is one of the subtleties of `pandas` syntax; you will get used to it with practice.

List behavior works just as expected.

```
#elections.loc[[0, 1, 2, 3], ['Year', 'Candidate', 'Party', 'Popular vote']] - Previous Ap
elections.iloc[[0, 1, 2, 3], [0, 1, 2, 3]]
```

	Year	Candidate	Party	Popular vote
0	1824	Andrew Jackson	Democratic-Republican	151271
1	1824	John Quincy Adams	Democratic-Republican	113142
2	1828	Andrew Jackson	Democratic	642806
3	1828	John Quincy Adams	National Republican	500897

And just like with `.loc`, we can use a colon with `.iloc` to extract all rows or columns.

```
elections.iloc[:, 0:3]
```

	Year	Candidate	Party
0	1824	Andrew Jackson	Democratic-Republican
1	1824	John Quincy Adams	Democratic-Republican
2	1828	Andrew Jackson	Democratic
3	1828	John Quincy Adams	National Republican
4	1832	Andrew Jackson	Democratic
...	...	...	...
182	2024	Donald Trump	Republican
183	2024	Kamala Harris	Democratic
184	2024	Jill Stein	Green
185	2024	Robert Kennedy	Independent
186	2024	Chase Oliver	Libertarian Party

187 rows × 3 columns

This discussion begs the question: when should we use `.loc` vs. `.iloc`? In most cases, `.loc` is generally safer to use. You can imagine `.iloc` may return incorrect values when applied to a dataset where the ordering of data can change. However, `.iloc` can still be useful — for example, if you are looking at a `DataFrame` of sorted movie earnings and want to get the median earnings for a given year, you can use `.iloc` to index into the middle.

Overall, it is important to remember that:

- `.loc` performances label-based extraction.
- `.iloc` performs integer-based extraction.

Context-dependent Extraction: Indexing with `[]`

The `[]` selection operator is the most baffling of all, yet the most commonly used. It only takes a single argument, which may be one of the following:

1. A slice of row numbers.
2. A list of column labels.

3. A single-column label.

That is, `[]` is *context-dependent*. Let's see some examples.

A slice of row numbers

Say we wanted the first four rows of our `elections` `DataFrame`.

```
elections[0:4]
```

	Year	Candidate	Party	Popular vote	Result	%
0	1824	Andrew Jackson	Democratic-Republican	151271	loss	57.210122
1	1824	John Quincy Adams	Democratic-Republican	113142	win	42.789878
2	1828	Andrew Jackson	Democratic	642806	win	56.203927
3	1828	John Quincy Adams	National Republican	500897	loss	43.796073

A list of column labels

Suppose we now want the first four columns.

```
elections[["Year", "Candidate", "Party", "Popular vote"]]
```

	Year	Candidate	Party	Popular vote
0	1824	Andrew Jackson	Democratic-Republican	151271
1	1824	John Quincy Adams	Democratic-Republican	113142
2	1828	Andrew Jackson	Democratic	642806
3	1828	John Quincy Adams	National Republican	500897
4	1832	Andrew Jackson	Democratic	702735
...	...	...	...	...
182	2024	Donald Trump	Republican	77303568
183	2024	Kamala Harris	Democratic	75019230
184	2024	Jill Stein	Green	861155

	Year	Candidate	Party	Popular vote
185	2024	Robert Kennedy	Independent	756383
186	2024	Chase Oliver	Libertarian Party	650130

187 rows × 4 columns

A single-column label

Lastly, `[]` allows us to extract only the `"Candidate"` column.

```
elections["Candidate"]
```

```
0         Andrew Jackson
1      John Quincy Adams
2         Andrew Jackson
3      John Quincy Adams
4         Andrew Jackson
             ...
182         Donald Trump
183        Kamala Harris
184           Jill Stein
185       Robert Kennedy
186         Chase Oliver
Name: Candidate, Length: 187, dtype: object
```

The output is a `Series`! In this course, we'll become very comfortable with `[]`, especially for selecting columns. In practice, `[]` is much more common than `.loc`, especially since it is far more concise.

Parting Note

The `pandas` library is enormous and contains many useful functions. Here is a link to its documentation. We certainly don't expect you to memorize each and every method of the library, and we will give you a reference sheet for exams.

The introductory Data 100 `pandas` lectures will provide a high-level view of the key data structures and methods that will form the foundation of your `pandas` knowledge. A goal of this course is to help you build your familiarity with the real-world programming practice of ... Googling! Answers to

your questions can be found in documentation, Stack Overflow, etc. Being able to search for, read, and implement documentation is an important life skill for any data scientist.

With that, we will move on to Pandas II!

← Data 100: Principles and Techniques of Data Science
Introduction

Data 100: Principles and Techniques of Data Science
Pandas II →

Visualization I

CONTENTS

- Visualizations in Data 8 and Data 100 (so far)
- Goals of Visualization
- An Overview of Distributions
- Variable Types Should Inform Plot Choice
- Qualitative Variables: Bar Plots
  - Plotting in Pandas
  - Plotting in Matplotlib
  - Plotting in `Seaborn`
- Distributions of Quantitative Variables
  - Histograms
    - Plotting Histograms
    - Overlaid Histograms
    - Interpreting Histograms
  - Box Plots
  - Side-by-Side Box plots
  - Violin Plots
- Kernel Density Estimation

Learning Outcomes

- Understand the theories behind effective visualizations and start to generate plots of our own with `matplotlib` and `seaborn`.
- Analyze histograms and identify the skewness, potential outliers, and the mode.
- Use `boxplot` and `violinplot` to compare two distributions.

Distributions of Quantitative Variables

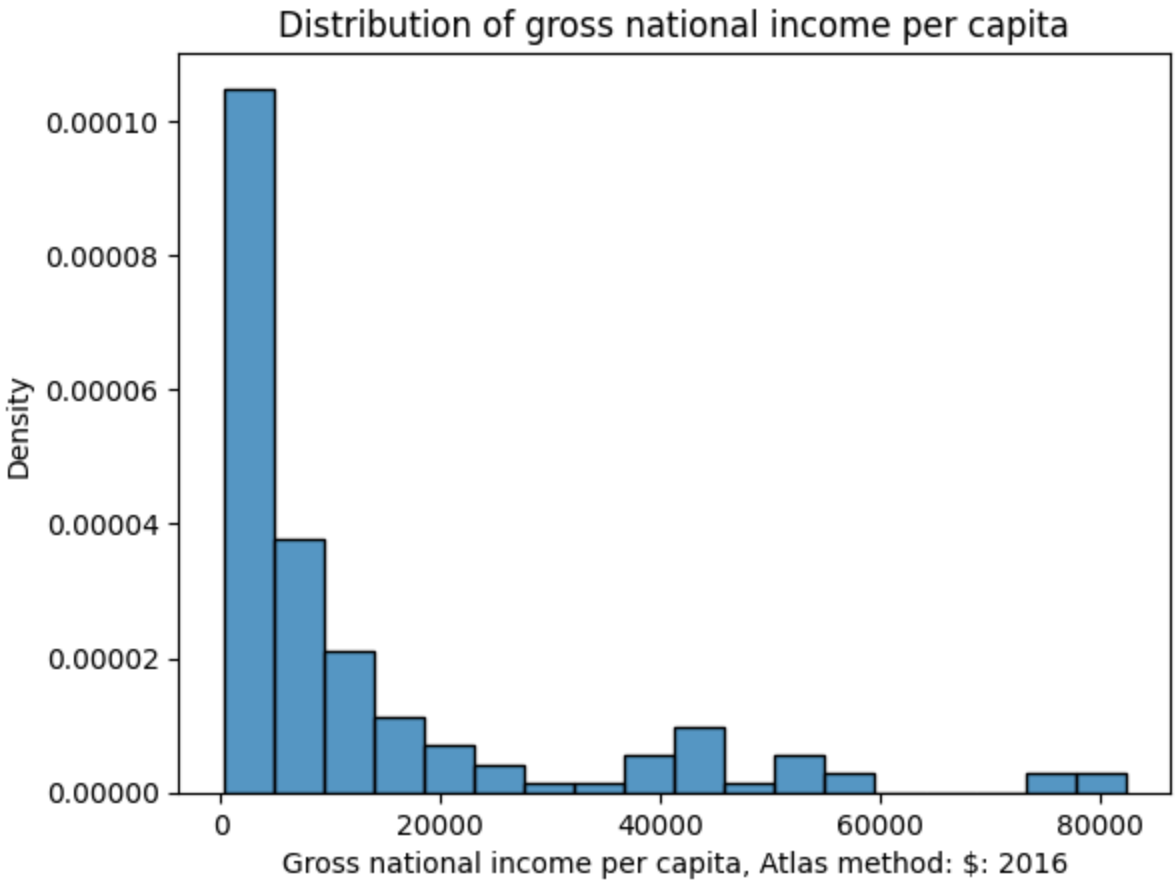
	Continent	Country	Primary completion rate: Male: % of relevant age group: 2015	Primary completion rate: Female: % of relevant age group: 2015	Lower secondary completion rate: Male: % of relevant age group: 2015	Lower secondary completion rate: Female: % of relevant age group: 2015	Youth literacy rate: Male: % of ages 15-24: 2005-14	Youth literacy rate: Female: % of ages 15-24: 2005-14	Adult literacy rate Male % ages 15 and older 2005-14
0	Africa	Algeria	106.0	105.0	68.0	85.0	96.0	92.0	83.0
1	Africa	Angola	NaN	NaN	NaN	NaN	79.0	67.0	82.0
2	Africa	Benin	83.0	73.0	50.0	37.0	55.0	31.0	41.0
3	Africa	Botswana	98.0	101.0	86.0	87.0	96.0	99.0	87.0
5	Africa	Burundi	58.0	66.0	35.0	30.0	90.0	88.0	89.0

5 rows × 47 columns

How should we define our categories for this variable? In the previous example, these were a few unique values of the `Continent` column. If we use similar logic here, our categories are the different numerical values contained in the `Gross national income per capita` column.

Under this assumption, let's plot this distribution using the `seaborn.countplot` function.

```
plot(data=wb, x="Gross national income per capita, Atlas method: $: 2016", stat="density")
e("Distribution of gross national income per capita");
```



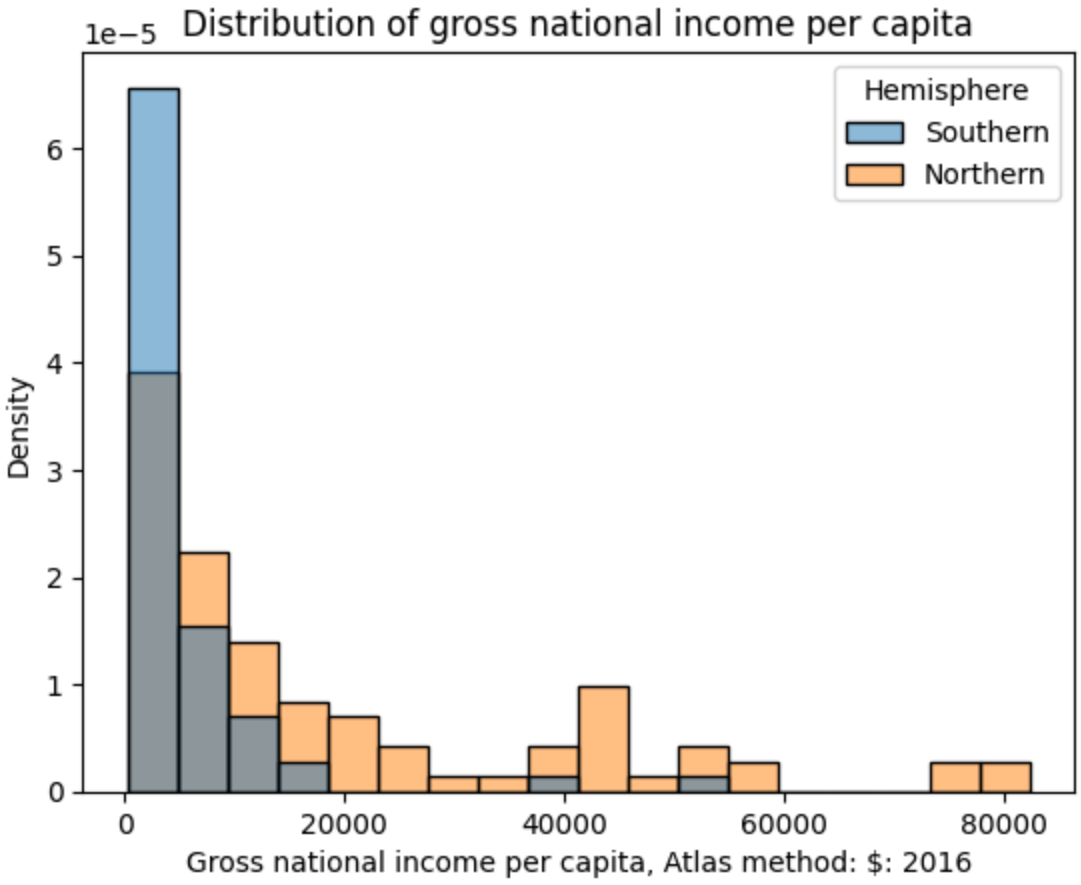
Overlaid Histograms

We can overlay histograms (or density curves) to compare **quantitative** distributions across **qualitative** categories.

The `hue` parameter of `sns.histplot` specifies the column that should be used to determine the color of each category. `hue` can be used in many `seaborn` plotting functions.

Notice that the resulting plot includes a legend describing which color corresponds to each hemisphere – a legend should always be included if color is used to encode information in a visualization!

```
sns.histplot(data=wb, x="Gross national income per capita, Atlas method: $: 2016", hue="He
plt.title("Distribution of gross national income per capita");
```

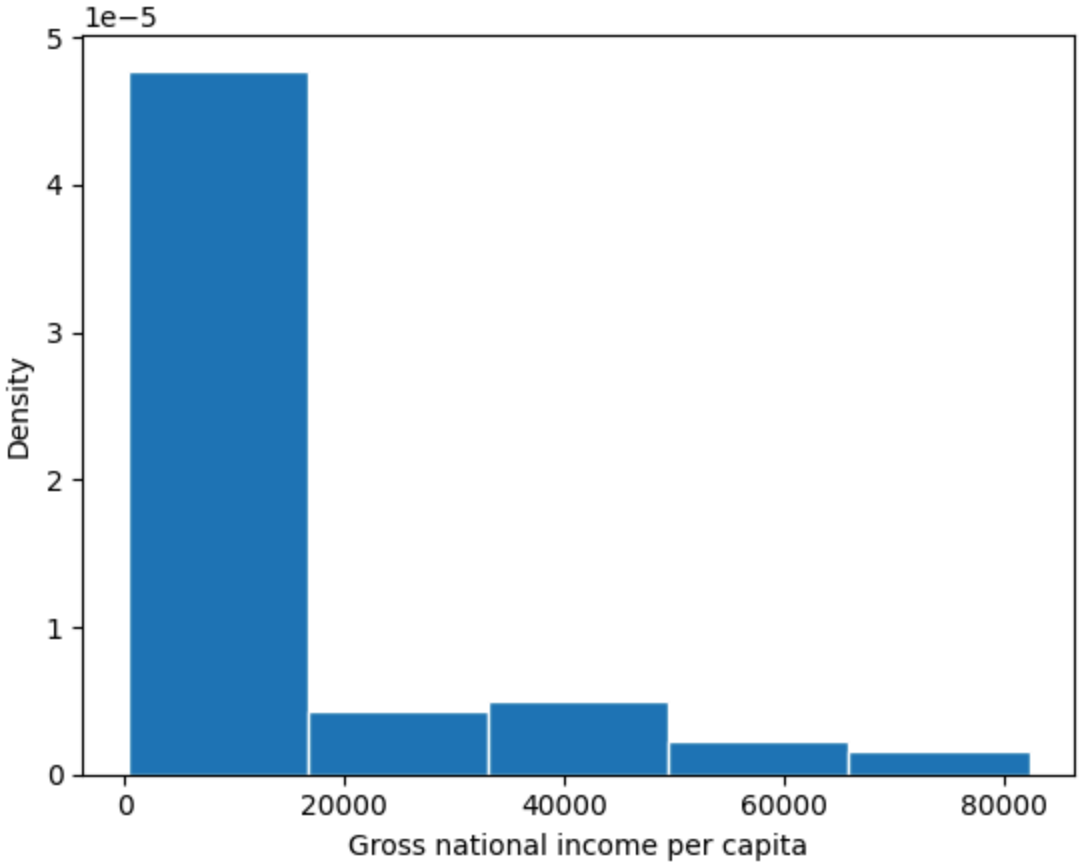


Again, each bin of a histogram is scaled such that its **area** is proportional to the **percentage** of all datapoints that it contains.

```
densities, bins, _ = plt.hist(gni, density=True, edgecolor="white", bins=5)
plt.xlabel("Gross national income per capita")
plt.ylabel("Density")
```

```
print(f"First bin has width {bins[1]-bins[0]} and height {densities[0]}")
print(f"This corresponds to {bins[1]-bins[0]} * {densities[0]} = {(bins[1]-bins[0])*densit
```

```
First bin has width 16410.0 and height 4.7741589911386953e-05
This corresponds to 16410.0 * 4.7741589911386953e-05 = 78.343949044586% of the data
```

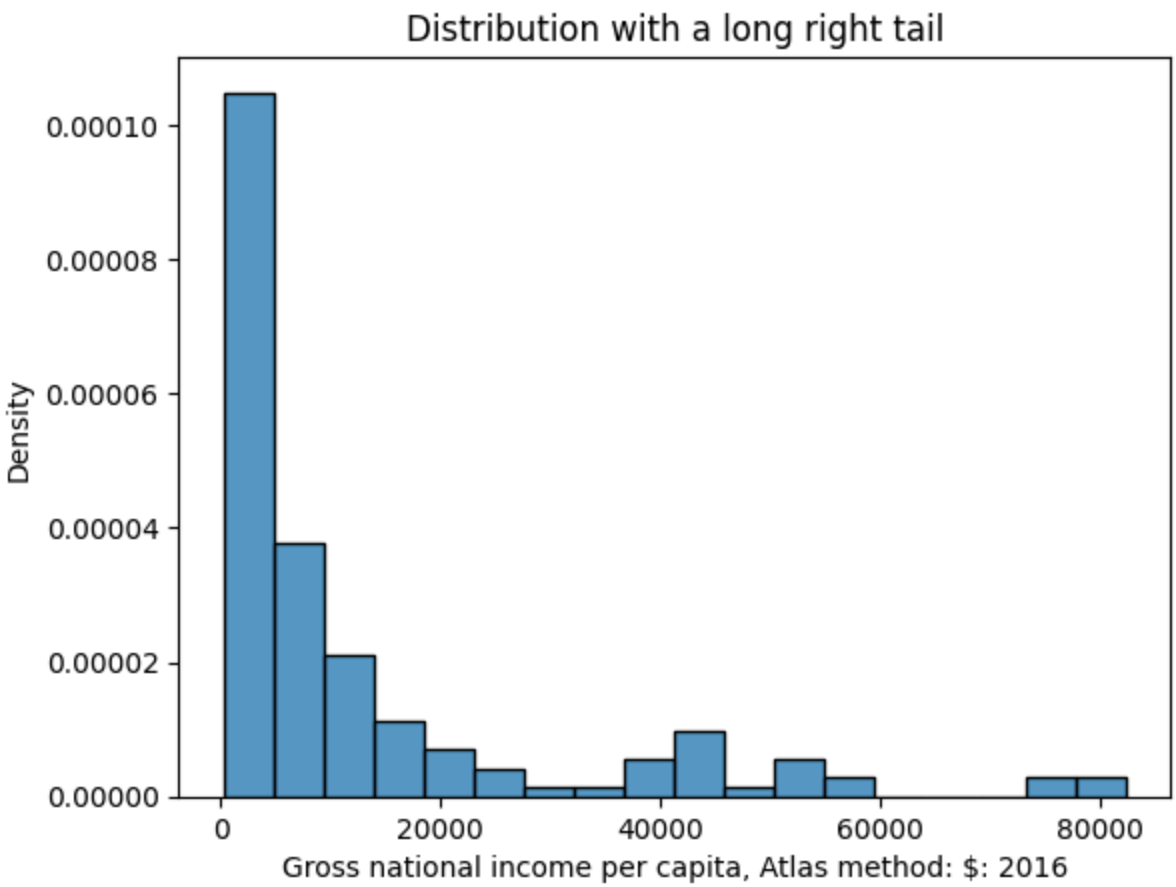


Interpreting Histograms

Histograms allow us to assess a distribution by their shape. There are a few properties of histograms we can analyze:

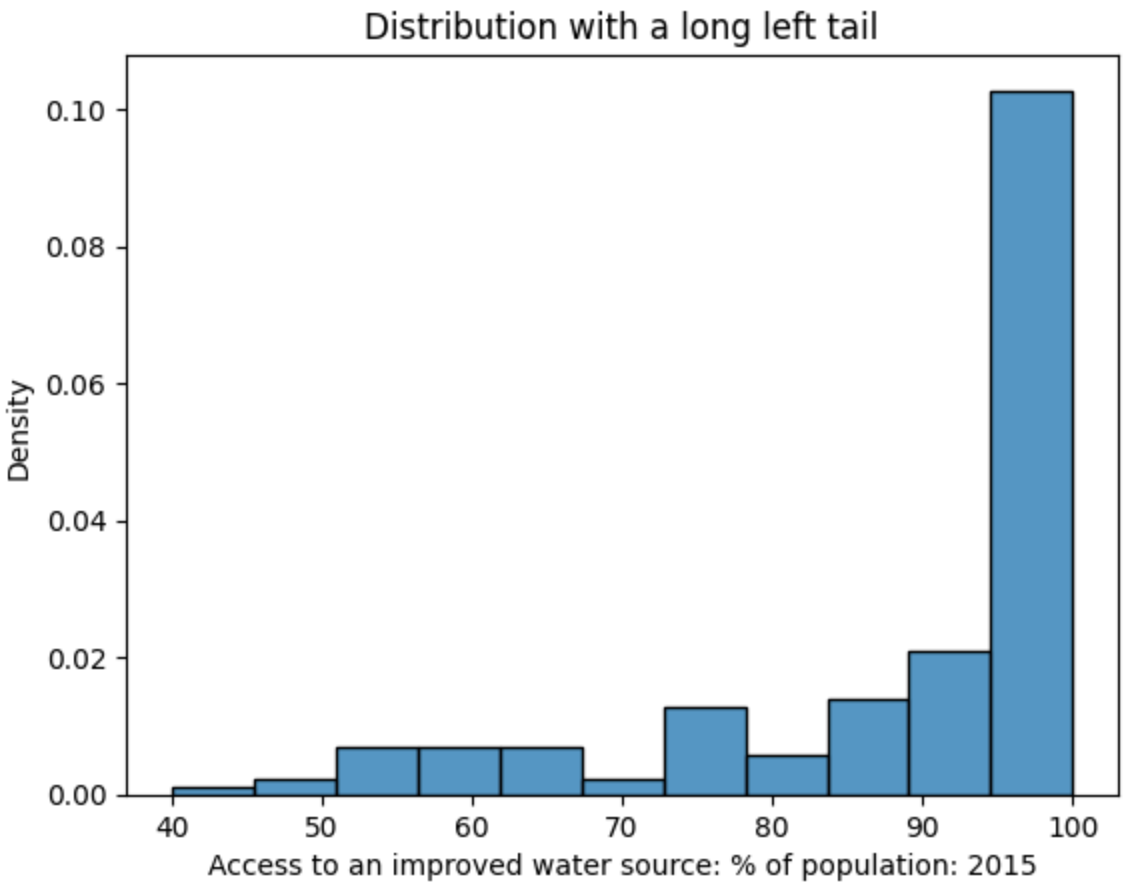
1. Skewness and Tails
    - Skewed left (negative skew) vs skewed right (positive skew)
    - Left tail vs right tail
2. Modes
    - Most commonly occurring data
3. Outliers

```
sns.histplot(data = wb, x = 'Gross national income per capita, Atlas method: $: 2016', sta
plt.title('Distribution with a long right tail');
```



- A distribution with a long left tail is **skewed left** (has **negative skew**) (such as `Access to an improved water source`). In a left-skewed distribution, the few small outliers "pull" the **mean**

```
sns.histplot(data = wb, x = 'Access to an improved water source: % of population: 2015', s
plt.title('Distribution with a long left tail');
```

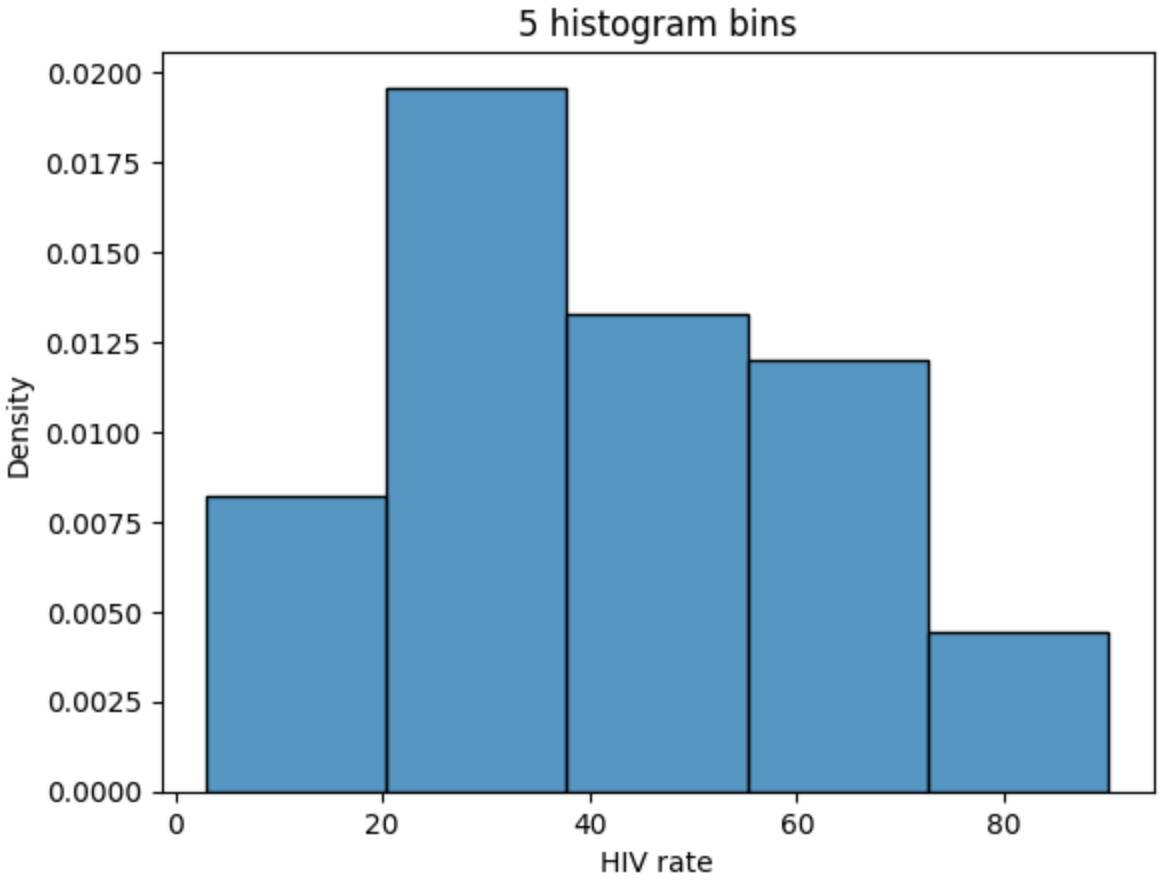


In the case where a distribution has equal-sized right and left tails, it is **symmetric**. The mean is approximately **equal** to the median. Think of mean as the balancing point of the distribution.

Modes

The **mode** is the **mo**st frequent value in a distribution. In Data 100, we describe a "mode" of a histogram as a peak in the distribution. As there may be multiple "peaks" in a distribution, we can talk about multiple modes. Often, however, it is difficult to determine what counts as its own "peak." For example, the number of peaks in the distribution of HIV rates across different countries varies depending on the number of histogram bins we plot.

```
# Rename the very long column name for convenience
wb = wb.rename(columns={'Antiretroviral therapy coverage: % of people living with HIV: 201
# With 5 bins, it seems that there is only one peak
sns.histplot(data=wb, x="HIV rate", stat="density", bins=5)
plt.title("5 histogram bins");
```




```
# With 10 bins, there seem to be two peaks

sns.histplot(data=wb, x="HIV rate", stat="density", bins=10)
plt.title("10 histogram bins");
```

Data 100: Principles and Techniques of Data Science
Regular Expressions

Data 100: Principles and Techniques of Data Science
Visualization II